

Software Testing with Finite State Machines

Gonalo Miranda

**Dissertation for the degree of Master of Science, Specialisation
Area of Software Engineering**

Supervisor: Isabel Azevedo

Porto, June 28, 2026

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore the work presented in this document is original and authored by me, having not previously been used for any other end.

The exceptions are explicitly recognised in the section “Ethical considerations” of the first chapter. This section also states how AI tools were used and for what purpose.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

ISEP, Porto, June 28, 2026

Abstract

Software testing from Finite State Machine models offers a systematic path to deriving test suites from behavioural specifications, but practical adoption is obstructed by three persistent challenges, such as generated suites can be excessively large, redundant test cases inflate execution cost without improving fault detection, and existing tools are rarely integrated into modern Continuous Integration and Continuous Delivery workflows.

This dissertation addresses these challenges through a Design Science Research methodology. An experimental prototype was built to evaluate four coverage strategies, four reduction strategies, and two prioritisation strategies in a factorial design of 32 configurations applied to five benchmark Finite State Machine models. The evaluation measured mutation score, transition coverage, Average Percentage of Faults Detected, suite size, total steps, and computational cost. The principal findings establish Transition Pair Coverage as the recommended default criterion for its consistently strong fault detection. Findings also show that reduction strategies can deliver 52-56% size savings at modest fault detection cost on small models but cause significant mutation score drops on larger models when combined with Transition Pair Coverage and reveal that Transition Addition mutants are structurally unkillable under any strategy derived from the original model, introducing a systematic bias in the aggregate mutation score.

These findings informed the design of a final artefact. A single .NET 8 executable that consolidates the validated strategy implementations into a tool ready for deployment in real-world engineering workflows. The artefact introduces a three-class mutant classification framework that excludes strategy-unreachable mutants from the mutation score denominator, yielding an adjusted mutation score that accurately reflects the fault detection capability of the generated suite. It also reports pre and post-reduction mutation scores to make the fault detection cost of reduction visible to the practitioner. Configuration is controlled entirely through a committed `toolconfig.json` file, outputs are written as structured JSON for programmatic consumption, and a reference GitHub Actions workflow demonstrates integration into an automated pipeline.

Keywords: Finite State Machine, Software Testing, Model-Based Testing, Automated Test Generation, Test Suite Optimisation, Industrial Integration

Resumo

A geração de testes a partir de Máquinas de Estados Finitos oferece uma abordagem sistemática para derivar conjuntos de testes a partir de especificações comportamentais do modelo, mas a sua prática é dificultada por três desafios persistentes. Os conjuntos de testes gerados podem ser excessivamente grandes, os casos de teste redundantes inflacionam o custo de execução sem melhorar a detecção de falhas, e as ferramentas existentes raramente se integram em fluxos de Continuous Integration and Continuous Delivery.

Esta dissertação aborda estes desafios através de uma metodologia de Design Science Research. Foi desenvolvido um protótipo experimental para avaliar quatro estratégias de cobertura, quatro estratégias de redução e duas estratégias de priorização, num total de 32 configurações aplicadas a cinco modelos de referência de Máquinas de Estados Finitos. A avaliação mediu a pontuação de mutação, a cobertura de transições, o Average Percentage of Faults Detected, o tamanho do conjunto de testes, o número total de passos e o custo computacional. Os principais resultados estabelecem a Transition Pair Coverage como critério de cobertura recomendado pela sua detecção de falhas consistentemente elevada. Também demonstram que as estratégias de redução podem reduzir o tamanho dos conjuntos de testes entre 52 e 56% em modelos de menor escala com um custo moderado na detecção de falhas, mas provocam quedas significativas na pontuação de mutação em modelos maiores quando combinadas com Transition Pair Coverage e revelam que os mutantes de Transition Addition são estruturalmente impossíveis de eliminar por qualquer estratégia derivada do modelo original.

Estes resultados informaram o design de um artefacto final. Um executável único em .NET 8 que consolida as implementações de estratégias validadas numa ferramenta pronta para utilização em ambientes de engenharia reais. O artefacto introduz um framework de classificação de mutantes em três classes que exclui os mutantes inacessíveis à estratégia do denominador da pontuação de mutação, produzindo uma pontuação de mutação ajustada que reflete com precisão a capacidade de detecção de falhas do conjunto de testes gerado. Reporta também as pontuações de mutação antes e após a redução, tornando visível ao utilizador o custo da redução na detecção de falhas. A configuração é controlada inteiramente através de um ficheiro `toolconfig.json` submetido ao repositório, os resultados são escritos como JSON estruturado para consumo, e um workflow de referência em GitHub Actions demonstra a integração em pipelines automatizados.

Acknowledgement

This dissertation is much more than the final requirement for my Master's degree. As cliché as it may sound, to me it marked another phase of my life, in which I had to learn how to prioritise relationships, work, university, and myself.

The good part about feeling like you never have time for anything is that you learn how to manage it.

I want to start by thanking my supervisor, Isabel Azevedo, for her availability and commitment to guiding me through this process.

To my friends, you were the light breeze on the harder days and the perfect distraction when motivation was lacking.

To my brother, thank you for always paving the way and for being a true example of discipline and hard work. Even though you are so many kilometres away now, you never fail to guide me through each phase of my life, and that's a true testament to what a great older brother you have become.

To my parents, you are my heroes, and, as if that were not enough, you always manage to reinvent yourselves in times of need, showing me that we always have the chance to persevere and turn things around.

This chapter is closer than ever to being closed, thanks to your support.

Thank you.

Contents

List of Figures	xv
List of Tables	xvii
List of Source Code	xix
List of Abbreviations	xxi
1 Introduction	1
1.1 Context	1
1.2 Problem	2
1.3 Objectives	3
1.4 Research Questions	3
1.5 Research Methodology	4
1.5.1 Research Process	4
1.6 Ethical Considerations	5
1.7 Document Structure	5
2 Skills Management and Project Planning	7
2.1 Skills Management	7
2.1.1 Competency Matrix	7
2.1.2 SWOT Analysis	8
2.1.3 Action Plan	9
Planning and Organisation	10
Assertiveness	10
Proactivity	10
2.2 Project Planning	10
2.2.1 Project Scope	10
2.2.2 Development Methodology	11
Research and Analysis	11
Design and Development	11
Demonstration	11
Evaluation	12
Communication	12
2.2.3 Deliverables	12
2.3 Timeline	13
2.3.1 Progress tracking	13
2.3.2 Assumptions and Restrictions	14
2.3.3 Risk Analysis	14
3 State of the Art	17

3.1	Conceptual Background	17
3.1.1	Software Testing and Model-Based Testing	17
3.1.2	Finite State Machines	18
3.1.3	Variants of FSMs	18
3.1.4	Challenges in FSM-Based Testing	19
3.1.5	Mutation Testing	19
3.2	Research Methods and Process	20
3.2.1	Data Sources	20
3.2.2	Search Strings	21
3.2.3	Screening Criteria	21
3.2.4	Data Extraction	22
3.2.5	Literature Review Questions	23
3.3	Results and Analysis	23
3.3.1	LRQ1 - Optimisation Strategies in FSM-Based Test Generation . .	23
	Redundancy in FSM-Based Test Suites and Strategies for Reduction	23
	Enhancing Fault Detection Through Test Generation	26
	Prioritisation as a Mechanism for Efficiency and Early Fault Detection	27
	Influence of Modelling Abstractions on Test Suite Quality and Scal-	
	ability	29
3.3.2	LRQ2 - Evaluation Criteria and Metrics	30
	Fault Detection Capability: Mutation Analysis	30
	FSM-Derived Coverage Criteria	31
	Efficiency and Computational Cost: Suite Size, Execution Time, and	
	Generation Overhead	33
	Prioritisation Metrics: APFD and Early Fault Detection Performance	33
	Qualitative and Practical Evaluation: Modelling Effort, Maintainabil-	
	ity, and Tooling Integration	34
3.3.3	LRQ3 - Industrial Adoption of FSM-Based Testing	34
3.3.4	LRQ4 - Limitations and Research Gaps	36
3.4	Summary and Research Gaps	36
4	From Experimental Prototype to Final Artefact Design	39
4.1	Experimental Artefact and Evaluation	39
4.1.1	Artefact Design	40
	Requirements	40
	Technology Selection	41
	Processing Stages	42
4.1.2	Strategy Implementation	43
	Test Generation Strategies	44
	Reduction Strategies	45
	Prioritisation Strategies	46
4.1.3	Implementation and Experimental Control	46
4.1.4	Experimental Setup	48
	Benchmark Selection	48
	Experimental Design	49
	Evaluation Metrics	50
4.1.5	Goal-Question-Metric Plan	52
4.1.6	Coverage Criteria and Mutation Effectiveness	53
4.1.7	Reduction Trade-offs	54

4.1.8	Prioritisation Effects	56
4.1.9	Mutation Operator Analysis	56
	Mutant Generation	56
	Fault Detection per Operator	57
4.1.10	Computational Cost	58
4.2	Design Implications for the Final Artefact	59
4.2.1	Coverage Criteria	59
4.2.2	Reduction	60
4.2.3	Prioritisation	60
4.2.4	Mutation	60
4.3	Integration into CI/CD Workflows	60
4.4	Strategy Selection	61
4.4.1	Coverage	61
4.4.2	Reduction	61
4.4.3	Prioritisation	62
4.5	Improvements Over the Experimental Artefact	62
4.5.1	Requirements	63
5	Final Artefact	65
5.1	Artefact Design	65
5.1.1	Architecture	65
	Input Validation	66
	Strategies	66
	Mutation Analysis	67
	Output Serialisation	67
5.1.2	Technology Selection	67
5.1.3	Processing Workflow	68
5.2	Mutant Classification	69
5.3	CI/CD Integration	71
5.3.1	Configuration Interface	71
5.3.2	Output Files	72
5.3.3	GitHub Actions Workflow	74
5.4	Improvements Over the Experimental Prototype	75
6	Conclusion	79
6.1	Summary	79
6.2	Research Questions	80
6.3	Limitations	81
6.4	Future Work	81
	Bibliography	83
	Appendix A Competencies Matrix	87
	Appendix B Project Charter	89
	Appendix C Gantt Chart	93
	Appendix D Systematic Mapping Study	99

Appendix E	Prototype Component Architecture	101
Appendix F	Test Suite Generation Workflow	103
Appendix G	Full Experiment Run Sequence	105
Appendix H	Mutation Testing Process	107
Appendix I	Final Artefact Component Architecture	109
Appendix J	Final Artefact — Startup and FSM Loading Sequence	111
Appendix K	Final Artefact — Full Pipeline Run Sequence	113
Appendix L	Final Artefact — Mutant Classification Sequence	115

List of Figures

1.1	Finite State Machine model example (Mealy)	2
2.1	Competency Assessment Strengths, Weaknesses, Opportunities, Threats (SWOT) Analysis	9
2.2	PREPD's WBS	12
2.3	DIMEI's WBS	13
3.1	PRISMA Diagram	22
3.2	Mutation score preserving reduction via Non-Deterministic Output-Error (NDOE) encoding.	25
3.3	Classification tree for test path P_i : three guarded transitions partitioned into boundary and above-boundary classes by Boundary Value Analysis (BVA).	26
3.4	PSMT Basic Coverage.	27
3.5	PSMT Extended Coverage.	28
3.6	Coverage-based versus weighted-transition prioritisation on a four-transition example.	29
3.7	EFSM versus Action-State representation of a PIN-entry flow with one retry.	30
3.8	State and transition coverage on a three-state FSM.	32
3.9	Transition-pair and round-trip coverage on a three-state FSM.	32
3.10	Generalised sequence coverage at depth $i = 2$ on a three-state FSM.	32
3.11	Requirement triple $R(q, x, Z)$ on a three-state example.	33
3.12	Incremental test generation with Featured Event Sequence Graphss (FESGs) [36].	35
4.1	FSM-based test generation processing stages	43
4.2	Mean mutation score per mutation operator, by Finite State Machine (FSM) group (Duplicate Removal baseline).	57
5.1	Processing workflow of the final artefact	68
A.1	Competencies Matrix	88
E.1	Component architecture of the experimental prototype	102
F.1	Sequence diagram of the test suite generation workflow	103
G.1	Sequence diagram of a full experiment run	105
H.1	Sequence diagram of the mutation testing process	108
I.1	Component architecture of the final artefact	110
J.1	Sequence diagram of the startup and FSM loading phase of the final artefact	112
K.1	Sequence diagram of a complete pipeline run of the final artefact	114

L.1	Sequence diagram of the mutant classification process of the final artefact	116
-----	---	-----

List of Tables

2.1	Filtered Competency Matrix (Importance ≥ 4)	8
2.2	Assumptions	14
2.3	Restrictions	14
2.4	Risk Identification Table	15
2.5	Risk Scoring Table	15
2.6	Risk Response Table	16
3.1	Characteristics of FSM formalisms synthesised from individual studies . . .	18
3.2	Databases used for identifying relevant publications	20
3.3	Search expressions used during the literature search	21
3.4	Search strings used during the literature search	21
3.5	Eligibility criteria	22
3.6	Comparison of redundancy reduction strategies for FSM-based test suites. .	24
4.1	Functional and non-functional requirements of the experimental artefact . .	41
4.2	Benchmark FSM models used in the evaluation	48
4.3	Experimental design: strategy dimensions and their combinations ($4 \times 4 \times 2 = 32$ configurations per FSM)	49
4.4	GQM plan for the experimental evaluation	53
4.5	Coverage criterion results for protocol FSMs, averaged across benchmark models (Duplicate Removal baseline)	53
4.6	Coverage criterion results for synthetic FSMs, averaged across benchmark models (Duplicate Removal baseline)	54
4.7	Reduction strategy trade-offs for protocol FSMs, averaged across all coverage criterion configurations	55
4.8	Reduction strategy trade-offs for synthetic FSMs, averaged across all coverage criterion configurations	55
4.9	<i>Mutation Score Preserving Reduction</i> generation time on synthetic FSMs (range across both prioritisation strategies)	59
4.10	Functional and non-functional requirements of the final artefact	64
5.1	Strategy interfaces and their concrete implementations	66
5.2	Mutant classification classes in the final artefact	70
5.3	<code>toolconfig.json</code> fields	72
5.4	Experimental prototype versus final artefact across key dimensions	76
D.1	List of included studies and their corresponding references (Part one) . . .	99
D.2	List of included studies and their corresponding references (Part two) . . .	100

List of Source Code

5.1	Structure of <i>toolconfig.json</i>	71
5.2	Schema of <i>test_suite.json</i>	72
5.3	Schema of <i>metrics_report.json</i>	73
5.4	GitHub Actions workflow for FSM test suite generation (simplified)	74

List of Abbreviations

AI	Artificial Intelligence.
APFD	Average Percentage of Faults Detected.
BFS	Breadth-First Search.
BMI	Biased Mutual Information.
BVA	Boundary Value Analysis.
CD	Continuous Delivery.
CI	Continuous Integration.
CLI	Command-Line Interface.
CSV	Comma-Separated Values.
DFS	Depth-First Search.
DIMEI	Dissertação.
DSR	Design Science Research.
DSRM	Design Science Research Methodology.
EFSM	Extended Finite State Machines.
ESG	Event Sequence Graph.
FESG	Featured Event Sequence Graphs.
FSM	Finite State Machine.
GQM	Goal-Question-Metric.
IEEE	Institute of Electrical and Electronics Engineers.
ISEP	Instituto Superior de Engenharia do Porto.
JSON	JavaScript Object Notation.
MBT	Model-Based Testing.
MCDC	Modified Condition Decision Coverage.
NDOE	Non-Deterministic Output-Error.
PREPD	Preparação para dissertação.
PRISMA	Preferred Reporting Items for Systematic reviews and Meta-Analyses.
PSMT	Prioritised State Machine Testing.

SFSM	Symbolic Finite State Machine.
SPL	Software Product Lines.
SWOT	Strengths, Weaknesses, Opportunities, Threats.
TIGER+	Test Script Generation Framework.
WBS	Work Breakdown Structure.

Chapter 1

Introduction

This chapter establishes the context and direction of the dissertation. Section 1.1 introduces the research context. Section 1.2 defines the problem. Section 1.3 states the research objectives. Section 1.4 formulates the research questions. Section 1.5 describes the adopted research methodology. Section 1.6 addresses ethical considerations. Section 1.7 outlines the overall structure of the dissertation.

1.1 Context

Software systems increasingly support complex functionalities and operate in highly dynamic environments, which makes rigorous testing essential to ensure their reliability and correctness. As modern applications evolve rapidly, the role of automated testing has become even more critical for guaranteeing behaviour across different execution scenarios [1]. In this context, Model-Based Testing (MBT) has gained relevance as an approach capable of improving test quality by relying on models that describe the expected behaviour of a system.

Among the modelling techniques used in MBT, Finite State Machine (FSM) remains one of the most widely adopted due to their ability to capture system states, transitions, and input–output relationships in a clear and structured way. FSMs have been successfully applied to model communication protocols, embedded systems, and safety-critical software, offering a foundation for deriving test suites [2]. Their formal nature enables testers to generate tests that systematically explore behavioural paths while ensuring consistency with the system specification [3].

Figure 1.1 illustrates a simple FSM representation following the Mealy machine model, in which outputs are associated with transitions rather than states. Each transition is labelled in the format *input/output*, where the value before the slash identifies the input that triggers the transition, and the value after the slash identifies the output produced as a result. For example, the transition from state S_0 (*approach*) to state S_1 (*action a*), labelled *1/3*, means that when the system is in state S_0 and receives input *1*, it moves to state S_1 and produces output *3*.

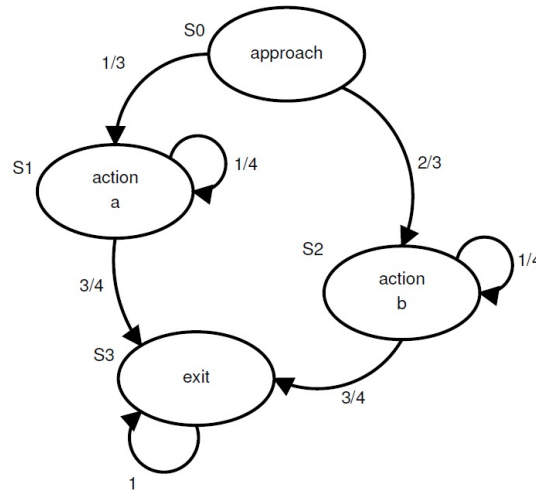


Figure 1.1: Finite State Machine model example (Mealy)

As research and industry continue to adopt MBT methodologies, FSMs play an important role in supporting the development of reliable software systems. The ability to express behavioural logic contributes to reliable validation processes and a better understanding of system functionality. These characteristics reinforce FSMs relevance across different software engineering contexts.

1.2 Problem

Finite State Machines (FSMs) provide a way to model the expected behaviour of software systems, enabling the use of model-based testing (MBT) to derive test cases. However, despite their advantages, the practical application of FSM-based testing still faces significant challenges. One of the most persistent issues is the state explosion problem, where the number of possible states and transitions grows rapidly as system complexity increases [4], leading to test suites that are excessively large and costly to execute [3].

A second difficulty concerns redundancy within the generated test suites. Many test cases derived from FSMs do not contribute to fault detection, yet they still require execution, storage, and maintenance [1]. This results in unnecessary consumption of time and resources, especially when integrated into continuous integration pipelines or automated industrial workflows.

Existing approaches for generating and optimising test suites often lack integration with real-world development processes, making it challenging for organisations to adopt FSM-based testing. Even when effective techniques exist, such as test prioritisation, variable-length path generation, or abstraction, they are not always projected for industrial automation [5].

As a consequence, software teams struggle to obtain high-coverage, non-redundant, and computationally manageable test suites from FSM models, limiting the practicality of FSM-based testing in industrial environments.

1.3 Objectives

This dissertation aims to investigate and develop an approach for generating efficient test suites from FSM models, improving the practicality and effectiveness of model-based testing in software engineering environments. The work focuses on understanding how FSMs can be used to support automated test generation, and how such processes can be optimised to balance coverage, fault detection capability, and computational efficiency.

To achieve the objective the following tasks have been carried out:

- Understand MBT, specially modelling for FSM models, as well as principles to generate test suites from these models.
- Analyse existing FSM-based test generation approaches, identifying their techniques, strengths, and limitations regarding coverage, redundancy reduction, computational cost, and industrial applicability.
- Explore strategies to mitigate identified issues, including heuristics, prioritisation methods, or model simplification techniques that can minimise computational overhead during test generation.
- Design and implement a framework to automatically generate optimised test suites from FSM models. It should also be capable of reducing redundant tests and computational costs while maintaining high fault detection potential. Simultaneously it should be compatible with CI/CD processes.
- Evaluate the proposed framework with other approaches, using criteria such as coverage, fault detection, redundancy, and computational cost.
- Investigate how the framework can be integrated into modern testing processes while making sure that it is compatible with automated pipelines and industrial workflows.

1.4 Research Questions

Research questions are central to shaping the direction of the study and ensuring alignment with these objectives. Based on the intended contributions, the following research questions have been defined:

- RQ1.** How can FSM-based automated test generation be optimised to reduce redundancy and computational cost while preserving coverage and fault detection?
- RQ2.** How can FSM-based test generation be integrated into modern software testing processes and CI/CD workflows?
- RQ3.** How should an FSM-based test generation framework be evaluated before implementation?

These research questions guide the overall dissertation. More specific literature review questions are introduced later in the State of the Art chapter in order to support the systematic analysis of existing work.

1.5 Research Methodology

This dissertation follows a Design Science Research (DSR) approach, which is a problem-solving research paradigm focused on the creation and evaluation of artefacts that address identified real-world problems [6]. DSR is particularly suitable for this work because its primary contributions take the form of two artefacts developed within the DSR design-evaluation cycle, both presented in Chapter 4. An experimental prototype, used to evaluate alternative strategies for generating and optimising FSM-based test suites and the design of the final framework, derived from the evidence produced by that evaluation. Together, these artefacts address a practical problem observed in software testing practice and provide the basis for answering the research questions defined in this dissertation.

This work adopts the Design Science Research Methodology (DSRM), which provides a structured process for conducting and presenting DSR [6]. DSRM organises the research into six interrelated activities: problem identification and motivation, definition of the objectives for a solution, design and development, demonstration, evaluation, and communication. These activities offer a guideline from the characterisation of the problem to the construction and validation of the artefact, while also supporting the communication of the results.

1.5.1 Research Process

The research process follows the six DSRM activities, each mapping to a concrete stage of the dissertation. The first two activities establish the foundation of the work. The challenges of FSM-based testing, including state explosion, redundancy, computational cost, and limited industrial applicability, are characterised in Section 1.2, supported by evidence from existing literature and from the analysis conducted in the state of the art. From this characterisation, the objectives and research questions defined in Sections 1.3 and 1.4 specify the desired properties of the solution, providing the criteria that guide all subsequent design decisions.

The design, development, and demonstration activities produce and validate the two artefacts presented in Chapter 4. The design is grounded in the conceptual background of Chapter 3, Section 3.1 and in the analysis of existing approaches in Section 3.3. The prototype is developed iteratively, with its architecture, optimisation strategies, and integration mechanisms refined throughout the process. Demonstration is then achieved by applying the prototype to benchmark FSM models. This activity illustrates the feasibility and applicability of the artefact in addressing the identified problem.

The evaluation phase consists of a controlled experimental study in which different strategy configurations are applied across the same benchmark models and assessed using metrics such as: coverage, fault detection capability, Average Percentage of Faults Detected (APFD), test suite size, and computational cost. The findings from this study directly inform the design of the final artefact. Finally, communication is realised through this dissertation and through the public availability of source code, experimental configurations, and models in a version-controlled repository [7], ensuring transparency, reproducibility, and accessibility of the research outcomes.

1.6 Ethical Considerations

The research conducted in this dissertation does not involve human participants, personal data, or sensitive organisational information. As such, the ethical risks traditionally associated with empirical studies involving human subjects are not applicable. The study is based exclusively on academic literature, benchmark models, and technical documentation.

All sources used throughout the dissertation are properly acknowledged, ensuring respect for intellectual property rights and compliance with academic integrity principles. No proprietary, confidential, or industrial artefacts are used.

Given that this work involves the design and implementation of a software artefact to support software testing activities, attention is given to applicable legal and regulatory considerations. In particular, although the developed prototype is not intended for production use, the work is aligned with the principles underlying the European Cyber Resilience Act, namely the promotion of secure-by-design practices, software quality, and risk awareness in software development. The proposed framework is designed to improve software reliability through systematic testing and does not introduce mechanisms that could pose security or misuse risks.

Reproducibility and transparency are considered essential ethical principles in this research. Source code, experimental configurations, and models are maintained in a version-controlled repository [7] to ensure traceability of changes, support validation of results, and enable future replication or extension of the work. The repository contains the experimental prototype, the final artefact, the benchmark FSM models and a reference Continuous Integration (CI) workflow.

Ethical responsibility regarding researcher competence is also addressed. In accordance with the Institute of Electrical and Electronics Engineers (IEEE) Code of Ethics, which states that professionals should ensure they are qualified for the work they undertake through an appropriate combination of education, training, and experience, this dissertation includes a structured skills management process [8]. The competency analysis, SWOT assessment, and action plan presented in Chapter 2 explicitly identify required competencies and define measures to mitigate gaps, ensuring that the work is conducted responsibly and within the author's technical and methodological capabilities.

Particular attention is also given to the responsible use of artificial intelligence tools in the development of this dissertation. Such tools may be used to support activities including literature exploration, language refinement, code assistance, and organisation of ideas. Any content produced with Artificial Intelligence (AI) support is critically reviewed by the author and validated against relevant academic sources, and AI-generated content is never incorporated into the work without prior verification. The final responsibility for all technical decisions, interpretations, and written content remains with the author.

Overall, the research adheres to good scientific practice by ensuring transparency in methodology, objectivity in reporting results, legal and ethical awareness in software development, and responsible management of competencies throughout the project.

1.7 Document Structure

This dissertation is organised into six main chapters, each addressing a specific component of the research, followed by supporting appendices.

Chapter 1 - Introduction: Presents the motivation and context of the dissertation by defining the problem under investigation, the objectives and research questions, as well as describing the adopted research approach, and ethical considerations.

Chapter 2 - Skills Management and Project Planning: Identifies the technical, methodological, and personal competencies relevant to the successful development of the dissertation, and an action plan defining measures to improve areas where the student is not comfortable or lacking skill. It also identifies all steps taken during the planning of the dissertation, such as project scope, timeline, progress tracking methodology used, assumptions and restrictions, and finally a risk analysis.

Chapter 3 - State of the Art: This chapter introduces a conceptual background section along with details of the systematic mapping review methodology, including data sources, search strategy, screening criteria, and the PRISMA-based study selection process. It also provides a review of the literature related to the subject of the dissertation.

Chapter 4 - From Experimental Prototype to Final Artefact Design: Presents the experimental artefact developed in this dissertation and explains how it supports the empirical evaluation of FSM-based test generation strategies. It describes the prototype in the context of the adopted DSR methodology, analyses the main experimental findings, and translates those findings into the design implications and strategy choices that should guide the final artefact.

Chapter 5 - Final Artefact: Presents the final artefact, a single executable tool that consolidates the design implications from Chapter 4 into a form suitable for deployment in real-world engineering workflows. It covers the component architecture, technology selection, processing workflow, mutant classification framework, and CI/Continuous Delivery (CD) integration, and summarises the improvements the final artefact introduces over the experimental prototype.

Chapter 6 - Conclusion: Closes the dissertation by summarising the work conducted, revisiting and answering the three research questions, identifying the limitations of the approach, and outlining directions for future work.

Appendices: The appendices contain supporting materials that complement the main chapters and provide additional detail without interrupting the flow of the dissertation. These materials include artefacts produced during the skills management and project planning phases, supplementary information related to the systematic mapping study, and architecture and sequence diagrams for both the experimental prototype and the final artefact.

Chapter 2

Skills Management and Project Planning

This chapter presents the skills assessment and project organisation that underpin the dissertation. Section 2.1 analyses the competencies required to successfully develop the proposed work, examining current technical and methodological skills and identifying areas for development. Section 2.2 outlines the planning framework that guides the execution of the dissertation, covering scope, methodology, assumptions, risk analysis, and progress tracking. Section 2.3 presents the timeline that structures the research activities over the project period.

2.1 Skills Management

2.1.1 Competency Matrix

To assess the competencies required for the successful development of this dissertation, a competency matrix was completed and is presented in Appendix A. This matrix evaluates a set of transversal and technical competencies according to their relevance to the dissertation, and the student's self-evaluation.

The analysis of these competencies supports the identification of strengths, areas requiring improvement, and the foundation for the Strengths, Weaknesses, Opportunities, Threats (SWOT) analysis and action plan presented in the subsequent sections. Table 2.1 reflects the most relevant competencies for the project.

Competency	Importance	Self-evaluation
Written communication	5	4
Critical thinking	5	4
Resilience	5	4
Time management	5	4
Problem analysis and resolution	5	4
Planning and organisation	5	3
Oral communication	4	4
Stress management	4	5
Active listening	4	4
Interpersonal relationships	4	4
Leadership	4	4
Empathy	4	5
Assertiveness	4	3
Proactivity	4	2
Adaptability and flexibility	4	4

Table 2.1: Filtered Competency Matrix (Importance ≥ 4)

By looking at the competency matrix, it's visible that "Proactivity", "Assertiveness" and "Planning and organisation" are the skills that require further development considering their importance to the project success. On another note "Written Communication", given its central role in producing a dissertation, is equally critical and must be continuously developed to ensure the clarity and academic quality of the written output.

2.1.2 SWOT Analysis

This section presents a SWOT analysis based on the competency assessment previously conducted. The aim is to identify the student's internal strengths and weaknesses, as well as external opportunities and threats that may influence the successful development of the dissertation. This analysis supports the definition of targeted actions, later detailed in the section 2.1.3.

Figure 2.1 illustrates the conducted SWOT analysis.

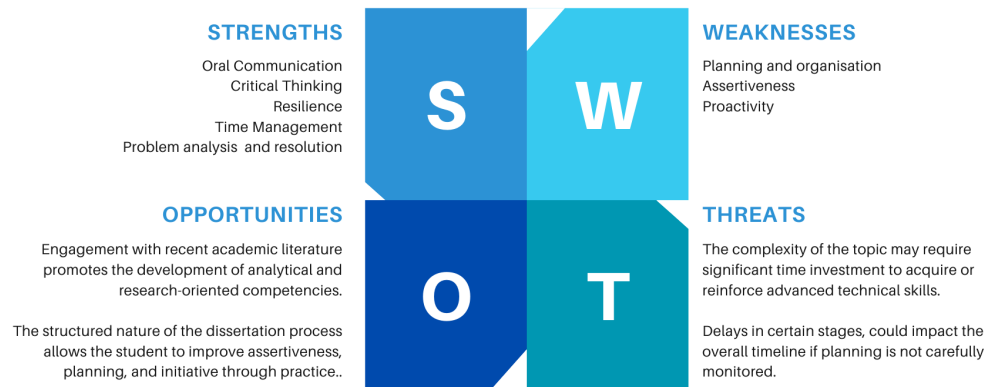


Figure 2.1: Competency Assessment SWOT Analysis

The opportunities identified in this analysis arise from the technical depth and ongoing evolution of the research area, offering an environment for academic growth. Students who are particularly strong in the following areas are best positioned to benefit from these opportunities:

- **Critical Thinking:** To analyse academic literature and justify design decisions within the proposed framework.
- **Problem Analysis and Resolution:** To address challenges associated with FSM modelling, optimisation and experimental design.
- **Resilience:** To maintain motivation and focus when exploring complex concepts.

On the other hand, the identified threats expose the need for consistent progress across the project's development stages. These threats pose higher risks for individuals lacking the following competencies:

- **Planning and Organisation:** To divide the dissertation into manageable tasks and anticipate workload peaks.
- **Assertiveness and Proactivity:** To seek clarification, request support when needed and take initiative in defining experimental strategies.

2.1.3 Action Plan

The *SWOT Analysis* highlights several competencies that will benefit from development throughout the dissertation. The action plan presented in this section defines measures to strengthen these competencies and ensure the successful execution of the project.

The proposed actions focus on improving *Planning and Organisation*, *Assertiveness*, and *Proactivity*, competencies that directly influence the progress of a research-oriented dissertation and which the student does not possess or is lacking.

Planning and Organisation

To address the challenges associated with structuring a complex project, a detailed work plan will be required.

Action: Establish weekly or monthly goals, break tasks into manageable units, and use project-management tools (such as *Kanban* boards) to monitor progress. Documenting decisions and tracking results will also support consistency across the work.

Assertiveness

Given the technical depth of the topic, situations may arise where clarification, or guidance become necessary.

Action: To strengthen assertiveness, the student will adopt a proactive communication strategy with the supervisor, seeking feedback regularly and addressing uncertainties as soon as they emerge. Preparing meeting agendas when adequate, formulating questions in advance, and summarising discussions will reinforce confidence in interactions.

Proactivity

The research should unfold through experimentation and iteratively. Proactivity is essential for a well managed process.

Action: Actively seek resources such as academic articles, technical documentation, rather than episodically to allow for informed decision making during the design and evaluation of the framework. To further encourage proactivity, the student will identify small, achievable tasks for moments of low energy or limited time, ensuring that progress continues even when motivation fluctuates.

2.2 Project Planning

The planning provides a roadmap for the development of the dissertation, establishing the main activities, milestones, and expected deliverables.

A project charter with the summary of key information addressed in this section is available in Appendix B.

2.2.1 Project Scope

The scope of this dissertation encompasses the study, design, implementation, and evaluation of an approach for optimised automated test suite generation based on Finite State Machines (FSMs). The project focuses on addressing known challenges in FSM-based testing, specifically redundancy, computational cost, and the integration of generated test suites into software testing workflows.

The goal is to develop a practical framework capable of producing efficient test suites while maintaining relevant behavioural coverage and supporting their incorporation into automated pipelines.

The project lays out in four stages:

- **Theoretical Research:** Review of the literature regarding FSMs, MBT, strategies for test suite optimisation, and empirical evaluation methods and criteria. The foundation for designing the test generation approach is defined at this stage.
- **Framework Design and Prototyping:** The definition of an architectural model and the development of a prototype capable of generating test suites from FSM specifications.
- **Experimental Evaluation:** Benchmark FSMs and test-generation methods identified in the first stage will be used to evaluate the proposed framework. The experiments will focus on assessing fault detection potential, coverage, redundancy, and computational cost.
- **Documentation and Analysis:** All findings, design decisions, and experimental results will be documented systematically. This includes critical analysis of the advantages, limitations, and applicability of the proposed approach, supported by evidence collected throughout the evaluation process.

The dissertation is limited to the context of software systems that can be modelled using finite state machines.

2.2.2 Development Methodology

The development of this dissertation follows the Design Science Research methodology previously introduced in Chapter 1. Consequently, the work plan is organised around the phases of research and analysis, design and development, demonstration, evaluation, and communication, which structure the project timeline, the Work Breakdown Structure (WBS), and the deliverables produced at each stage.

Research and Analysis

In the project plan, this phase covers the initial portion of the timeline, dedicated to literature review and to the analysis of existing FSM-based testing approaches. The corresponding tasks are grouped under the early stages of the *Preparação para dissertação* (PREPD) WBS and produce the state of the art and the problem characterisation as the main deliverables.

Design and Development

This phase corresponds to the conceptual design of the framework and its prototype implementation. In the *Dissertação* (DIMEI) WBS it aligns with the Design and Build stages, resulting in the framework's architecture definition and the working software artefact as deliverables.

Demonstration

Demonstration is scheduled after the main implementation milestones, when the prototype is applied to benchmark FSM models. It appears in the DIMEI WBS as part of the Evaluate stage and produces the demonstration scenarios used to show the framework in operation.

Evaluation

The evaluation phase occupies the later stages of the timeline and corresponds to a controlled experimental study. In the DIMEI WBS it is grouped with the Evaluate and Conclusion stages, producing the experimental results and the analysis that supports the answers to the research questions.

Communication

Communication spans the full duration of the project and concentrates at its final milestones. It is reflected across the PREPD WBS in the writing and delivery of the initial dissertation report and in the DIMEI WBS through the public availability of source code, models, and experimental configurations as final deliverables [7].

2.2.3 Deliverables

The execution of this dissertation will result in several deliverables that reflect the different stages of the project.

The project deliverables are organised and presented using WBS diagrams that describe phases, corresponding deliverables and tasks required to produce them.

PREPD's WBS covers the artefacts required for the partial delivery of the dissertation, and DIMEI's WBS, which maps the Design–Implement–Evaluate–Conclude lifecycle followed during the development and experimental evaluation.

Figure 2.2 presents PREPD's WBS, showing how the activities and deliverables are organised.

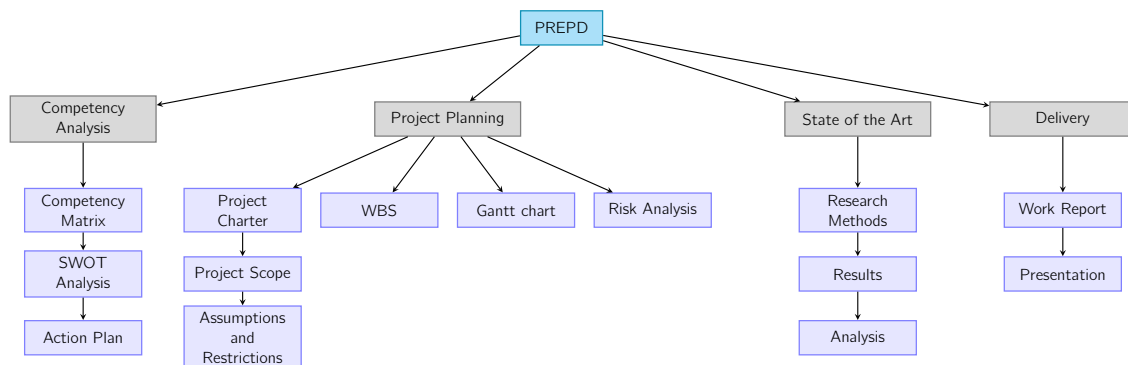


Figure 2.2: PREPD's WBS

Figure 2.3 presents DIMEI's WBS, highlighting the lifecycle followed for the design, implementation, evaluation, and conclusion of the dissertation work.

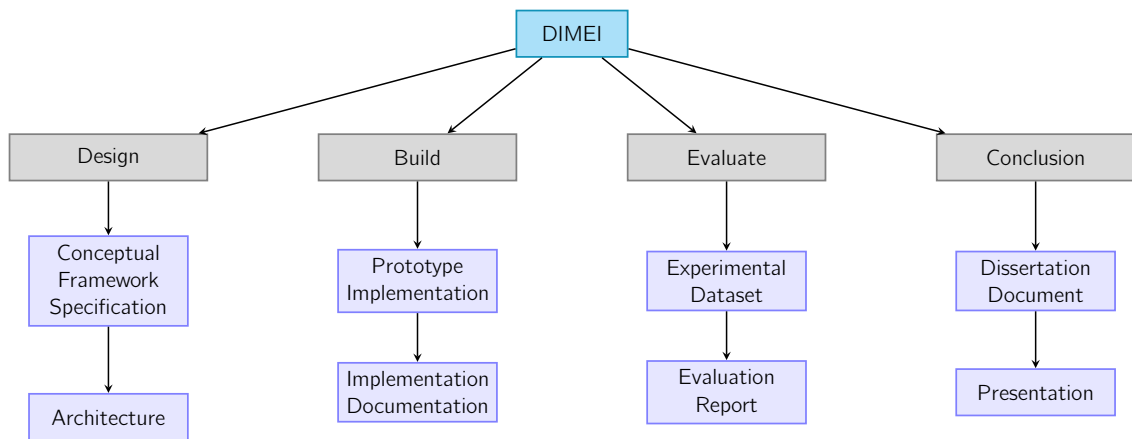


Figure 2.3: DIMEI's WBS

2.3 Timeline

The execution of this dissertation is organised into two major phases:

- **PREPD:** initial planning, problem analysis, and research preparation
- **DIMEI:** core lifecycle encompassing artefact design and development, demonstration, evaluation, and conclusion

Each stage of the work should be completed in sequence. The timeline has been structured to guarantee this is the case throughout the lifecycle of the project. It serves as a reference framework for monitoring progress throughout the project, ensuring that tasks remain aligned with deadlines and that the workload is distributed appropriately over the dissertation period.

It integrates the milestones defined in the Project Charter, the work breakdown structures (WBS) for PREPD and DIMEI, and the *Gantt chart* included in Appendix C.

2.3.1 Progress tracking

Throughout the project, progress will be monitored through a combination of predefined milestones, periodic reviews and continuous updates to the project's planning artefacts.

The primary tool for tracking progress is the *Gantt chart* included in Appendix C, which outlines the duration, dependencies and expected completion dates of all major tasks in both the PREPD and DIMEI phases. This chart will be reviewed regularly to verify whether activities are advancing according to plan and to adjust timelines when necessary.

Progress will also be monitored through ad-hoc meetings with the supervisor, during which completed work, ongoing tasks and upcoming challenges will be discussed. These meetings will serve as checkpoints to validate the direction of the project.

Additionally, the WBS diagrams for PREPD and DIMEI provide a hierarchical view of all deliverables and tasks, enabling systematic verification of completed items.

Finally, a personal progress log will be maintained to record key decisions, intermediate results, encountered obstacles and planned next steps.

2.3.2 Assumptions and Restrictions

The execution of this dissertation is subject to a number of assumptions and restrictions that influence the scope, feasibility and direction of the work.

Table 2.2 presents the assumptions considered during the planning of the dissertation.

ID	Description
1	Availability of literature and benchmark models.
2	Time allocated for each planned step is adequate.

Table 2.2: Assumptions

It is assumed that sufficient available research, technical documentation and benchmark FSM models exist to support the design, demonstration, and evaluation of the proposed approach, and that the time allocation is accurate for each step or phase of the dissertation.

This work is restricted by scope limitations on industrial integration, because the project does not include the deployment of the solution in real industrial pipelines. This means that its validation will rely on controlled experiments rather than industrial case studies. The dissertation will also not focus on producing a production-ready tool, rather demonstrating its feasibility and effectiveness. Table 2.3 presents the identified restrictions that delimit the scope and expected fidelity of the work.

ID	Description
1	Scope limitations on industrial integration.
2	Prototype Fidelity.

Table 2.3: Restrictions

2.3.3 Risk Analysis

The development of this dissertation involves several risks that may affect the project's schedule, scope or quality of outcomes. Identifying these risks and defining mitigation strategies is essential to ensure that potential issues are addressed and that the project remains aligned with its objectives.

The technical risks relate mainly to the complexity of FSM-based optimisation, and the availability of suitable benchmark models. The viability of the evaluation may be affected by these risks.

A risk impacting time and the overall management of the project is considered an organisational risk. If the conclusion of an early stage is delayed, it will consequently compromise the following stages, possibly failing to comply with the expected deadlines.

The personal risks reflect challenges linked to motivation, assertiveness and engagement. Due to the demanding and research intensive nature of the dissertation, these factors may influence productivity and responsiveness to obstacles encountered during the project.

To provide a structured view of these risks, they have been divided into Tables 2.4, 2.5 and 2.6. Table 2.4 identifies the main risks, their causes, and their expected effects on the project.

Risk ID	Description	Cause	Effect
TR1	Complexity of FSM optimisation techniques	Difficulty in understanding or implementing optimisation strategies	Delays in design and development of the artefact
TR2	Limited availability of benchmark FSM models	Lack of diverse, published models or datasets	Restricted evaluation
OR1	Schedule slippage	Underestimation of required time for tasks	Delays of later project phases
PR1	Difficulty maintaining motivation over time	Technical complexity and iterative workload	Reduced productivity and inconsistent progress
PR2	Limited assertiveness when facing uncertainty	Hesitation to request help or feedback promptly	Prolonged blockers and unnecessary delays

Table 2.4: Risk Identification Table

Table 2.5 presents the probability-impact assessment used to prioritise the identified risks.

Risk ID	Probability (1–5)	Impact (1–5)	PI Score
TR1	3	3	9
TR2	2	3	6
OR1	3	4	12
PR1	3	3	9
PR2	1	3	3

Table 2.5: Risk Scoring Table

Table 2.6 presents the selected response strategies intended to mitigate or avoid the identified risks.

Risk ID	Expected Result, No Action	Risk Response Type	Response Description
TR1	Simplified or incomplete optimisation	Mitigation	Begin exploration early and prototype on small FSMs
TR2	Limited evaluation validity due to insufficient benchmark diversity	Mitigation	Use synthetic FSMs if necessary
OR1	Accumulated delays impact later stages	Mitigation	Break tasks into smaller pieces and prioritise early critical tasks
PR1	Loss of momentum and missed milestones	Avoid	Use structured routines
PR2	Blockers persist longer and slow progress significantly	Avoid	Prepare meeting agendas

Table 2.6: Risk Response Table

Chapter 3

State of the Art

This chapter reviews the theoretical foundations and existing literature relevant to the dissertation. Section 3.1 introduces the fundamental concepts required to understand FSM-based automated test generation. Section 3.2 describes the research methods used to conduct the systematic literature review. Section 3.3 presents the main findings. Section 3.4 summarises the identified research gaps that motivate the work developed in this dissertation.

3.1 Conceptual Background

3.1.1 Software Testing and Model-Based Testing

Software testing is concerned with determining whether a system behaves according to its specified requirements and expected behaviour. Traditional testing approaches usually derive test cases directly from requirements documents, user stories, source code, or tester experience. In practice, this often means manually selecting representative inputs and execution scenarios. This approach can be effective for smaller systems but becomes increasingly difficult as the number of possible behaviours grows [9].

Model-Based Testing (MBT) differs from traditional testing in that test cases are derived from an explicit model of the system under test. This model acts as an abstraction of the expected behaviour and captures the relevant states, transitions, events, inputs, and outputs of the system [10, 11]. The model therefore becomes the central artefact from which tests can be generated, analysed, and maintained.

This distinction is particularly important when systems exhibit many behavioural paths. As the number of possible event sequences increases, manually reasoning about all relevant scenarios becomes costly and error-prone. MBT is useful in such cases because the behavioural model provides a structured representation of the possible executions of the system, making it easier to systematically derive test cases that explore alternative paths, boundary situations, and interaction sequences [11, 12].

The relationship between behavioural models and generated test cases is direct: the model defines the permitted system behaviour, while the generated test cases correspond to paths, sequences, or scenarios extracted from that model. The quality of the generated test suite depends not only on the model itself, but also on the coverage criteria and generation strategy used to traverse it [11, 13].

3.1.2 Finite State Machines

Finite State Machines (FSMs) describe a system in terms of a finite number of states and the transitions that connect them. A transition is triggered by an input and may produce a corresponding output. This makes FSMs especially suitable for reactive and event-driven systems such as communication protocols, embedded systems, and interactive software [2, 14].

Formally, an FSM can be represented as a tuple (S, I, O, T, s_0) , where S is the set of states, I the set of inputs, O the set of outputs, T the set of transitions, and s_0 the initial state. Under this representation, the execution of the system begins at the initial state s_0 and evolves through transitions in T as inputs from I are applied and outputs from O are produced. This formalisation provides a concise and rigorous basis for reasoning about system behaviour and for deriving test cases in a systematic way [2, 3, 15].

3.1.3 Variants of FSMs

Although this dissertation focuses on classical FSMs, the literature also includes richer variants designed to represent systems whose behaviour cannot be captured conveniently by a simple finite set of states and transitions.

One variant is the Extended Finite State Machine (EFSM). EFSMs add to the classical FSM formalism variables, guards, and data-dependent operations, making them more suitable for enterprise applications, workflow-oriented systems, and other data-intensive contexts where behaviour depends not only on control flow but also on internal data values [16, 17].

Another relevant variant is the Symbolic Finite State Machine (SFSM), in which transitions may be expressed using symbolic constraints rather than only concrete values. This increased expressiveness comes with higher computational cost, since symbolic reasoning and constraint handling generally make generation and analysis more expensive than in classical FSM-based approaches [18, 19].

These variants are acknowledged because they are relevant in the broader FSM-based testing landscape, but the choice of formalism for this dissertation requires justification. Table 3.1 summarises key characteristics of the three formalisms, synthesised from the individual studies reviewed in this section.

Formalism	Modelling capability	Computational Cost	Tool/Benchmark Availability
Classical FSM	Limited	Low	High
EFSM	High	Medium	Medium
SFSM	High	High	Low

Table 3.1: Characteristics of FSM formalisms synthesised from individual studies

Regarding EFSMs, Ngamsaowaros and Sophatsathit propose a reduction framework specifically to manage the enlarged test domain that arises from EFSM data operations [16], which is evidence that EFSMs carry a higher computational cost than classical FSMs. Batth et al. demonstrate that EFSM models support richer fault modelling through data-dependent transitions, confirming their higher expressiveness [17]. For SFSMs, Petrenko confirms that checking experiments for symbolic FSMs require dedicated symbolic techniques that go beyond standard FSM methods [19]. Classical FSMs, by contrast, admit well-defined and

complete test suite generation algorithms [3], and their use is supported by a range of established benchmark models and evaluation baselines, as reflected in the breadth of the literature reviewed. These observations, taken together, support the selection of classical FSMs as the formalism for this dissertation. They provide sufficient expressiveness for the reactive and event-driven systems targeted here, while maintaining tractable computational cost and the standardised benchmarks required for the empirical evaluation in Chapter 4.

3.1.4 Challenges in FSM-Based Testing

The use of FSMs offers a clear and formal basis for MBT, but it does not eliminate the practical difficulties of generating effective and economical test suites. Once behavioural models are used to derive tests systematically, challenges emerge from the scale and cost of exploring that structure.

One of the most widely discussed problems is the state explosion effect, where the number of relevant states, paths, and transition combinations increases rapidly as the behavioural complexity of the model grows [4]. Even when the underlying formalism is manageable, exhaustive exploration may lead to test suites that are too large to generate, execute, or maintain efficiently.

A related issue is redundancy. Multiple generated test cases may exercise highly similar behaviours or contribute little additional fault detection value, while still increasing execution time and maintenance overhead [20–22]. This means that high structural coverage alone is not sufficient, test suites must also be assessed in terms of efficiency, distinctiveness, and practical usefulness.

Finally, there is the challenge of balancing this with applicability in real testing processes. Test generation strategies must account not only for coverage, but also for computational cost, test data selection, prioritisation, and integration with automated workflows [12, 23]. These challenges motivate the need for optimisation strategies and provide the background for the literature review developed in the remainder of this chapter.

3.1.5 Mutation Testing

Mutation testing is a fault-based technique for evaluating the fault detection capability of a test suite. It works by applying *mutation operators* to the specification under test, in this context a Finite State Machine, each producing a modified version known as a *mutant*. Each mutant differs from the original FSM by exactly one structural change, representing a single injected fault [23].

A mutant is *killed* if at least one test case in the suite produces a different output when executed against the mutant than when executed against the original machine. A mutant that no test case distinguishes from the original is said to have *survived*. A special category of surviving mutant is the *equivalent mutant*, which is a mutant whose modified structure produces exactly the same output as the original FSM for every possible input sequence. Since no test can ever produce a distinguishing output for an equivalent mutant, it cannot be killed by construction, and including it in a fault-detection measure would penalise the test suite for a structurally undetectable fault [23].

The *mutation score* is defined as the ratio of killed mutants to the total number of non-equivalent mutants:

$$\text{Mutation Score} = \frac{\#km}{\#tm - \#em} \quad (3.1)$$

where $\#km$ is the number of killed mutants, $\#tm$ the total number of generated mutants, and $\#em$ the number of equivalent mutants. A higher mutation score indicates stronger fault detection capability against the injected fault catalogue.

A further distinction that is relevant to this dissertation concerns surviving mutants that are not equivalent. Beyond equivalence, a mutant may survive not because the suite fails to detect the injected fault in principle, but because the test generation strategy cannot produce any input sequence that exercises it. Such mutants are structurally unkillable under the chosen strategy, yet they differ from equivalent mutants in that a different strategy or a manually constructed test could, in principle, distinguish them from the original. This distinction between genuinely surviving faults and strategy-constrained ones motivates the three-class mutant classification framework introduced in Chapter 5. The specific mutation operators applied throughout this dissertation are described in Section 3.3.2 as part of the analysis of how mutation testing is used across the reviewed literature.

3.2 Research Methods and Process

The literature review supporting this chapter was designed to provide a clear overview of current research. To achieve this, a structured review methodology was adopted, consistent with practices commonly recommended in empirical software engineering research [24].

A *Systematic Mapping Review* was chosen as the guiding methodology. This type of review helps organise the research by identifying recurring themes, categorising contributions, and revealing gaps that remain unexplored, which is particularly useful when a field contains diverse methods and perspectives [25].

The process also incorporated elements of the Preferred Reporting Items for Systematic reviews and Meta-Analyses (PRISMA) framework, which provides guidance on how to document study identification, screening, eligibility, and inclusion [26].

3.2.1 Data Sources

Three academic databases were selected to gather relevant studies. Priority was given to platforms with strong coverage of software engineering publications and to which full access is available through Instituto Superior de Engenharia do Porto (ISEP) institutional subscriptions, ensuring that the review is not limited to open-access or preprint versions of articles. This allows the inclusion of peer-reviewed conference and journal papers that may not be freely available but are relevant to the topic. Table 3.2 lists the selected sources.

Database	URL
ACM Digital Library	https://dl.acm.org
IEEE Xplore	https://ieeexplore.ieee.org
ScienceDirect	https://www.sciencedirect.com

Table 3.2: Databases used for identifying relevant publications

3.2.2 Search Strings

Search strings were defined by identifying core concepts associated with the dissertation topic and the research questions introduced in Section 1.4. The keywords and their corresponding search expressions are summarised in Table 3.3.

Keyword	Search Terms
Finite State Machines (FSM)	FSM / finite state machine
Test Generation	test generation / test case generation
Test Suite Optimisation and Evaluation	coverage / fault detection / redundancy / computational cost

Table 3.3: Search expressions used during the literature search

These terms were combined using Boolean operators and applied to the abstract field of each database. Restricting the search to abstracts increases the precision of the results by ensuring that all three concept areas are represented in the core description of each article. The resulting queries are presented in Table 3.4.

Data Source	Search String
ACM Digital Library	[[Abstract: fsm] OR [Abstract: "finite state machine"]] AND [[Abstract: "test generation"] OR [Abstract: "test case generation"]] AND [[Abstract: "coverage"] OR [Abstract: "fault detection"] OR [Abstract: "redundancy"] OR [Abstract: "computational cost"]]
IEEE Xplore	("Abstract":"FSM" OR "Abstract":"finite state machine") AND ("Abstract":"test generation" OR "Abstract":"test case generation") AND ("Abstract":"coverage" OR "Abstract":"fault detection" OR "Abstract":"redundancy" OR "Abstract":"computational cost")
ScienceDirect	("FSM" OR "finite state machine") AND ("test generation" OR "test case generation") AND ("coverage" OR "fault detection" OR "redundancy" OR "computational cost")

Table 3.4: Search strings used during the literature search

3.2.3 Screening Criteria

Articles were retained if they were published from 2010 onward, were accessible through ISEP institutional subscriptions or otherwise freely available, and did not constitute duplicate entries across databases. The specificity introduced by restricting the search to abstracts also justifies extending the date range to 2010 onward, since the reduced result set is unlikely to be dominated by outdated work.

3.2.4 Data Extraction

Before proceeding with data extraction, an eligibility process was performed, where studies were reviewed in full and consequently removed if the criteria present in Table 3.5 were not met.

ID	Criteria
C1	Presented a technique or methodology related to FSMs, MBT, or test optimisation
C2	Included an empirical assessment or formal justification

Table 3.5: Eligibility criteria

This last step marked the end of the coding phase of the PRISMA framework. Figure 3.1 summarises the execution of this process, and a complete list of the included studies can be found in Appendix D.

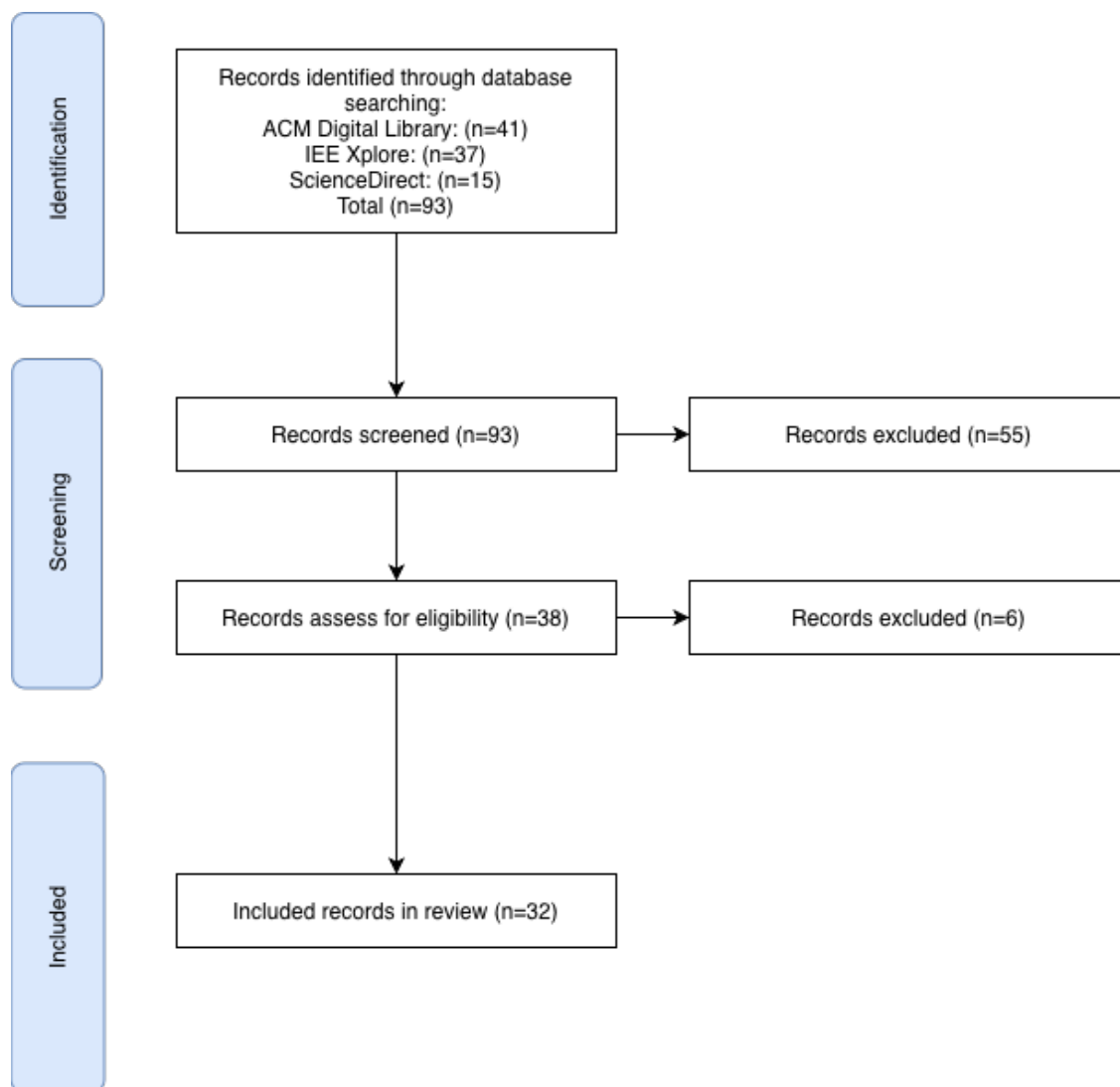


Figure 3.1: PRISMA Diagram

For each included publication, a structured extraction process was applied. Information was gathered regarding:

- The type of behavioural model used
- The test generation strategy adopted
- Optimisation goals or techniques addressed
- Evaluation metrics, benchmarks and challenges identified by the authors

This categorisation enabled consistent comparison across studies and forms the foundation of the discussion presented in the next section.

3.2.5 Literature Review Questions

The dissertation research questions, introduced in Chapter 1, guide the overall direction of the research and the development of the proposed artefact. However, the literature review requires a more specific set of questions to structure the analysis of existing work and to ensure that the reviewed studies are examined in a consistent and focused way. For this reason, a set of Literature Review Questions (LRQs) is defined.

- LRQ1.** Which optimisation strategies are used in FSM-based test generation to reduce redundancy and computational cost?
- LRQ2.** Which criteria and metrics are used to evaluate FSM-based test generation approaches?
- LRQ3.** Which factors influence the industrial adoption of FSM-based testing?
- LRQ4.** Which limitations remain unresolved in current FSM-based approaches?

3.3 Results and Analysis

This section analyses the identified literature presented in the previous section and organises the findings according to the Literature Review Questions defined above. The discussion therefore focuses on the optimisation strategies reported in the literature, the criteria used to evaluate existing approaches, the factors that influence industrial adoption, and the limitations that remain unresolved.

3.3.1 LRQ1 - Optimisation Strategies in FSM-Based Test Generation

The literature was organised into four main categories: redundancy reduction, fault detection improvement, prioritisation, and modelling abstraction. These categories were selected because they consistently appear across the reviewed studies as the primary dimensions influencing the efficiency and effectiveness of FSM-based test generation.

Redundancy in FSM-Based Test Suites and Strategies for Reduction

A consistent observation in the literature is that FSM-derived test suites often contain substantial redundancy, resulting in unnecessary execution cost without meaningful improvements in fault detection. Four strategies for addressing this redundancy can be identified in the literature: *duplicate removal*, *mutation score preserving reduction*, *coverage preserving reduction*, and *similarity reduction*. Table 3.6 compares them according to the criteria most relevant to this dissertation.

Table 3.6: Comparison of redundancy reduction strategies for FSM-based test suites.

Strategy		Formal guarantee	Fault detection preserved	Transition coverage preserved	Computational cost	FSM type
Duplicate removal	re-	None	Yes (by definition)	Yes (by definition)	Low	Classical
Mutation preserving reduction	score re-	Mutation score	Yes	Not guaranteed	Medium–High	Classical
Coverage preserving reduction	pre-	Transition coverage	Not guaranteed	Yes	Low	Classical
Similarity reduction	reduc-	None	Not guaranteed	Not guaranteed	Low–Medium	Classical / EFSM

Duplicate removal is the most elementary form of reduction, where any test case that is an exact duplicate of another is discarded before execution. Identical test cases contribute nothing structurally distinct to the suite, so their removal cannot reduce coverage and requires no formal justification. While the concept is self-evident, it serves as a necessary baseline in empirical studies by establishing the proportion of trivial redundancy already present before more sophisticated strategies are applied.

Mutation score preserving reduction aims to minimise suite size while providing a formal guarantee that fault detection capability is maintained. The literature proposes a search-based approach for classical, deterministic, completely specified, and initially connected FSMs [20], implemented using an Non-Deterministic Output-Error (NDOE) representation.

Test sequences are encoded using the NDOE scheme, which treats the entire test suite as a single continuous input sequence rather than isolated runs separated by system resets. Whenever the FSM has a transition connecting the state at which one test case ends to a state from which the next can begin, the two sequences are *chained* directly and no reset is required between them. A reset is inserted only where no such linking path exists. Multi-objective optimisation then searches for an ordering and subset of test sequences that simultaneously minimises total input length, minimises the number of resets, and maximises the mutation score against a suite of mutant FSMs. Test cases that kill no mutant not already killed by another case in the suite are eliminated, as their removal cannot lower the mutation score. Figure 3.2 illustrates both mechanisms. The top row shows the original four-case suite with three resets and the bottom row shows TC_3 eliminated. Since every mutant it kills (M_2) is already killed by TC_1 , and TC_2 chained directly to TC_4 via a linking FSM transition, reducing resets from three to one while all four mutants remain detectable.

The mutation score guarantee is embedded in the search objective: no candidate solution is retained unless it preserves the mutation score of the original suite. Applied to six benchmark FSMs, the method reduced suite size by up to 47% while maintaining mutation scores, at a computational cost ranging from a few seconds to a few minutes depending on model size [20].

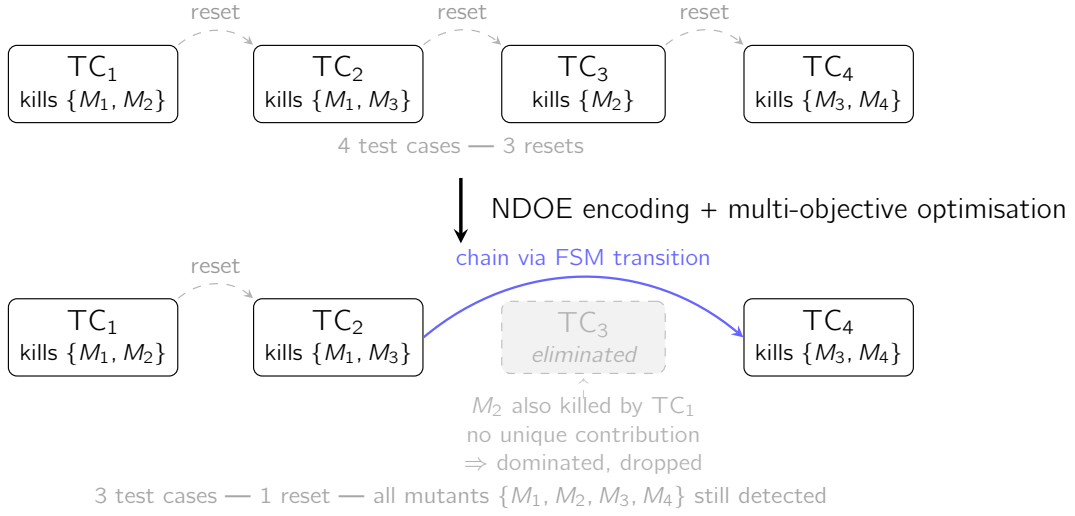


Figure 3.2: Mutation score preserving reduction via NDOE encoding.

Coverage preserving reduction aims to minimise suite size while ensuring that every transition in the FSM remains exercised by at least one retained test case. This objective is formalised as a set-cover problem, where given the original test suite and the set of transitions each case exercises, select the smallest subset of test cases such that every transition in the model is covered by at least one selected case [23]. The greedy set-cover repeatedly adds the test case that covers the largest number of as-yet uncovered transitions until the entire transition set is covered. Since the criterion is structural rather than fault-based, the guarantee is that the reduced suite satisfies transition coverage.

Similarity reduction selects a diverse subset of test cases under the assumption that diverse tests are less redundant and collectively exercise more distinct system behaviours. Biased Mutual Information (BMI) is a pairwise similarity measure between FSM test cases [21], defined as:

$$\text{bmi}(t_1, t_2) = \sum_{x \in t_2} \frac{n_x \cdot \log_2(m_x + 1)}{m_x}$$

where n_x is the count of input/output pair x in t_1 (capped at t_2 's count for normalisation) and m_x is the number of FSM transitions that carry the label x . Rare labels, those carried by few transitions, receive higher weight than common ones. The normalised form $\text{bmi}(t_1, t_2)/\text{bmi}(t_2, t_2)$ lies in $[0, 1]$ and represents the fraction of t_2 's informational content already covered by t_1 . A value close to 1 indicates high overlap, while a value close to 0 indicates that the two test cases exercise largely distinct input/output patterns. A test case is considered redundant with respect to an already-selected case when this normalised value meets or exceeds a similarity threshold. Empirical evidence shows that this strategy can occasionally reduce transition coverage on complex models [21].

The Test Script Generation Framework (TIGER+) framework illustrates how coverage and similarity concerns can be combined in an industrial setting, for Extended Finite State Machines (EFSM) models rather than classical FSMs. It reduces an EFSM-generated test suite in two sequential stages [27, 28]. First, a Modified Condition Decision Coverage (MCDC) greedy algorithm classifies test cases by the guard conditions they satisfy and selects the minimal subset that ensures every individual condition has independently affected the decision outcome. Second, within each group a similarity filter removes test cases whose sets of

triggered actions overlap too heavily with already selected cases. Applied to a fire detection subsystem of the Train Control Management System (TCMS) at Alstom Rail AB, TIGER+ reduced a suite of 307 MBT-generated test cases by 85%-92%, yielding subsets of 46 and 22 tests that preserved mutation scores between 95% and 100% [27]. It should be noted, however, that these results were obtained in an EFSM setting, where guard conditions and action variables substantially enlarge the test domain relative to classical deterministic FSMs. The reduction rates and preservation guarantees reported by TIGER+ are therefore specific to that richer modelling context and cannot be assumed to transfer directly to the classical, completely specified FSM setting addressed in this thesis.

Redundancy also arises at the model level. Work on minimising characterising sets demonstrates the theoretical difficulty of computing minimal distinguishing sequences, highlighting the necessity of heuristic approaches rather than exact optimisation [29]. Although such heuristics cannot guarantee globally minimal suites, they significantly improve practical efficiency without compromising behavioural soundness.

Enhancing Fault Detection Through Test Generation

Coverage alone does not reliably correlate with fault detection effectiveness.

A EFSM test generation approach derived test paths with generated input data, proceeding in three stages, such as extracting feasible test paths from the EFSM, constructing a classification tree for each path from the conditions on its transitions, and applying t-way combinatorial testing to derive a minimal set of concrete input assignments [30].

The classification step applies Boundary Value Analysis (BVA) directly to each condition (guard). For a condition such as $x \geq 10$, the input domain is partitioned into two equivalent classes, a *boundary* class ($x = 10$) and an *above-boundary* class ($x > 10$). For a relational condition like $x > y$, the corresponding classes are $x = y + 1$ (boundary) and $x > y + 1$ (above-boundary). Each transition in the path contributes one sub-tree whose leaves are these equivalent classes. The path's full classification tree is formed by combining all such sub-trees, as illustrated in Figure 3.3.

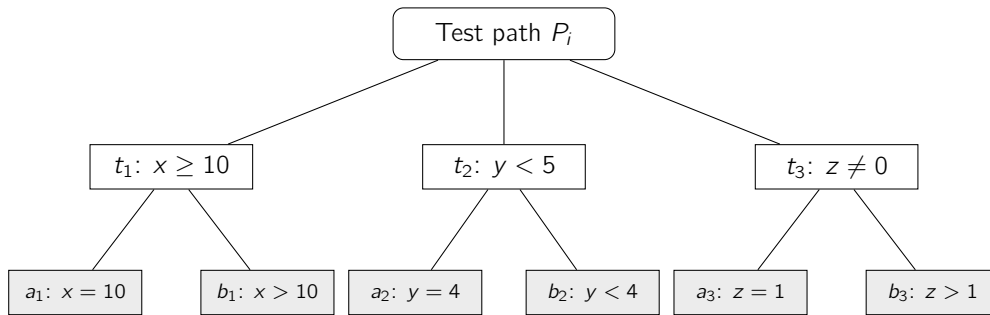


Figure 3.3: Classification tree for test path P_i : three guarded transitions partitioned into boundary and above-boundary classes by BVA.

Concrete test data is derived by constructing a *covering array* over the leaf classes using pairwise ($t = 2$) combinatorial testing. Pairwise coverage requires only that every combination of values from any two transitions appears in at least one test case. For a path with three transitions each contributing two classes, this reduces the required test data combinations from $2^3 = 8$ to just 4, and any combination that covers no new pair is discarded as redundant.

Applied to a lift-system EFSM with 6 states and 36 transitions, the approach produced 53 test sequences, substantially more than the 10 from the conventional MBT baseline, reflecting the broader combinatorial input space explored. Both approaches achieved identical transition coverage and the same code coverage (80.5%). However, the mutation score improved from 71.32% to 77.94% (+6.62%) [30].

This demonstrates that input diversity and boundary sensitivity are essential contributors to fault detection, when applied to EFSM models. Because this approach relies entirely on guard conditions and data variables, features absent from classical, completely specified FSMs, it falls outside the direct scope of this thesis. It is included here to contextualise the broader landscape of FSM-based test generation research and to highlight that fault detection can be further strengthened.

Prioritisation as a Mechanism for Efficiency and Early Fault Detection

As test suites grow, prioritisation becomes essential to maintain efficiency.

Prioritised State Machine Testing (PSMT) addresses three concurrent requirements that arise in industrial FSM-based testing: test paths must start and end only in defined states of the model, only priority-marked states and transitions need to be covered, and test paths must fall within a specified length range [31]. These requirements are practically motivated, as starting or ending tests in arbitrary states can be costly or infeasible in real systems, and excessively long or short paths create maintenance and reliability problems.

To support these requirements, PSMT accepts numerical priority values assigned by the tester to states and transitions in the FSM, reflecting domain knowledge such as risk or requirements criticality. Two coverage criteria are defined. PSMT Basic Coverage requires that all priority states and transitions reachable within the length constraints appear in the test suite. PSMT Extended Coverage additionally requires that all edges adjacent to priority vertices and all pairs of adjacent priority edges are covered. Once an initial set of paths satisfying the selected criterion is generated, a reduction step removes redundant paths. Several reduction variants are supported, including greedy sorting by coverage, among others. Figures 3.4 and 3.5 illustrate both criteria on a small example. In both diagrams, double circles mark priority states and thick arrows mark priority transitions. Extended Coverage additionally requires the dashed adjacent transitions, while thin arrows are not required by either criterion.

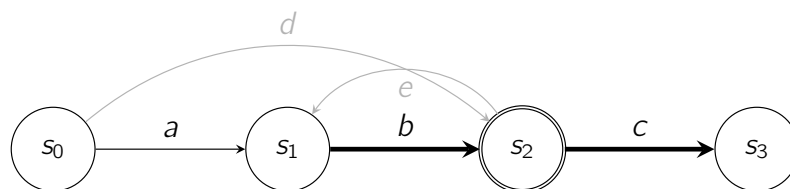


Figure 3.4: PSMT Basic Coverage.

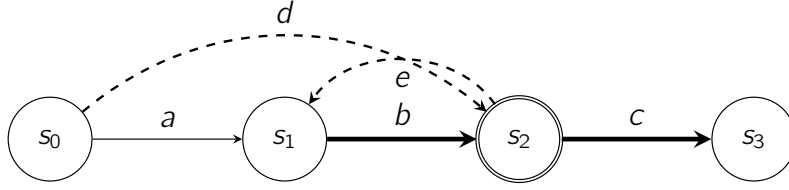


Figure 3.5: PSMT Extended Coverage.

Evaluated across 180 problem instances derived from real automotive and defence systems as well as artificially generated FSM models, the sort variant of PSMT achieved a 20% improvement in defect detection efficiency, while also producing smaller test suites [31]. These results indicate that priority-based path selection with length constraints can meaningfully improve testing effectiveness, and that prioritisation at the behavioural level can be systematically leveraged to enhance efficiency without enlarging the test suite itself.

Two broad prioritisation strategies can be identified in the literature and applied as post-generation ordering mechanisms on an already-generated test suite.

Coverage-based prioritisation orders an existing test suite greedily by structural coverage contribution. At each step, the unexecuted test case that covers the largest number of unvisited states or transitions is placed next in the execution sequence [1]. This approach requires no domain knowledge beyond the FSM structure itself and is straightforward to implement. Its advantage is that it minimises the number of tests that must run before full structural coverage is reached, ensuring the most structurally distinct tests execute earliest. Its limitation is that it treats all structural elements as equally important, providing no mechanism to prioritise transitions that carry higher risk or are more likely to reveal faults.

Weighted-transition prioritisation extends coverage-based ordering by assigning numerical priority weights to individual FSM transitions, reflecting domain knowledge such as requirements criticality or functional risk. Test cases are then ordered to maximise the cumulative weight of the transitions covered in the early portion of the execution sequence. This concept is present in PSMT, where transition and state priorities guide which paths must be generated [31]. Applying the same priority mechanism purely as an ordering criterion on an already-generated suite, rather than as a generation constraint, retains the domain-knowledge advantage without imposing the start-and-end state requirements or path-length constraints that characterise the full PSMT formulation. The strategy's principal limitation is that its effectiveness depends directly on the quality of the assigned weights. If weights are uniform or poorly calibrated, the ordering degrades to coverage-based prioritisation.

Figure 3.6 contrasts the two strategies on a minimal example. Four transitions are defined: t_1 , t_2 , t_3 each with weight 1, and t_4 (a high-risk transition) with weight 5. Three test cases are generated: TC_1 covers $\{t_1, t_2, t_3\}$, TC_2 covers $\{t_4\}$, and TC_3 covers $\{t_1, t_4\}$. Coverage-based prioritisation places TC_1 first because it covers the most transitions (3), then TC_2 and finally TC_3 — the critical transition t_4 is not reached until the second test. Weighted-transition prioritisation instead computes the total weight of newly covered transitions: TC_3 contributes weight $1 + 5 = 6$ (the highest), so it executes first and surfaces t_4 immediately; TC_1 follows (new transitions t_2 , t_3 contribute weight 2), then TC_2 (no new transitions, weight 0).

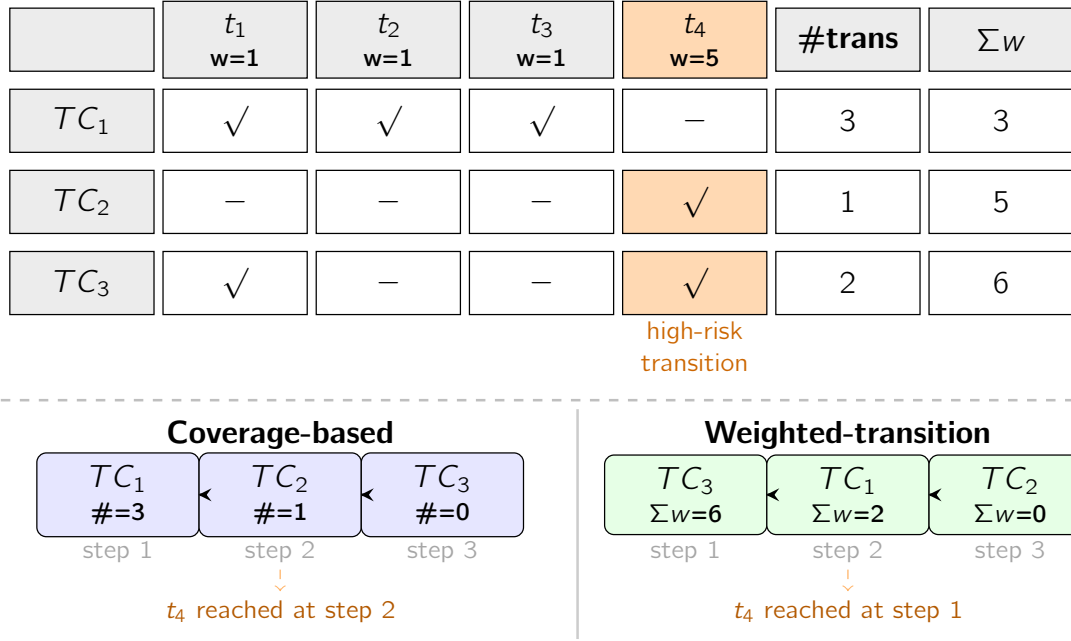


Figure 3.6: Coverage-based versus weighted-transition prioritisation on a four-transition example.

Influence of Modelling Abstractions on Test Suite Quality and Scalability

Optimisation outcomes in MBT depend heavily on the modelling abstractions used.

The *Action-State* testing model is a hybrid approach combining scenario-based and state-based testing while eliminating the modelling overhead of EFSMs [32]. Its core building block is the *step*, a combination of a user action (input), zero or more system responses (outputs), and an optional resultant state. Steps are arranged into a directed graph where sequential steps form a test case, forked steps distribute alternative scenarios across separate test cases, and joined steps bring forked paths back to a shared continuation.

The principal advantage over EFSMs is the elimination of guard conditions. In an EFSM, guards are code fragments embedded in transitions that determine which path can be taken from a given state, requiring programming knowledge and complicating model maintenance. Action-State removes this requirement by modelling each valid scenario as an explicit path in the graph, since every path represents a known and complete execution sequence, all transitions are inherently valid without conditional annotations. The test selection criterion applied is the 0-switch (all-edge) criterion, which requires every edge to appear in at least one test, keeping the suite size linear in the number of transitions. Evaluated on three applications using a mutation testing framework, action-state testing achieved 100% defect detection, outperforming GPT-4-generated tests (67%) and classical stateful modelling (93%) [32]. Figure 3.7 contrasts both representations on a PIN-entry example with one retry.

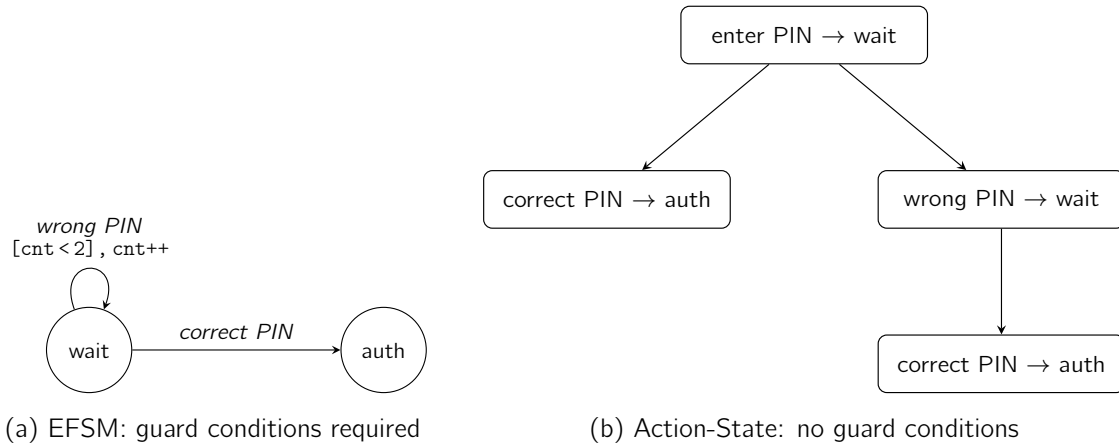


Figure 3.7: EFSM versus Action-State representation of a PIN-entry flow with one retry.

On the other hand, Symbolic Finite State Machine (SFSM) models expand modelling capabilities by introducing symbolic constraints and property-based specifications. While this permits more precise test derivation, it also introduces computational overhead. As such, symbolic methods offer improved behavioural accuracy at the cost of higher resource consumption, necessitating selective application to critical behaviours [18].

3.3.2 LRQ2 - Evaluation Criteria and Metrics

Evaluating FSM-based test generation approaches requires methodological rigour, as differences in model structure, test-generation objectives, and application domains can significantly affect results.

The literature demonstrates a diverse but converging set of evaluation strategies, with emphasis on fault detection, behavioural suitability, execution efficiency, and practical applicability in development contexts.

Fault Detection Capability: Mutation Analysis

Across the surveyed studies, mutation analysis emerges as the predominant method for quantifying fault detection effectiveness.

Mutation testing works by injecting controlled defects into the system model, producing a set of *mutants*, each differing from the original FSM by exactly one structural change. A mutant is said to be *killed* when at least one test case in the suite produces an output that differs from the output produced by the original machine, otherwise it survives. The ratio of killed mutants to the total number of non-equivalent mutants yields the *mutation score* [23]:

$$\text{Mutation Score} = \frac{\#km}{\#tm - \#em} \quad (3.2)$$

where $\#km$ is the number of killed mutants, $\#tm$ the total number of generated mutants, and $\#em$ the number of equivalent mutants. Equivalent mutants are those that are behaviourally indistinguishable from the original FSM and therefore cannot be killed by any test case. They are excluded to avoid penalising the test suite for faults that are structurally

impossible to detect. The mutation score quantifies the fault detection effectiveness of the test suite.

The mutation operators applied in FSM-based mutation analysis target the core structural elements of the machine. Three operators are formally defined in the literature for deterministic, completely specified FSMs [23]. The *Transition Target Mutation* operator redirects the destination state of an existing transition to a different state, simulating a *transfer fault* in which the machine reaches an incorrect state after processing an input. The *Output Mutation* operator replaces the output produced by a transition with a different one from the output alphabet, simulating an *output fault* in which the machine reaches the correct state but produces a wrong response. The *Initial State Mutation* operator changes the designated initial state to a different state, simulating an *initialisation fault* in which the machine begins execution in an incorrect configuration. A fourth operator, *Transition Addition*, introduces a new transition between two existing states, representing an unexpected path in the machine [20]. Two further operators extend this catalogue beyond what is covered by these sources. *Transition Removal* eliminates an existing transition, simulating a missing-path fault in which a valid input goes unhandled and *Input Mutation* substitutes the trigger input of a transition with a different input, simulating a wrong-trigger fault. Together, these six operator classes represent the dominant categories of structural fault that arise in FSM specifications.

Approaches such as TIGER+ apply mutation testing to demonstrate that extensive suite reduction, up to 85%–92%, can still preserve 95%–100% mutation scores, indicating that many model-generated tests do not contribute meaningfully [27].

Studies that analyse detection patterns across mutant types demonstrate that improved TDG strategies selectively enhance detection of specific fault categories, thereby offering insight into the strengths and weaknesses of each MBT method. This level of granularity is crucial for evaluating frameworks intended for safety-critical contexts.

FSM-Derived Coverage Criteria

Although coverage alone is insufficient for predicting fault detection, it remains essential for establishing behavioural suitability. Existing works use multiple FSM-derived coverage criteria.

State and transition coverage form the most basic structural criteria. A test suite achieves state coverage if, for every state in the FSM, at least one test case reaches that state. Transition coverage extends this requirement to every edge, so that each edge is exercised at least once [23]. These criteria guarantee that no state or edge goes unvisited, but they impose no constraint on the order in which transitions are taken and therefore provide no assurance that interactions between consecutive transitions have been exercised. Figure 3.8 illustrates the difference where under state coverage, transition *d* goes untested, while transition coverage requires every edge to be exercised at least once.

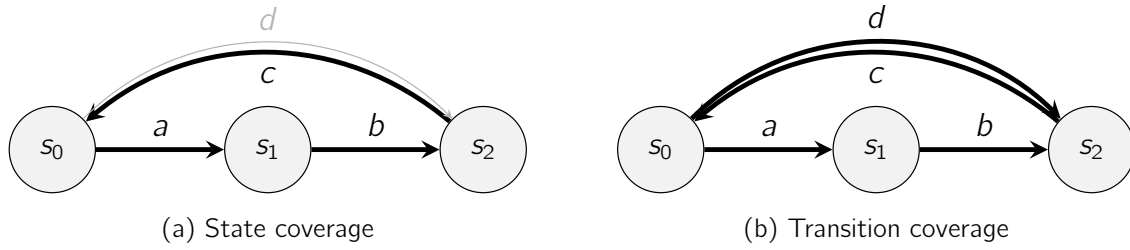


Figure 3.8: State and transition coverage on a three-state FSM.

Transition-pair and round-trip coverage address exactly this limitation by requiring that consecutive transition pairs be covered. A transition pair is any sequence of two edges that share an intermediate state. Covering all such pairs ensures that every possible two-step interaction within the model is tested [23]. Round-trip coverage further requires that every cycle reachable from the initial state be traversed at least once, capturing repetitive behaviour that single-pass traversal may miss. Longer test cases, covering more transition pairs and round-trips, correlate positively with higher fault detection ratios, confirming that richer structural coverage translates to stronger fault-revealing power [23]. Figure 3.9 illustrates both criteria on a three-state FSM.

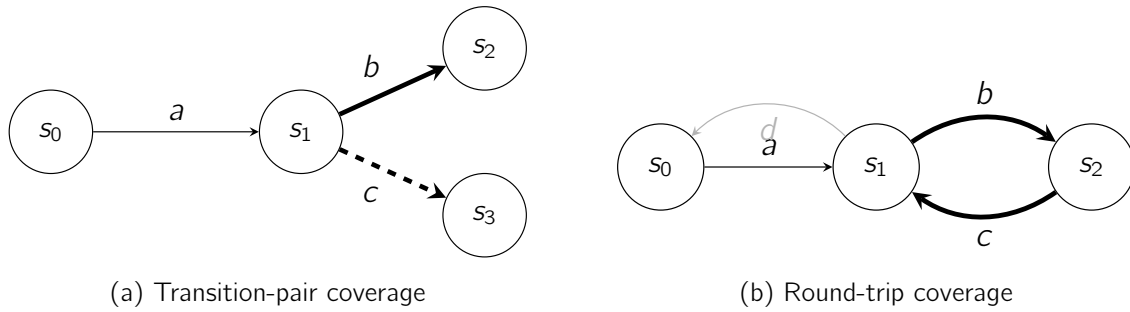
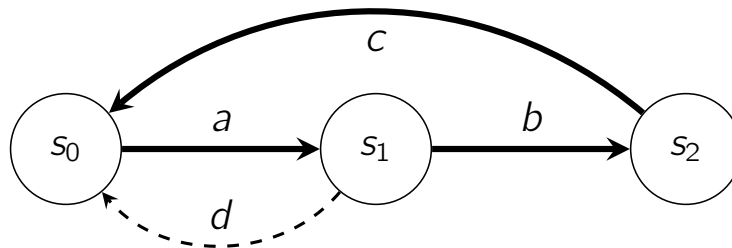


Figure 3.9: Transition-pair and round-trip coverage on a three-state FSM.

Generalised sequence coverage extends the idea of transition-pair coverage to longer interaction sequences [33]. Rather than requiring only consecutive pairs of transitions to be covered, it requires every consecutive sequence of length i , a configurable parameter, to appear in at least one test. For $i = 2$ this coincides with transition-pair coverage, increasing i captures deeper interaction patterns at the cost of a larger test suite. Figure 3.10 illustrates the pairs required at $i = 2$ on a small example.

Figure 3.10: Generalised sequence coverage at depth $i = 2$ on a three-state FSM.

Requirements-based completeness shifts the focus from structural paths to explicit requirements [3]. Each requirement is encoded as a triple $R(q, x, Z)$, specifying that from state q under input x the system must reach a state in the set Z (Figure 3.11). Two testing strategies are defined: an *exhaustive* strategy, where a passed test suite guarantees that all requirements are satisfied, and a *complete* strategy, where a passed test suite passes if and only if the requirements hold. Applied to an aircraft cabin pressure controller, this approach achieves a 20–60% reduction in test suite size.

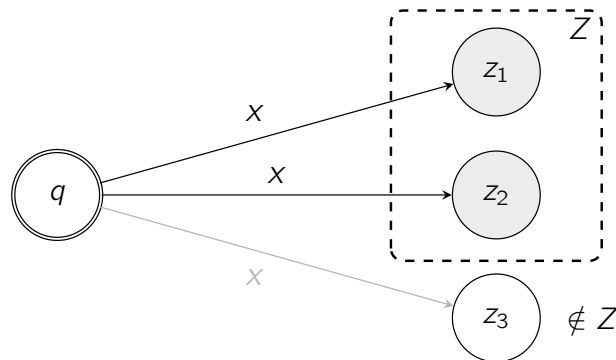


Figure 3.11: Requirement triple $R(q, x, Z)$ on a three-state example.

State and transition coverage offer a lightweight baseline with no domain knowledge required, but leave transition interactions untested. Transition-pair and round-trip coverage increase depth at the cost of larger suites. Generalised sequence coverage provides a parameterisable middle ground that can be tuned to the desired interaction depth. Requirements-based completeness delivers the strongest semantic guarantee but demands a formal requirements model. The appropriate choice depends on the context, available artefacts, and the trade-off between suite size and assurance level.

Efficiency and Computational Cost: Suite Size, Execution Time, and Generation Overhead

Efficiency metrics focus primarily on test suite size, execution time, and the computational cost of test generation. Studies on minimisation show that FSM-based suites can be extremely large prior to reduction, motivating the need for metrics such as:

- Reduction ratio
- Execution-time savings following reduction or prioritisation
- Generation-time cost

Although many works report significant reduction ratios or prioritisation improvements, several note that computation time is under reported. This gap highlights the need for explicit reporting of runtime metrics when evaluating modern MBT frameworks [23].

Prioritisation Metrics: APFD and Early Fault Detection Performance

Efficient execution ordering is essential when suites are too large to run exhaustively. To operationalise prioritisation effectiveness, studies employ the APFD metric, which quantifies how quickly a prioritised test suite exposes faults [1].

APFD is computed as:

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{nm} + \frac{1}{2n} \quad (3.3)$$

where n is the number of test cases in the suite, m is the number of mutants killed by at least one test case in the current suite, and TF_i is the position of the first test case in the ordered suite that detects fault i . APFD takes values in $[0, 1]$: a value of 1 indicates that all faults are detected as early as possible, a value near 0.5 approximates the expected rate of a randomly ordered suite, and values above 0.5 indicate progressively faster detection. A higher APFD therefore directly reflects a more effective prioritisation.

Behavioural prioritisation methods such as PSMT show improved early detection by ordering FSM paths according to risk or structural significance [31]. Prioritisation metrics therefore complement mutation and coverage metrics by evaluating not only whether faults are detected, but how quickly this occurs.

Qualitative and Practical Evaluation: Modelling Effort, Maintainability, and Tooling Integration

Several studies emphasise the importance of qualitative criteria that measure how well MBT techniques integrate into real development workflows. These include:

- Modelling effort and maintainability
- Automation level
- Tool integration

Studies on automated model-to-test tools highlight that practical adoption depends as much on tooling maturity and model usability as on raw fault detection performance [34].

3.3.3 LRQ3 - Industrial Adoption of FSM-Based Testing

The reviewed literature indicates that the industrial adoption of FSM-based testing depends not only on the existence of effective test generation techniques, but also on whether those techniques can be integrated into existing development processes with manageable effort and cost.

One major challenge concerns integration with automated development workflows such as CI/CD pipelines. In principle, MBT can support automation by deriving executable tests from behavioural models, but in practice its integration into continuous delivery processes requires tooling, repeatable model transformations, and workflows that fit the industrial software evolution. This is not a recent concern, the difficulty of bridging MBT with practical delivery pipelines was already identified in early industrial deployments [35] and recent studies confirm that the challenge persists in modern CI/CD contexts [34, 36].

Another barrier is the cost of model creation and maintenance. Automated model-to-test pipelines can reduce some manual effort, but they remain strongly dependent on the availability of accurate and maintainable models [34].

Scalability also remains a recurring adoption issue. As systems grow in behavioural complexity, the size of models and the volume of generated test artefacts can become difficult to manage. Even when generation is technically feasible, execution cost, update cost, and

regeneration effort can limit operational usefulness [34, 36]. Incremental approaches based on Featured Event Sequence Graphs (FESG) demonstrate that reuse across product evolution can alleviate part of this burden, though they do not eliminate the general scalability problem [36].

FESG extend Event Sequence Graphs (ESGs), a graph-based notation equivalent in expressive power to FSMs, to support Software Product Lines (SPL) testing. In this model, each feature of the product line is represented as a separate feature-specific ESG (f-ESG), while core system behaviour is captured in a core ESG (c-ESG). A concrete product's model is then composed from the c-ESG and the f-ESGs of its included features, making explicit which event sequences belong to which feature. For incremental test generation, when a new feature is added to an existing product, the approach avoids rebuilding the entire model from scratch. The test set for the existing product is retained in full and only the f-ESG of the new feature is composed with it to derive the additional test sequences required [36].

Figure 3.12 illustrates this incremental workflow. When a new feature is added to an existing product, only the f-ESG for that feature must be composed with the retained test set, the rest of the model is unchanged.

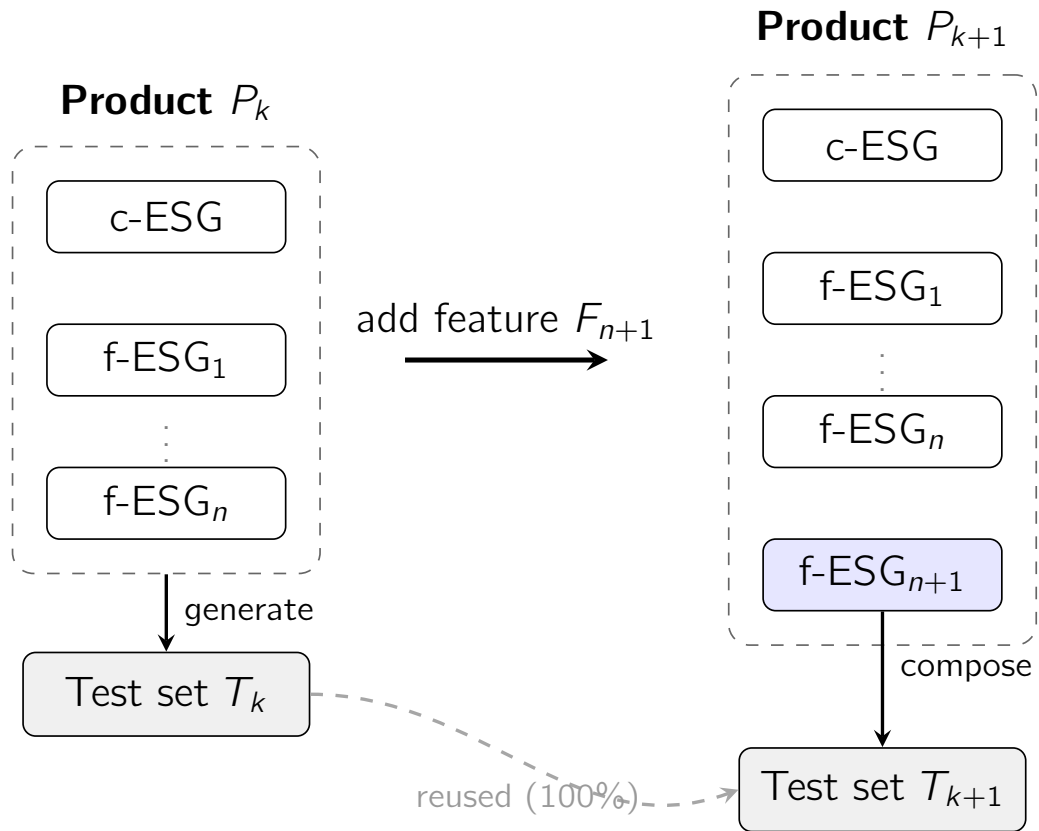


Figure 3.12: Incremental test generation with FESGs [36].

An empirical evaluation across three SPLs, a bank account system, an e-mail client, and a student attendance system, showed that this strategy achieved complete reuse of existing test sets (100%), generated tests up to 34 times faster than remodelling from scratch, produced test suites that were on average 9% larger, and detected 1.33% more faults [36]. These results are promising for settings where a product line evolves continuously.

The literature also highlights a strong dependency on model quality. If the behavioural model is incomplete, outdated, or oversimplified, the generated test suite inherits these weaknesses. This creates a practical risk for industrial adoption, because the success of the testing process becomes tied to modelling discipline, modelling expertise, and the availability of reliable abstractions [34, 35].

Finally, these technical and organisational issues contribute to resistance to adoption in industry. Organisations may hesitate to introduce FSM-based testing when the expected benefits are offset by the need for specialised modelling practices, additional maintenance effort, uncertain scalability, or disruption to established workflows [34–36]. The literature suggests that the primary barrier to industrial adoption is not the lack of test generation techniques, but the difficulty of integrating them into existing development workflows with acceptable modelling effort and maintenance cost.

3.3.4 LRQ4 - Limitations and Research Gaps

The reviewed studies demonstrate that FSM-based testing has a mature theoretical foundation and a diverse set of optimisation and evaluation techniques. However, the literature also reveals several limitations that remain unresolved and that directly motivate the need for this dissertation.

First, the reporting of computational cost remains inconsistent. While several studies discuss reduction ratios, prioritisation performance, or fault detection improvements, generation-time and execution-time costs are not always reported with the same level of detail [23]. As a result, it is often difficult to judge the practical trade-offs between effectiveness and resource consumption.

Second, industrial integration remains a persistent gap. FSM-based and MBT approaches can be valuable in practice, but their adoption is constrained by modelling effort, tooling limitations, maintenance overhead, and workflow integration challenges [34–36].

Finally, current approaches are often specialised rather than unified. Existing studies frequently address a particular concern such as reduction, prioritisation, or test data generation, but rarely provide an end-to-end framework that combines path exploration, test data derivation, redundancy suppression, prioritisation, and practical integration into a single coherent approach [18, 27, 30, 31].

Taken together, these gaps justify the need for further research on FSM-based automated test generation. In particular, they support the development of an approach that not only seeks strong fault detection and adequate coverage, but also addresses scalability, computational efficiency, and compatibility with modern software development workflows.

3.4 Summary and Research Gaps

The literature reviewed in this chapter reveals a field with a solid theoretical foundation and a diverse set of testing techniques, but also with recurring limitations that have not yet been completely addressed. This section presents the main findings across the four literature review questions and identifies the research gaps that directly motivate the design decisions made in Chapter 4.

Across the reviewed studies, three main limitations occur consistently. First, existing approaches are largely specialised rather than unified. This gap motivates the architecture

adopted in the experimental artefact, a structured sequence of configurable generation, reduction, and prioritisation stages and informs the strategy selection principles established in Chapter 4, Section 4.4.

Second, the reporting of computational cost across studies is inconsistent. Reduction ratios and fault detection improvements are frequently reported, but generation time and execution time costs are not always disclosed with the same rigour [23], making it difficult to assess the practical trade-offs between effectiveness and resource consumption. In response, computational cost is included as an explicit evaluation metric in the experimental study of this dissertation, alongside mutation score, coverage, APFD, and suite size, allowing the feasibility of different strategy configurations to be assessed.

Third, industrial integration remains a persistent gap. Despite the availability of effective test generation techniques, their adoption in practice is constrained by modelling effort, tooling limitations, and the difficulty of embedding them into workflows [34–36]. This gap is addressed in Chapter 4 by treating CI/CD compatibility as a design objective of the final artefact.

The absence of a unified, computationally transparent, and industrially integrable approach to FSM-based test generation defines the scope and direction of the contribution proposed in this dissertation, and provides the justification for the experimental work and the design principles developed in the remainder of the document.

Chapter 4

From Experimental Prototype to Final Artefact Design

This chapter follows the DSR design-evaluation cycle by presenting and evaluating an experimental prototype, the initial artefact built to instantiate and empirically test the proposed approach, and then translating the resulting evidence into the design of the final artefact. The experimental prototype is a configurable test generation framework that combines coverage, reduction, and prioritisation strategies over classical FSM models, allowing the effects of different configurations to be measured and compared under controlled conditions. The evaluation evidence produced by the prototype informs a set of explicit design decisions for the final artefact, which refines and consolidates the prototype into a more complete solution.

The chapter is organised as follows. Section 4.1 describes the experimental prototype and presents the evaluation results. Section 4.2 derives design implications for the final artefact from those findings. Section 4.3 addresses integration with CI/CD workflows as a design objective of the final artefact. Section 4.4 defines the strategy selection principles that guide the final artefact's configuration. Section 4.5 summarises the improvements that the final artefact introduces over the experimental prototype.

4.1 Experimental Artefact and Evaluation

Within the DSR design-evaluation cycle, the experimental artefact developed for this dissertation serves as the bridge between conceptual solution design and empirical assessment. Its role is not merely to implement the ideas identified in the previous chapters, but to provide an operational environment in which those ideas can be instantiated, combined, and evaluated under consistent conditions. In this sense, the artefact embodies the research contribution in a form that can be observed, exercised, and analysed, thereby allowing the study to move from theoretical propositions about FSM-based test generation to measurable evidence regarding their practical consequences.

The artefact was designed as an experimental prototype whose primary purpose is to support the systematic comparison of alternative strategies for generating and optimising test suites derived from classic FSM models. More specifically, it enables the combination of coverage strategies, reduction strategies, and prioritisation strategies, so that different configurations can be executed over the same benchmark models and assessed using the same evaluation procedures. The coverage dimension includes State, Transition, TransitionPair, and Sequence(*i*) strategies. The reduction dimension includes Duplicate Removal, Mutation Score Preserving Reduction, Coverage Preserving Reduction, and Similarity Reduction.

The prioritisation dimension includes coverage-based and weighted-transition approaches. By structuring the prototype around these configurable dimensions, the artefact makes it possible to investigate not only the isolated effect of each strategy, but also the interaction effects that emerge when they are applied together.

An experimental prototype is necessary because the research problem addressed in this dissertation cannot be evaluated adequately through conceptual analysis alone. The effectiveness of a test generation strategy depends on observable trade-offs between several competing outcomes, including fault detection capability, structural coverage, execution ordering, suite size, and processing effort. These properties only become evident when the strategies are exercised against models in a repeatable setting. The artefact therefore provides the infrastructure required to execute test generation workflows and collect comparable measurements.

The evaluation supported by the artefact focuses on the impact of the selected strategy combinations on mutation score, coverage, APFD, test suite size, total steps, and computational cost. These criteria were selected because they reflect the central objectives of the dissertation. Mutation score is used to assess fault detection capability, coverage indicates the extent to which the behavioural structure of the model is exercised, and APFD captures the extent to which faults are revealed early in the prioritised execution order. Test suite size provides an indication of compactness and maintainability, total steps captures the execution cost independently of suite size, while computational cost reflects the practical feasibility of applying a given strategy in realistic development settings.

From a DSR perspective, the value of the artefact lies not only in its implementation, but also in its usefulness as an instrument for evaluation and learning.

4.1.1 Artefact Design

This section describes the design of the experimental artefact, covering the requirements that constrained it, the technology selected for its implementation, and the sequence of processing stages that structures its execution.

Requirements

The design of the experimental artefact was conditioned by a set of functional and non-functional requirements established prior to implementation. Table 4.1 lists these requirements and the design decisions that follow are direct responses to them.

ID	Type	Requirement
FR1	Functional	Accept a deterministic FSM as input and produce test sequences from it
FR2	Functional	Support configurable coverage strategies: State, Transition, TransitionPair, and Sequence(<i>i</i>)
FR3	Functional	Support configurable reduction strategies: Duplicate Removal, Mutation Score Preserving Reduction, Coverage Preserving Reduction, and Similarity Reduction
FR4	Functional	Support configurable prioritisation strategies: coverage-based and weighted-transition
FR5	Functional	Report evaluation metrics per configuration: mutation score, coverage, APFD, suite size, total steps, and computational cost
NFR1	Non-functional	Deterministic and reproducible execution to ensure experimental control
NFR2	Non-functional	Extensible architecture allowing new strategies to be integrated without modifying the existing processing stages
NFR3	Non-functional	Configurable strategy selection to support a full factorial experimental design

Table 4.1: Functional and non-functional requirements of the experimental artefact

The acceptance criteria for these requirements are operationalised as a suite of unit tests covering each strategy module individually. Each test verifies that a given strategy produces the expected output for a known FSM input, ensuring that the measurements reported in Section 4.1.4 reflect the actual behaviour of the implemented strategies rather than implementation defects.

Technology Selection

The experimental artefact was implemented in C# using .NET 8 and was designed as a research prototype centred on deterministic FSMs. The choice of C# and .NET 8 was motivated by several factors. First, it reflects the researcher’s primary area of professional expertise, which allowed implementation effort to remain focused on the experimental design rather than on language or framework familiarisation. Second, C#’s strong typing and object-oriented model are well suited to the processing structure required. The processing stages map naturally to class hierarchies, and the FSM, State, and Transition abstractions benefit from the explicit type system. Third, .NET 8 is a Long-Term Support release with mature standard libraries for collections, graph traversal, and data processing. Finally, .NET 8 integrates readily with modern CI/CD toolchains, which is relevant given that CI/CD compatibility is an explicit objective of the final artefact. The choice of deterministic FSMs as the modelling formalism is aligned with the scope of the dissertation, since deterministic machines provide a precise representation of behaviour through explicit state transitions and associated outputs.

Unit testing was performed using NUnit as the testing framework and NSubstitute for test doubles, enabling each component to be verified in isolation against known FSM inputs without coupling the tests to unrelated processing stages.

Regarding input and output formats, FSM models are specified as JSON files, each describing the state set, input and output alphabets, transition relation, and initial state. JSON was chosen for its human-readability and straightforward deserialisation support in .NET. Evaluation results are exported as Comma-Separated Values (CSV) files, with one row per experimental configuration recording all collected metrics and supporting values: SuiteSize, TotalSteps, PreReductionTotalSteps, ReductionRatio, ExecutionTimeSavings, StateCoverage, TransitionCoverage, MutationScore, per-operator mutation scores, APFD, and GenerationTime. This format was selected for its simplicity and wide compatibility with data analysis tools, making the results straightforward to process and interpret.

The prototype source code, benchmark FSM model specifications, and raw experimental results reported in this chapter are publicly available in the project repository [7].

Processing Stages

At the architectural level, the artefact is organised around two concerns: behavioural representation and experimental processing. The behavioural representation is supported by a core model composed of the abstractions *FSM*, *State*, and *Transition*. These elements provide the structural basis required to describe benchmark models consistently. The experimental processing concern is organised as a sequence of stages in which generated test suites are successively subjected to generation, reduction, and prioritisation.

The first stage is responsible for test generation according to a selected coverage strategy. In this context, a *coverage criterion* defines which elements of the FSM a test suite must exercise. A *coverage strategy* is the algorithm that generates test sequences to satisfy that criterion. Four coverage strategies are supported, each operationalising a distinct criterion: *State Coverage*, *Transition Coverage*, *Transition Pair Coverage*, and *Sequence Coverage* (i), for a detailed treatment of each, see Section 3.3.2. For this experiment, $i = 3$ was selected. Sequence coverage at $i = 1$ and $i = 2$ is subsumed by Transition Coverage and Transition Pair Coverage respectively, making $i = 3$ the smallest sequence-length value that exercises qualitatively new interaction depth. Within each experimental configuration, exactly one coverage strategy is applied. The four strategies are mutually exclusive, and their comparative effects are reported in Section 4.1.6.

The second stage applies a reduction strategy to remove unnecessary tests from the generated suite. Four strategies are supported: *Duplicate Removal*, *Mutation Score Preserving Reduction*, *Coverage Preserving Reduction*, and *Similarity Reduction*. Each operates under a different removal criterion and offers a different preservation guarantee. For a detailed treatment of each, see Section 3.3.1.

Within each experimental configuration, exactly one reduction strategy is applied. Although the strategies are not inherently mutually exclusive and could in principle be chained, doing so in this experiment would mix their individual contributions to the outcome metrics and it would be difficult to attribute an observed change in mutation score or suite size to a specific strategy. The factorial design is therefore structured to measure each strategy's effect in isolation while exploring combinations is left as future work.

The third stage applies a prioritisation strategy to order the remaining tests according to different assumptions about early value delivery. Two strategies are supported: *Coverage-Based Prioritisation* and *Weighted-Transition Prioritisation*. Both are formally introduced with their respective trade-offs in Section 3.3.1. The latter adapts the priority weight mechanism from PSMT [31] as a post-generation ordering criterion.

Figure 4.1 illustrates the structure of these processing stages.

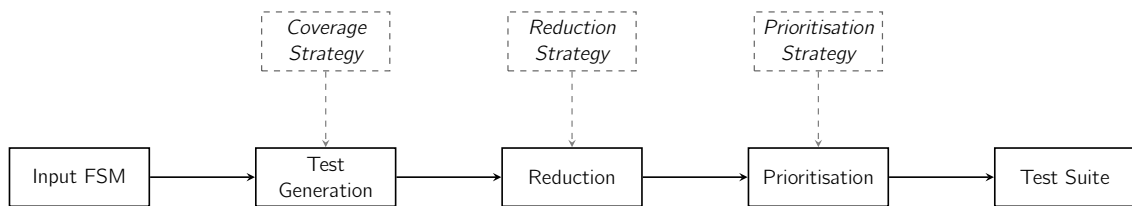


Figure 4.1: FSM-based test generation processing stages

The overall design follows a full factorial structure in which four coverage strategies, four reduction strategies, and two prioritisation strategies are combined, resulting in thirty two configurations for each FSM under evaluation.

This design decision is necessary to support systematic comparison. Evaluating isolated techniques independently would not be sufficient, because the effect of one strategy may depend on the choice of another. For example, a reduction method that performs well after one coverage strategy may behave differently when the initial suite is generated under a more demanding criterion.

For the purposes of this dissertation, the architecture of the artefact is therefore not only a software design decision but also an experimental design instrument. This is essential for a DSR-oriented evaluation, since the goal is not simply to demonstrate that the prototype functions, but to use it as a structured means of generating evidence about which combinations of design choices offer the most favourable balance between fault detection capability, coverage, suite compactness, prioritisation effectiveness, and computational cost.

It should be noted that integration with CI/CD pipelines falls outside the scope of the experimental artefact. The prototype was designed exclusively as an instrument for strategy evaluation, with the objective of generating comparable empirical evidence under controlled conditions.

The component architecture of the prototype is detailed in Appendix E. It shows how the *ExperimentRunner* orchestrates the *TestGenerator*, *TestReducer*, *TestPrioritiser*, and *MutationTester* components, each of which interacts with its corresponding strategy family through the interfaces *ICoverageStrategy*, *IReductionStrategy*, *IPrioritisationStrategy*, and *IMutationOperator* respectively. Isolating each strategy family behind an interface ensures that any strategy can be substituted without modifying the consuming components, directly supporting NFR2. The detailed sequence of interactions during a single test suite generation run, covering path generation from the FSM, test case conversion, reduction, prioritisation, and result collection, is illustrated in Appendix F.

4.1.2 Strategy Implementation

This section describes how each strategy across the three processing stages was concretely implemented in the experimental artefact.

Test Generation Strategies

Four of the five coverage criteria identified in Section 3.3.2 were implemented: State Coverage, Transition Coverage, Transition-Pair Coverage, and Sequence Coverage. Requirements-based completeness was excluded from the experimental scope. That criterion presupposes formal requirements that maps individual requirements to specific FSM elements. The benchmark models used in this experiment do not provide such requirements and constructing artificial ones for each benchmark would introduce an uncontrolled variable into the experiment.

All four strategies are implemented as graph traversal algorithms over the FSM model, operating on the set of reachable states and transitions from the initial state.

State Coverage applies a Breadth-First Search (BFS) from the initial state. The initial state is marked covered at the start, as it is the starting point of every execution and requires no dedicated path. The strategy then iterates over every reachable state in BFS discovery order. For each state not yet covered, the shortest path from the initial state to that target is computed via BFS, and every state visited along that path is immediately marked covered, so later iterations skip states already encountered as intermediaries. The result is a set of shortest paths whose union covers all reachable states, with a suite size of at most $|S_r| - 1$, where S_r is the set of reachable states, and often fewer due to prefix sharing.

Transition Coverage generates one test path per reachable transition. For each transition t , the strategy computes the shortest path from the initial state to t 's source state via BFS and then appends t to that prefix. Transitions whose source state is unreachable from the initial state are skipped. The suite size before reduction is exactly $|T_r|$, the number of reachable transitions. Since each transition receives its own dedicated path, the suite contains natural prefix redundancy, which makes it a well-structured input for the downstream reduction strategies. Transition Coverage subsumes State Coverage.

Transition-Pair Coverage enumerates all valid consecutive transition pairs (t_1, t_2) where t_1 's target state equals t_2 's source state. For each uncovered pair, the strategy builds a path as the BFS shortest path from the initial state to t_1 's source state, followed by t_1 and then t_2 . After adding a path, all consecutive pairs that appear anywhere within it are marked covered, not only the target pair. This incidental coverage means that a single path can cover multiple previously uncovered pairs, including pairs embedded in the BFS prefix. Iteration stops when all pairs are covered and pairs whose source state is unreachable from the initial state are skipped. The suite size before reduction is at most $|P_r|$, where P_r is the set of reachable consecutive pairs, and usually fewer due to incidental coverage. Transition-Pair Coverage subsumes Transition Coverage.

Sequence Coverage at $i = 3$ implements the consecutive-sequence coverage criterion described in Section 3.3.2. The strategy performs a Depth-First Search (DFS) from the initial state, permitting revisits so that cyclic paths are explored. Every partial path of length 1 to i encountered during the traversal is recorded. When a path reaches length i , the DFS backtracks. After the traversal, a redundancy filter removes any path that is a strict prefix of another path already in the set, since the longer path already covers the shorter one as a contiguous leading subsequence. $i = 1$ and $i = 2$ are subsumed by Transition Coverage and Transition-Pair Coverage respectively, as established in Section 4.1.1. $i = 3$ is therefore the smallest value that exercises qualitatively new interaction depth not already covered by the other criteria in the experiment.

Reduction Strategies

Duplicate Removal iterates over the generated suite and discards any test case whose sequence of transition IDs is identical to a case already retained and duplicates are detected using a hash set. The retained set is built in generation order, so the first occurrence of each distinct path is kept and all subsequent duplicates are removed.

Coverage Preserving Reduction implements the greedy set-cover algorithm described in Section 3.3.1. The coverage target is defined as the union of all transition IDs exercised by the original suite, rather than the full set of FSM transitions, so that transitions unreachable under the selected generation strategy are excluded from the target. The algorithm iteratively selects the test case that covers the largest number of uncovered transitions from this target, adding it to the retained set, and stops when the full coverage target has been achieved or no remaining candidate adds new coverage. The result is a minimal subset that guarantees 100% of the transition coverage achievable by the original suite.

Mutation Score Preserving Reduction is inspired by the NDOE based approach described in Section 3.3.1, which pursues three objectives simultaneously: maximising mutation score, minimising total input length, and minimising the number of resets between test cases. Implementing a full multi-objective evolutionary algorithm for this context would have represented a significant implementation effort for a proof-of-concept prototype subject to time constraints. A greedy approximation was adopted instead, which is both faster to implement and more transparent for a research prototype.

The implementation proceeds in three steps. First, all six mutation operators are applied to the FSM to produce the full mutant set, and every test case is executed against every mutant. Mutants killed by no test case (equivalent mutants) are excluded from the coverage target. Second, the end state of the last-selected test case is tracked throughout the selection process: a candidate can be chained directly if its start state matches this end state, otherwise a reset is counted. Third, the algorithm repeatedly selects the candidate with the highest score:

$$\text{Score} = \alpha \cdot \frac{|\text{uniqueKills}|}{|\text{totalMutants}|} - \beta \cdot \frac{|\text{length}|}{|\text{maxLength}|} - \gamma \cdot \text{resetsAdded}$$

where $|\text{uniqueKills}|$ is the number of mutants not yet killed by the already selected set, $|\text{length}|$ is the number of steps in the candidate, $|\text{maxLength}|$ is the length of the longest test case in the suite, and resetsAdded is 0 if the candidate can be chained and 1 otherwise. The weights $\alpha = 1.0$, $\beta = 0.1$, and $\gamma = 0.1$ were chosen so that mutation score dominates the selection while length and reset penalties serve as tiebreakers. Selection stops when all killable mutants are covered or no remaining candidate kills any new mutant.

Similarity Reduction operates as a two-stage process. In the first stage, the same greedy set-cover algorithm used by Coverage Preserving Reduction is applied, producing a minimal subset with full transition coverage. In the second stage, a BMI-based filter is applied over that result. The transition label weights m_x , the number of FSM transitions carrying each input/output pair x , are computed once over the FSM at the start of this stage. For each candidate test case, its normalised BMI is computed against every already selected test case. If the score meets or exceeds a similarity threshold for any selected case, the candidate is considered redundant and removed. The threshold was set to 0.8, representing a strong similarity signal where a candidate sharing 80% or more of a selected test's informational content is treated as sufficiently redundant for removal. This value was

not theoretically derived from the BMI formulation, as BMI is applied as a suite-level selection criterion rather than a pairwise filter, and prescribe no filtering threshold. The 0.8 value was adopted as an empirical starting point consistent with conventional high-similarity cutoffs in test redundancy literature. Stage 1 provides a formal transition-coverage guarantee and Stage 2 operates on the already-minimal set and preserves coverage in most cases, but can marginally reduce it in edge cases on complex models where the filter drops a test case that uniquely covered a transition after the set-cover reorganised the selection order.

Prioritisation Strategies

Both prioritisation strategies share the same greedy skeleton described in Section 3.3.1: a set of already covered transition IDs is maintained and grows with each selection. At each step, the unselected test case with the highest score over the remaining uncovered transitions is chosen and moved to the front of the ordered suite. The process repeats until all test cases have been placed. The two strategies differ only in how the score at each step is computed.

Coverage-Based Prioritisation scores each candidate by the count of its covered transitions whose IDs are not yet in the covered set, the standard additional-coverage count from regression testing literature [1]. The first selected test case is always the one that covers the most transitions. Subsequent selections favour tests that explore transitions not yet exercised. Once full transition coverage has been achieved, all remaining candidates have an additional coverage count of zero and are appended in their original relative order. Since the algorithm front-loads tests that cover the most new transitions, fault-detecting tests are pushed toward the front of the ordered sequence, directly optimising the APFD metric for any fault model that correlates with transition coverage.

Weighted-Transition Prioritisation replaces the count-based score with a weighted sum over the same uncovered transitions. Each transition t is assigned a weight equal to the inverse of its frequency across the full original suite:

$$w(t) = \frac{1}{|\{ tc \in \text{suite} \mid t \in tc.CoveredTransitions \}|}$$

Transitions that appear in only one test case receive weight 1.0 (maximum rarity) and transitions that appear in every test case receive weight $1/|\text{suite}|$ (minimum rarity). A transition not found in the frequency map defaults to 1.0. Weights are computed once over the original suite before the loop begins and remain fixed throughout.

As described in Section 3.3.1, the weighted-transition strategy is conceptually inspired by PSMT [31], which assigns domain-level weights to FSM transitions to reflect requirements criticality or functional risk. Since the benchmark FSM models used in this experiment carry no domain-specific weight annotations, the implementation derives weights from structural rarity in the suite instead. A transition exercised by few test cases is treated as more valuable to surface early than one exercised by many. When two candidates cover the same number of new transitions but one exercises a transition unique to that candidate and the other exercises only widely-shared transitions, the weighted strategy selects the former. Coverage-Based Prioritisation would treat the two as tied and resolve the choice arbitrarily.

4.1.3 Implementation and Experimental Control

Beyond its architectural structure, the artefact was implemented with explicit controls intended to preserve fairness and comparability across experimental runs. Since the objective

of the experiment is to compare the effect of coverage, reduction, and prioritisation strategies, it is important that all other relevant conditions remain as stable as possible.

One control concerns the computation of expected outputs for generated test suites. For each suite, the expected outputs are computed once and then reused in subsequent stages of evaluation. This decision ensures that observed differences between configurations are attributable to the selected strategies rather than to variations in support computations.

A second control concerns mutation analysis. Mutants are generated once for each FSM and then reused across all thirty two configurations associated with that model. This is a critical experimental safeguard because mutation score is one of the primary dependent variables in the study. Reusing a common mutant set for all configurations of a given FSM ensures that each strategy combination is exposed to the same fault-injection conditions and that mutation-based results remain directly comparable.

In mutation testing, a mutant is considered killed once any test case in the suite detects the injected fault by producing an output different from the original system, that status then persists for the remainder of that run. Since mutants are shared across all thirty two configurations of a given FSM, carrying this status across runs would cause earlier configurations to inflate the mutation scores of later ones. To prevent this, an *Is Killed* flag associated with each mutant is reset between runs, so that no mutant enters a subsequent configuration pre-marked as detected and each configuration is evaluated against the same clean mutation baseline. The mutation process itself incorporates six operators: *Transition Target Mutation*, *Output Mutation*, *Transition Removal*, *Transition Addition*, *Input Mutation*, and *Initial State Mutation*. Each operator, its structural effect, and its corresponding fault type are described in Section 3.3.2. The detailed two-phase sequence of the mutation testing process, including expected output caching and early exit on mutant detection, is illustrated in Appendix H.

Several implementation constraints are relevant to the interpretation of the mutation results. The *Output Mutation* operator produces no mutants for a given FSM when its output alphabet contains fewer than two distinct symbols, since there is no alternative output to substitute on any transition. Similarly, the *Input Mutation* operator produces no mutants when the input alphabet contains fewer than two distinct symbols. The *Transition Addition* operator applies a different constraint, where it skips any source state for which all input symbols already have an outgoing transition defined, that is, states with no free inputs. Where every input is already handled by an existing transition, no additional transition can be introduced without creating a non-deterministic machine, so the operator produces no mutant for that state. This means that the number and distribution of generated mutants may vary according to model characteristics, while still remaining consistent across all configurations evaluated for the same FSM.

In practical terms, the artefact is implemented so that differences in mutation score, coverage, APFD, test suite size, total steps, or computational cost can be interpreted as consequences of the evaluated configurations.

The complete interaction sequence across all participating components during a full experiment run, including the outer loop over FSMs, the inner loop over strategy combinations, mutant reuse and reset logic, metric evaluation, and CSV export, is illustrated in Appendix G.

4.1.4 Experimental Setup

This section describes the experimental setup used to evaluate the artefact. It covers the benchmark FSM models selected for evaluation, the factorial design that structures the strategy combinations, and the metrics collected for each experimental run.

Benchmark Selection

The evaluation uses five benchmark FSM models, three are derived from real-world communication protocols with well-established specifications, and two are synthetic models generated at controlled scales to assess algorithmic behaviour beyond the complexity of real protocol specifications. The structural range spans from 11 states and 19 transitions to 100 states and 400 transitions, allowing the evaluation to cover both the complexity levels typical of real protocol models and larger scales needed to examine scalability. Table 4.2 summarises the structural characteristics and source of each model.

FSM Model	States	Transitions	Source
TcpConnectionFSM	11	19	Reference [37]
TlsClientFSM	20	34	Reference [38]
MqttClientFSM	15	34	References [39, 40]
Synthetic50FSM	50	200	Programmatically generated (seed 42)
Synthetic100FSM	100	400	Programmatically generated (seed 137)

Table 4.2: Benchmark FSM models used in the evaluation

`TcpConnectionFSM` encodes the TCP connection lifecycle as specified in RFC 9293 [37]. The 11 states correspond directly to the connection lifecycle stages named in the RFC 9293 state diagram: CLOSED, LISTEN, SYN_SENT, SYN_RECEIVED, ESTABLISHED, FIN_WAIT_1, FIN_WAIT_2, CLOSE_WAIT, CLOSING, LAST_ACK, and TIME_WAIT. Paths involving RST signals and the simultaneous-open scenario were omitted to keep the model deterministic and completely specified, as required by the classical Mealy machine formalism adopted in this work. The TCP connection management diagram is one of the most widely reproduced FSM diagrams in networking literature, and deriving the model directly from the Internet Standard ensures reproducibility and academic traceability.

`TlsClientFSM` encodes the TLS 1.2 handshake from the client perspective, covering session establishment, optional resumption, renegotiation, and error and closure paths across 20 states and 34 transitions. The model is grounded in the literature [38], where active automata was applied to infer the state machines of real TLS implementations, establishing learned protocol FSMs as a legitimate artefact class for protocol testing research. This model is structurally more complex than `TcpConnectionFSM`, introducing greater behavioural diversity.

`MqttClientFSM` encodes the MQTT 3.1.1 client-side session lifecycle as specified in the OASIS standard [39]. The 15 states cover connection establishment, subscription management, QoS 0 fire-and-forget publish, QoS 1 publish with acknowledgement, QoS 2 two-phase publish and reception handshakes, keep-alive ping, graceful disconnect, and network error recovery. The adoption of the Mealy machine formalism for MQTT protocol behaviour is

supported by the literature [40], where active automata was applied to five MQTT 3.1.1 broker implementations, confirming that MQTT is an established benchmark in the FSM-based testing community.

`Synthetic50FSM` and `Synthetic100FSM` are programmatically generated using a deterministic pseudo-random algorithm with fixed seeds 42 and 137 respectively. Both share the same input alphabet $\{a, b, c, d\}$ and output alphabet $\{\text{out1}, \text{out2}, \text{out3}\}$. All states are reachable from the initial state and transitions for the remaining inputs are assigned pseudo-randomly. At 50 and 100 states respectively, these models are substantially larger than all three protocol benchmarks, enabling investigation of whether performance trends observed on smaller real-world models generalise to larger-scale inputs.

In the interest of transparency and in accordance with the ethical considerations stated in Section 1.6, it is noted that AI assistance was used in the construction of the five benchmark models. For `TcpConnectionFSM`, `TlsClientFSM`, and `MqttClientFSM`, AI assistance supported the translation of each model from its respective source specification to the JavaScript Object Notation (JSON) input format required by the prototype and all three models were subsequently verified by the author against the original sources. For `Synthetic50FSM` and `Synthetic100FSM`, AI assistance was used to generate the models under deterministic conditions (fixed seeds), and the author verified that both satisfy the structural properties described above. The JSON specification files for all five benchmark models are available in the project repository [7].

Experimental Design

The experiment follows a design in which four coverage strategies, four reduction strategies, and two prioritisation strategies are combined, as summarised in Table 4.3. This yields thirty two distinct configurations for each FSM model, and one hundred and sixty experimental runs in total across the five benchmark models. Each configuration is identified by a unique combination of one coverage strategy, one reduction strategy, and one prioritisation strategy, applied in sequence through the processing stages described in the previous subsection.

Coverage ($\times 4$)	Reduction ($\times 4$)	Prioritisation ($\times 2$)
State Coverage	Duplicate Removal	Coverage-Based Prioritisation
Transition Coverage	Mutation Score Preserving Reduction	Weighted-Transition Prioritisation
Transition Pair Coverage	Coverage Preserving Reduction	
Sequence Coverage(i)	Similarity Reduction	

Table 4.3: Experimental design: strategy dimensions and their combinations
($4 \times 4 \times 2 = 32$ configurations per FSM)

For each run, the following measurements are collected. Mutation score serves as the primary indicator of fault detection capability. State coverage and transition coverage assess the structure of the generated suite. APFD evaluates the ordering effectiveness of the prioritisation stage. Test suite size, expressed as the number of test cases remaining after reduction, characterises suite compactness. Total steps, expressed as the sum of transition firings across all test cases, captures execution cost independently of suite size. Generation time captures the computational overhead of each configuration. Finally, per-operator mutation score is computed individually for each of the six mutation operators.

Evaluation Metrics

This section defines how each collected measurement is computed in the experimental artefact.

Suite Size is the number of test cases in the suite at the point of evaluation, after reduction has been applied:

$$\text{Suite Size} = |\text{TestCases}|$$

Total Steps is the sum of transition firings across all test cases in the suite:

$$\text{Total Steps} = \sum_{tc \in \text{TestCases}} |tc.Steps|$$

where each step corresponds to one FSM transition firing. This metric serves as a proxy for execution cost that is independent of suite size. A suite of few long test cases may incur a higher total execution cost than a larger suite of shorter ones.

Reduction Ratio is the fraction of test cases eliminated by the reduction stage relative to the pre-reduction suite:

$$\text{Reduction Ratio} = 1 - \frac{\text{SuiteSize}}{\text{PreReductionSuiteSize}}$$

where *PreReductionSuiteSize* is the number of test cases produced by the coverage strategy before any reduction is applied, and *SuiteSize* is the number of test cases remaining after reduction. A value of 0 indicates that the reduction stage removed nothing and values approaching 1 indicate that nearly all test cases were eliminated. Since Duplicate Removal is unlikely to remove any test cases when the coverage strategy generates structurally distinct paths, this metric can be close to 0 for that strategy and higher for the more aggressive reduction strategies.

Execution-Time Savings is the fraction of transition firings eliminated by the reduction stage relative to the pre-reduction total:

$$\text{Execution-Time Savings} = 1 - \frac{\text{TotalSteps}}{\text{PreReductionTotalSteps}}$$

where *PreReductionTotalSteps* is the sum of transition firings across all test cases before reduction, and *TotalSteps* is the equivalent sum after reduction. This metric captures the reduction in execution cost independently of suite size where a suite consisting of fewer but longer test cases may incur lower savings than one consisting of many shorter cases. A value of 0 indicates no reduction in execution workload and higher values indicate proportionally greater savings.

State Coverage is the fraction of reachable states visited by at least one test case:

$$\text{State Coverage} = \frac{|\text{covered states}|}{|\text{reachable states}|}$$

The reachable state set is computed from the initial state via graph traversal. A state is counted as covered if it appears as the source state of any step in any test case, or as the final destination state of the last step of any test case. The denominator excludes states that are present in the FSM but unreachable from the initial state.

Transition Coverage is the fraction of FSM transitions exercised by at least one test case:

$$\text{Transition Coverage} = \frac{|\text{hit transitions}|}{|\text{FSM transitions}|}$$

The denominator is the total count of transitions defined in the FSM, including any that may be unreachable from the initial state under the selected generation strategy. A transition is counted as hit if its identifier appears in the covered-transition set of at least one test case.

Mutation Score is the fraction of generated mutants killed by the suite:

$$\text{Mutation Score} = \frac{\#km}{\#tm}$$

where $\#km$ is the number of mutants killed and $\#tm$ is the total number of generated mutants. A mutant is killed if at least one test case in the suite produces a different output sequence when executed against the mutated FSM, or if the mutated execution produces an error. This formula differs from the theoretical definition presented in Section 3.3.2, which excludes equivalent mutants from the denominator ($\#tm - \#em$). In the experimental artefact, equivalent mutant detection was not implemented and all generated mutants are counted in the denominator regardless of whether they are behaviourally distinguishable from the original FSM. As a consequence, the reported mutation scores represent a lower bound on the theoretically achievable score, since equivalent mutants that cannot be killed by any test case inflate the denominator without being killable.

Per-Operator Mutation Score applies the same formula restricted to the subset of mutants generated by each of the six mutation operators individually. If a given operator produces no mutants for the FSM under test (for example, `Input Mutation` on an FSM with a single input symbol), the per-operator score for that operator is undefined and reported as not applicable.

APFD measures how quickly the prioritised suite detects faults relative to a random ordering. For each mutant i killed by at least one test case, TF_i denotes the one-based index of the first test case in the current execution order that kills it. Let m be the count of mutants killed by at least one test case in the suite, and n the total number of test cases. Then:

$$\text{APFD} = 1 - \frac{\sum_{i=1}^m TF_i}{n \cdot m} + \frac{1}{2n}$$

Mutants not killed by any test case in the suite are excluded from m . If the suite is empty, there are no mutants, or $m = 0$, APFD is reported as 0. A key consequence of this definition is that reduction strategies which remove the only test case killing a mutant cause that mutant to drop out of m , which lowers APFD. This makes the cost of aggressive reduction directly visible in the metric.

Generation Time is the elapsed time in milliseconds for the complete test suite construction process, including path generation by the coverage strategy, path-to-test-case conversion, reduction, and prioritisation. These four steps are wrapped by a single timer, so the reported value reflects the full cost of producing the final optimised and ordered suite. Mutation analysis is excluded, as it runs separately and is not part of the suite construction cost. Generation time can therefore be used to compare the computational overhead of different strategy combinations directly.

To ensure comparability across all configurations, two execution controls are applied consistently throughout the experiment. First, mutants are generated once per FSM and reused across all thirty two configurations associated with that model, and second, expected outputs are computed once per test suite and reused in subsequent evaluation stages. Together, these controls ensure that any observed differences in measurement outcomes are attributed to the experimental variables rather than execution conditions.

4.1.5 Goal-Question-Metric Plan

The Goal-Question-Metric (GQM) approach [41] provides a structured mechanism for operationalising research objectives into measurable evaluation criteria. In the GQM model, a research goal is made precise by deriving a set of questions that define what must be known in order to determine whether the goal has been achieved. Each question is then associated with one or more concrete metrics, that is, observable and measurable properties that can be collected from experimental data. This three-level hierarchy ensures that every collected measurement serves a defined analytical purpose.

In the context of this dissertation, the GQM plan operationalises the evaluation of the experimental artefact and establishes a direct traceability link between the three research questions defined in Chapter 1 and the evaluation metrics collected in the experiment. Two evaluation goals are defined. The first goal addresses the fault detection capability and efficiency of the strategy combinations explored by the prototype, thereby operationalising RQ1. The second goal concerns the computational feasibility of integrating FSM-based test generation into automated delivery pipelines, which corresponds to RQ2. The third research question, RQ3, which concerns the criteria and methods that should guide the evaluation of an FSM-based test generation framework before implementation, is addressed by the GQM plan as a whole. The definition of goals, questions, and metrics presented here constitutes the evaluation methodology from which the answer to RQ3 is derived.

Goal G1 is defined as: *evaluate the fault detection capability and efficiency of FSM-based test generation strategy combinations, for the purpose of identifying which configurations produce the most favourable balance between mutation effectiveness, structural coverage, and suite compactness, from the viewpoint of a software tester, in the context of classical deterministic FSM models.* Four questions are derived from this goal. Question Q1.1 asks which coverage criterion produces the strongest fault detection capability, answered by the mean mutation score per criterion. Question Q1.2 asks what effect test suite reduction has on fault detection and compactness, answered jointly by the change in mutation score expressed in percentage points and the resulting suite size. Question Q1.3 asks how effective prioritisation is in surfacing faults early in the execution order, answered by the APFD metric. Question Q1.4 asks whether increasing coverage strength always yields a proportional improvement in mutation effectiveness, answered by comparing mutation score, transition coverage, and state coverage across criteria.

Goal G2 is defined as: *assess the computational feasibility of incorporating FSM-based test generation into modern software delivery pipelines, for the purpose of determining whether the approach can be adopted in a CI/CD context without incurring prohibitive overhead, from the viewpoint of a software engineer maintaining an automated pipeline.* One question is derived from this goal. Question Q2.1 asks whether the computational overhead of FSM-based test generation is acceptable for use within an automated pipeline, answered by the measured generation time and total execution time per configuration.

Table 4.4 presents the complete GQM plan, mapping each goal to its derived questions, the associated metrics, and the corresponding research question.

Goal	Question	Question Description	Metric	RQ
G1	Q1.1	Which coverage criterion produces the strongest fault detection capability?	Mean Mutation Score per criterion	RQ1
G1	Q1.2	What is the effect of test suite reduction on fault detection and compactness?	Mutation Score change (pp); Test Suite Size; Reduction Ratio; Execution-Time Savings	RQ1
G1	Q1.3	How effective is prioritisation at surfacing faults early in execution?	APFD	RQ1
G1	Q1.4	Does increasing coverage strength always yield a proportional improvement in mutation effectiveness?	Mutation Score; Transition Coverage; State Coverage	RQ1
G2	Q2.1	Is the computational overhead of FSM-based test generation compatible with automated pipeline constraints?	Generation Time; Execution Time	RQ2

Table 4.4: GQM plan for the experimental evaluation

The metrics listed in Table 4.4 correspond directly to the measurements collected during the experiment and reported in the subsections that follow, ensuring that the analysis presented in this chapter provides a complete and traceable response to each evaluation goal.

4.1.6 Coverage Criteria and Mutation Effectiveness

This section addresses Q1.1 and Q1.4 from the GQM plan. Results are presented separately for the three protocol benchmarks (TcpConnectionFSM, TlsClientFSM, and MqttClientFSM) and the two synthetic benchmarks (Synthetic50FSM and Synthetic100FSM), since the two groups differ substantially in size and structure. All values correspond to the Duplicate Removal baseline, isolating the effect of the coverage criterion before any reduction is applied. Tables 4.5 and 4.6 present the results for each group respectively.

Coverage Criterion	Mean Suite Size	Mean Trans. Cov. (%)	Mean Mutation Score	Mean APFD
State Coverage	14	50	0.39	0.87
Transition Coverage	29	100	0.76	0.85
Transition Pair Coverage	67	100	0.87	0.89
Sequence Coverage (3)	9	43	0.33	0.69

Table 4.5: Coverage criterion results for protocol FSMs, averaged across benchmark models (Duplicate Removal baseline)

Coverage Criterion	Mean Suite Size	Mean Trans. Cov. (%)	Mean Mutation Score	Mean APFD
State Coverage	74	25	0.12	0.88
Transition Coverage	300	100	0.31	0.84
Transition Pair Coverage	1 061	100	0.99	0.70
Sequence Coverage (3)	64	28	0.10	0.72

Table 4.6: Coverage criterion results for synthetic FSMs, averaged across benchmark models (Duplicate Removal baseline)

Across both groups, **Transition Pair Coverage** achieves the highest mean mutation score (0.87 for protocol FSMs and 0.99 for synthetic FSMs), providing a clear answer to Q1.1. This result holds consistently across all five benchmark models and confirms that requiring the suite to exercise every consecutive pair of transitions forces exploration of a broader set of behavioural interactions, exposing a larger fraction of the available mutants. On the protocol benchmarks it also produces the highest APFD (0.89), indicating that fault exposure is distributed favourably across the execution order.

State Coverage is confirmed as an insufficient primary criterion across both groups, with mutation scores of 0.39 and 0.12 for protocol and synthetic FSMs respectively. Reaching every state does not expose the transition-level faults that dominate the mutant set, particularly **Transition Target Mutation** and **Output Mutation**, which require specific input sequences to be triggered.

Transition Coverage occupies an intermediate position. On protocol FSMs it achieves a mean mutation score of 0.76, substantially above **State Coverage** and approaching **Transition Pair Coverage**. On the synthetic FSMs the score drops to 0.31, reflecting the difficulty of exercising diverse behavioural paths on large models.

Sequence Coverage(3) performs poorly on both groups, but the effect is especially pronounced on the synthetic FSMs, where it is the weakest criterion overall with a mean mutation score of 0.10. On the 50 and 100 state models, depth-3 paths cover only 20–36% of transitions, because the fixed maximum path length is too short to reach most states from the initial state in a single traversal. This is an expected limitation of the fixed-length parameterisation where its effectiveness depends on the path length being calibrated to the model under test. With a fixed parameter of $i = 3$, the criterion degrades to a weak baseline on large FSMs.

Addressing Q1.4, the answer depends on how coverage strength is measured. When defined as the transition coverage percentage actually achieved, the relationship with mutation score is proportional where the criteria that covers more transitions consistently detects more mutants. The criteria that reaches 100% transition coverage, **Transition Coverage** and **Transition Pair Coverage**, produces substantially higher mutation scores than those that do not. **State Coverage** and **Sequence Coverage (3)** achieve only 50% and 43% on protocol FSMs, and as little as 25% and 28% on synthetic FSMs, which directly explains their weaker mutation scores.

4.1.7 Reduction Trade-offs

This section addresses Q1.2 from the GQM plan. Tables 4.7 and 4.8 report the mean suite size reduction and mutation score change for each reduction strategy, averaged across all coverage criterion configurations within each FSM group. The baseline is the Duplicate

Removal configuration, which produces no reduction, as confirmed across all 160 experimental runs since the coverage strategies generate structurally distinct paths with no exact duplicates.

Reduction Strategy	Mean Size Reduction (%)	Mean MS Change (pp)
Duplicate Removal	0	0.0
Coverage Preserving Reduction	52	−2.0
Similarity Reduction	56	−5.0
Mutation Score Preserving Reduction	47	0.0

Table 4.7: Reduction strategy trade-offs for protocol FSMs, averaged across all coverage criterion configurations

Reduction Strategy	Mean Size Reduction (%)	Mean MS Change (pp)
Duplicate Removal	0	0.0
Coverage Preserving Reduction	43	−11.6
Similarity Reduction	57	−16.1
Mutation Score Preserving Reduction	3	0.0

Table 4.8: Reduction strategy trade-offs for synthetic FSMs, averaged across all coverage criterion configurations

On protocol FSMs, *Coverage Preserving Reduction* and *Similarity Reduction* both achieve substantial size reductions (52% and 56% respectively) at the cost of modest mutation score losses (−2.0 and −5.0 percentage points). The magnitude of the loss is sufficiently contained to regard reduction as a practical optimisation in this setting, where the efficiency gain clearly outweighs the fault detection cost.

On synthetic FSMs the aggregate losses are substantially larger (−11.6 and −16.1 percentage points). These figures are driven by a specific interaction effect. When *Transition Pair Coverage* is combined with either *Coverage Preserving Reduction* or *Similarity Reduction* on the 50 or 100 state models, the mutation score drops by approximately 44–62 percentage points (from ≈ 0.99 to ≈ 0.37 – 0.55). This occurs because preserving transition coverage does not guarantee preserving the test cases responsible for killing mutants. On large FSMs, many mutants require specific multi-transition sequences to be exercised, and a suite that covers every transition individually may still fail to include those sequences once redundant test cases are removed. For the remaining coverage criteria on synthetic FSMs, mutation score changes remain small (typically below 5 percentage points), confirming that the severe loss is confined to this specific strategy combination.

Mutation Score Preserving Reduction is unique among the four strategies in preserving mutation score exactly (0.0 pp change in all 160 configurations). This makes it the only reduction strategy that provides a guaranteed null effect on fault detection. However, it achieves this at the cost of substantially lower reduction ratios on synthetic FSMs (mean 3%), compared to approximately 47% on protocol FSMs. On the large models, the scoring function evaluates a large number of candidate subsets and consistently converges on solutions that remove very few test cases, making it effectively equivalent to *Duplicate Removal* in practice for those benchmarks. The implications of this behaviour for scalability are discussed further in Section 4.1.10.

4.1.8 Prioritisation Effects

This subsection addresses Q1.3 from the GQM plan. Across both FSM groups and all coverage and reduction configurations, the two prioritisation strategies produce APFD values that differ by less than two percentage points. Neither strategy consistently dominates the other. In some configurations Coverage-Based Prioritisation achieves a marginally higher APFD, while in others Weighted-Transition Prioritisation produces slightly better results, and in many configurations the values are identical to four decimal places. On average, Coverage-Based Prioritisation shows a marginal advantage, most visible in configurations where the suite retains full transition coverage after reduction.

The small magnitude of the difference confirms that prioritisation operates as a secondary refinement rather than a primary determinant of fault detection ordering. The APFD outcome is shaped primarily by the coverage criterion and the reduction strategy, which together determine the composition and size of the suite that prioritisation receives as input. Once those factors are fixed, the choice of prioritisation strategy introduces at most a marginal adjustment.

This interpretation is especially relevant when compared with the differences observed between coverage criteria. Whereas the choice of coverage strategy produced variation in mutation score and suite characteristics, the choice of prioritisation strategy produced only small changes in APFD.

For the purposes of this dissertation, prioritisation should be treated as a complementary optimisation stage whose benefits are real but limited, and the choice between the two strategies is less consequential than the choice of coverage criterion.

4.1.9 Mutation Operator Analysis

This section examines the mutation testing component of the experimental evaluation in detail. It first describes how each of the six operators generates its mutant set, and then analyses the fault detection results disaggregated at operator level, showing that the aggregate mutation score contains significant variation that is only visible when operators are considered individually.

Mutant Generation

The experimental framework applies the six mutation operators described in Section 3.3.2 exhaustively to each benchmark FSM. The generation rule for each operator is as follows.

Transition Target Mutation generates one mutant per transition per alternative target state. For each transition (s_i, x, y, s_j) , one mutant is produced for every state $s_k \neq s_j$, replacing only the target with s_k while the source state, input, and output remain unchanged. The total mutant count is $|T| \times (|S| - 1)$, where $|T|$ is the number of transitions and $|S|$ the number of states.

Output Mutation requires at least two distinct output symbols to be present in the FSM. If fewer exist, no mutants are produced, as substituting a label not drawn from the FSM's own output alphabet would not create realistic faults. Otherwise, for each transition (s_i, x, y, s_j) , one mutant is generated per alternative output symbol $y' \neq y$, replacing only the output while the source state, target state, and input are preserved. The total mutant count is $|T| \times (|O| - 1)$, where $|O|$ is the number of distinct output symbols.

Transition Removal produces one mutant per transition by removing that transition entirely, leaving all remaining transitions unchanged. The total mutant count equals $|T|$.

Transition Addition produces one mutant per valid (source state, target state) pair in which at least one *free input* exists for the source state. A free input is any symbol in the FSM's input alphabet that does not already have an outgoing transition defined from that state and states with no free inputs are skipped. Each mutant appends one new transition using the first available free input and a fixed synthetic output label, ensuring the added transition is reachable and does not shadow an existing one. If $|F|$ denotes the number of states that have at least one free input, the total mutant count is $|F| \times (|S| - 1)$.

Input Mutation applies the same constraint as **Output Mutation**: if fewer than two distinct input symbols exist in the FSM, no mutants are produced. Otherwise, for each transition (s_i, x, y, s_j) , one mutant is generated per alternative input symbol $x' \neq x$, replacing only the input label while the source state, target state, and output are preserved. The total mutant count is $|T| \times (|I| - 1)$, where $|I|$ is the number of distinct input symbols.

Initial State Mutation generates one mutant per non-initial state. Each mutant designates a different state $s_k \neq s_0$ as the new initial state while preserving the full transition structure. The total mutant count is $|S| - 1$.

Fault Detection per Operator

Disaggregating the mutation results by operator reveals patterns that are consistent across benchmark groups but differ in magnitude. Figure 4.2 shows the mean score per operator for each FSM group separately, averaged across all coverage strategies using the Duplicate Removal baseline.

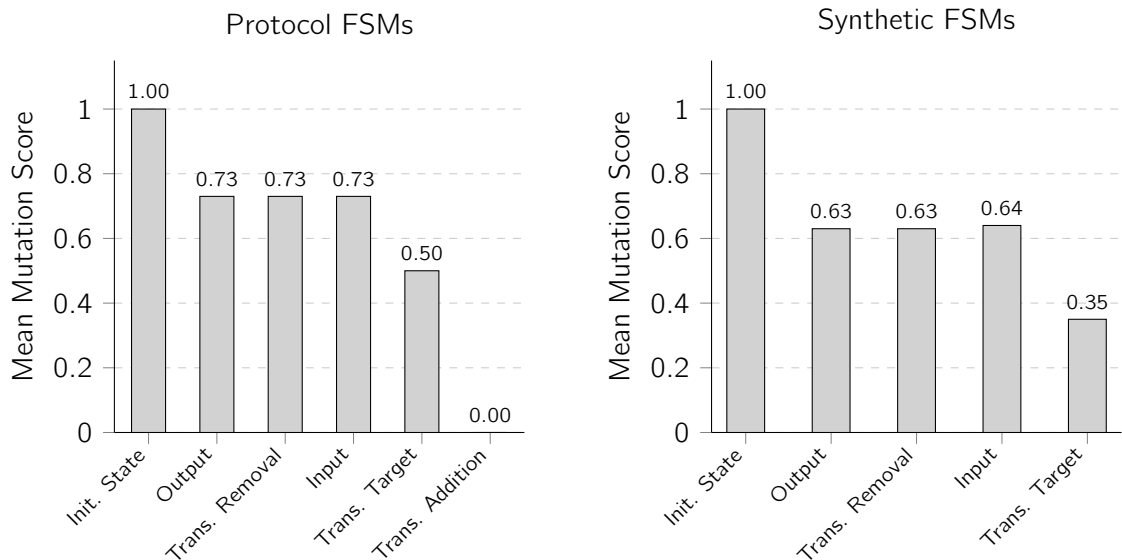


Figure 4.2: Mean mutation score per mutation operator, by FSM group (Duplicate Removal baseline).

The most consistent finding across both groups is that **Initial State Mutation** is killed by every configuration, achieving a score of 1.000 universally. This operator designates a non-initial state as the new start state while preserving all transitions. Since any generated

test case begins at the initial state, even a single step immediately exposes the incorrect starting behaviour, making these mutants trivially detectable regardless of coverage criterion.

Transition Addition behaves differently between the two groups. For protocol FSMs, the operator generates mutants because free inputs exist in these partially specified models, but achieves a score of 0.000 across all configurations. Coverage-based generation only constructs test cases that exercise defined transitions in the original FSM, so free inputs are never included in any generated path and newly added transitions are never triggered. For synthetic FSMs, which are completely specified with every state-input pair defined, no free inputs exist and no *Transition Addition* mutants are generated, making the operator not applicable to this group.

Transition Target Mutation is the most discriminating operator across both groups. On protocol FSMs its mean score across all coverage strategies is approximately 0.50, ranging from 0.20 (*State Coverage* on small models) to 1.00 (*Transition Pair Coverage* on *TcpConnectionFSM* and *MqttClientFSM*). On synthetic FSMs the mean drops to approximately 0.35, reflecting the difficulty of reaching specific mutant-triggering sequences in larger models. Notably, *Transition Pair Coverage* achieves near-perfect detection (0.99–1.00) for this operator even on the synthetic FSMs, confirming that pairwise transition coverage is particularly effective at exposing target-substitution faults.

Output Mutation, *Transition Removal*, and *Input Mutation* form a group of moderately detectable operators with similar scores. On protocol FSMs each achieves a mean of approximately 0.73 across all coverage strategies; on synthetic FSMs the group mean is approximately 0.63. For all three operators, *Transition Coverage* and *Transition Pair Coverage* achieve near-perfect scores (1.00), while *State Coverage* and *Sequence Coverage*(3) produce lower values proportional to the fraction of transitions they exercise.

These findings confirm that the aggregate mutation score is an insufficient summary statistic. The score of 1.000 contributed by *Initial State Mutation* inflates the aggregate, while the 0.000 of *Transition Addition* deflates it. Neither operator provides meaningful discrimination between coverage strategies, and both are artefacts of the operator composition rather than evidence of detection capability. Operator-level disaggregation is therefore necessary to assess whether a strategy’s mutation effectiveness is substantive or an artefact of the mutation mix.

4.1.10 Computational Cost

This subsection addresses Q2.1 from the GQM plan. The computational behaviour of the prototype differs substantially between the two groups of benchmark models and between *Mutation Score Preserving Reduction* and all other strategies.

For the three protocol FSMs, all configurations not using *Mutation Score Preserving Reduction* complete in under 5 ms regardless of coverage criterion or reduction strategy. *Mutation Score Preserving Reduction* adds overhead on these models but remains within acceptable bounds, reaching at most approximately 570 ms (*TlsClientFSM* with *Transition Pair Coverage*). These timings indicate that the full evaluation workflow is computationally negligible for FSMs of this scale.

For the synthetic FSMs, all strategies other than *Mutation Score Preserving Reduction* complete in under 500 ms even for *Synthetic100FSM* with *Transition Pair Coverage*, where the pre-reduction suite contains 1 410 test cases. This confirms that coverage generation,

coverage-preserving and similarity-based reduction, and both prioritisation strategies all scale acceptably to models of this size.

Mutation Score Preserving Reduction exhibits a substantial increase in generation time as model size grows. On *Synthetic50FSM*, times range from approximately 1.4 seconds (*State Coverage*) to approximately 5 seconds (*Transition Pair Coverage*). On *Synthetic100FSM* the same strategy requires between approximately 13 seconds (*Sequence Coverage(3)*) and approximately 46 seconds (*Transition Pair Coverage*), as detailed in Table 4.9.

Coverage Criterion	Synthetic50FSM (ms)	Synthetic100FSM (ms)
State Coverage	1 376–1 520	21 696–23 597
Transition Coverage	2 081–2 163	22 852–25 409
Transition Pair Coverage	4 954–4 971	45 553–45 931
Sequence Coverage (3)	1 552–1 557	12 765–16 051

Table 4.9: *Mutation Score Preserving Reduction* generation time on synthetic FSMs (range across both prioritisation strategies)

These results indicate that *Mutation Score Preserving Reduction* does not scale to large FSMs within constraints compatible with CI/CD integration. The growth in evaluation cost as suite size increases makes this strategy practical only for small FSMs or in contexts where time constraints are relaxed. This scalability limitation, combined with the finding from Section 4.1.7 that *Mutation Score Preserving Reduction* achieves minimal suite size reduction on synthetic FSMs (mean 3%), substantially narrows the conditions under which this strategy is the appropriate choice.

The multi-criteria assessment framework, the operator-level mutation analysis, and the experimental design together constitute the empirical contribution through which the third research question, concerning the evaluation of FSM-based test generation approaches, is addressed.

4.2 Design Implications for the Final Artefact

The experimental evaluation provides a clear basis for refining the design of the final artefact. The results can be interpreted as design evidence regarding which strategic choices contribute most effectively to the objectives of the dissertation. In this sense, the final artefact should not simply reproduce all strategies explored in the experimental prototype with equal emphasis, but should instead embody the design principles supported by the evaluation.

4.2.1 Coverage Criteria

The first implication concerns the selection of coverage criteria. The experimental results show that *State Coverage* consistently produced weak mutation effectiveness across all benchmark models, establishing it as an insufficient primary adequacy criterion when fault detection is the primary objective. *Transition Pair Coverage* demonstrated the strongest and most consistent fault detection performance, making it the most defensible default criterion for the final artefact.

The second implication follows from *Sequence Coverage(3)*. Although the criterion is theoretically more demanding than transition coverage, the experimental results show that a

fixed path length of $i = 3$ is insufficient to explore the behavioural space of larger FSMs, producing the weakest mutation scores in the synthetic benchmark group. The final artefact should therefore expose the sequence length as a configurable parameter, allowing practitioners to calibrate it to the diameter of the model under test rather than relying on a fixed value.

4.2.2 Reduction

The results on reduction suggest a different design conclusion. On protocol-scale FSMs, *Coverage Preserving Reduction* and *Similarity Reduction* both deliver substantial size reductions at a modest fault detection cost, and either is suitable as a default reduction strategy. On larger FSMs, however, both strategies can cause severe mutation score drops when combined with *Transition Pair Coverage*, due to the interaction effect described in Section 4.1.7. The final artefact should therefore expose the choice of reduction strategy as an explicit configuration option and make the pre and post-reduction mutation scores visible in the output, so that practitioners can assess the cost of reduction for their specific model and criterion combination. *Mutation Score Preserving Reduction* uniquely guarantees zero mutation score loss but scales poorly to large models and it should be offered as an option for small-scale contexts where its computational cost is acceptable.

4.2.3 Prioritisation

The evaluation of prioritisation suggests a more restrained design position. Because the observed differences between prioritisation strategies were limited, prioritisation should be included in the final artefact as a complementary optimisation layer rather than as a central design driver.

4.2.4 Mutation

The mutation analysis introduces an additional implication for the final artefact. Since aggregate mutation score can be distorted by mutants that are never killable under a given generation strategy, the final artefact should be designed to support more transparent interpretation of results. Concretely, this means that before a mutant is included in the denominator of the mutation score (as defined in Equation 3.2), it must first be classified as either *equivalent*, behaviourally indistinguishable from the original FSM for all possible inputs, or *unreachable under the chosen generation strategy*, not exercisable by any test case the strategy can produce. Only mutants that are genuinely killable should contribute to the denominator. Including unkillable mutants artificially lowers the score and misrepresents the true fault detection capability of the suite. Evaluation outputs should therefore not rely exclusively on a single aggregate mutation score, but should instead preserve visibility into operator-level behaviour so that unkillable mutants can be identified and excluded before the score is computed.

Taken together, these findings imply that the final artefact should be guided by a design principle of balanced effectiveness.

4.3 Integration into CI/CD Workflows

A natural extension of the FSM-based test generation framework presented in this dissertation is its integration into automated software delivery pipelines. In a CI/CD workflow, the

test generation process would be triggered automatically whenever the behavioural model of the system under development is updated. This could occur following a commit that modifies the FSM specification, or as part of a model validation stage within the build pipeline.

Within a typical CI/CD pipeline, the test generation and execution stages would be positioned after compilation and model validation, and before any deployment stage. The generated test suite would be executed against the current build, with results recorded and reported as part of the standard pipeline feedback cycle. This arrangement ensures that regressions introduced by model changes are detected.

The prioritisation and reduction strategies explored in this dissertation are relevant in a CI/CD context. Prioritisation determines the execution order of test cases, enabling faults to be surfaced as early as possible within a pipeline run. In an environment in which developers expect fast results, the ability to detect failures before the full suite has completed provides practical value. Reduction strategies contribute by limiting the number of test cases submitted for execution, thereby reducing the total execution time of the pipeline stage.

Integration of FSM-based test generation into CI/CD workflows is not implemented within the experimental artefact developed in this dissertation. The prototype was designed exclusively as an instrument for controlled strategy evaluation. CI/CD integration is therefore treated as a design objective of the final artefact.

4.4 Strategy Selection

Based on the results discussed throughout this chapter, the final system should adopt a selective combination of strategies.

4.4.1 Coverage

The evidence shows that *State Coverage* should be avoided as a primary generation strategy, since its mutation effectiveness was consistently too weak to support the fault detection objectives of the dissertation. *Transition Coverage* is a viable choice and produces strong mutation scores, particularly on protocol-scale FSMs. However, *Transition Pair Coverage* demonstrated the strongest and most consistent fault detection across both benchmark groups and should therefore be the default criterion for the final artefact.

For sequence-based coverage, the experimental results show that a fixed path length is insufficient for larger FSMs, and the final artefact should expose the sequence length as a configurable parameter. With this adjustment, sequence-based strategies become a complementary option for practitioners who require path-depth guarantees.

4.4.2 Reduction

The final system should include a reduction stage as a standard part of the workflow, but the appropriate default strategy depends on the scale and characteristics of the model under test. On protocol-scale FSM models, *Coverage Preserving Reduction* and *Similarity Reduction* both delivered substantial suite size reductions of approximately 52% and 56% respectively, at a modest fault detection cost of 2-5 percentage points in mutation score. On larger synthetic FSM models, however, the same strategies can cause drops of 11-16 percentage points when combined with *Transition Pair Coverage*, due to the interaction effect described in Section 4.1.7. Reduction should therefore be offered as a configurable choice rather than

applied unconditionally, and the final artefact should surface both pre and post-reduction mutation scores so that practitioners can assess the cost of reduction for their specific model and criterion combination.

4.4.3 Prioritisation

With regard to prioritisation, neither strategy consistently dominated the other across the full set of configurations. Both *Coverage-Based Prioritisation* and *Weighted-Transition Prioritisation* produced similar APFD values, with differences that were modest and did not favour one approach across all benchmark models and coverage criteria. Both strategies should therefore be included in the final artefact as equivalent options, and practitioners should select between them based on the specific model context rather than a fixed default.

Overall, the strategy selection for the final artefact should therefore reflect the effects observed in this chapter, meaning the final system remains grounded in the experimental evidence rather than in purely theoretical preference. In doing so, this section directly addresses the first research question by providing concrete, evidence-based recommendations for optimising FSM-based test generation with respect to coverage strength, suite size, and fault detection capability.

4.5 Improvements Over the Experimental Artefact

From a DSR perspective, the final artefact represents an iterative refinement of the experimental prototype rather than a separate or unrelated solution. The purpose of the prototype was to make the relevant design alternatives observable under controlled conditions, whereas the purpose of the final artefact is to incorporate those findings into a system that is more suitable for practical use.

The artefact design at this stage had the objective to compare strategies, while the final artefact should be projected to be used in real world environments. This means that its architecture and workflow should support more robust integration into software engineering practice, including compatibility with CI/CD processes.

The experimental evaluation identified several limitations, which lead to design improvements. Mutation results can be misrepresented when mutants are unkillable not because they are equivalent to the original FSM, but because no test case produced by the chosen coverage criterion can ever exercise the injected fault. This was observed with the *Transition Addition* operator on protocol-scale FSMs where the added transitions involve free inputs that lie outside the reachable alphabet of any coverage criterion, making these mutants strategy-unreachable rather than behaviourally equivalent. As a result, they contributed a score of zero in all configurations without providing any discriminating signal. The final artefact should therefore automatically classify *Transition Addition* mutants as strategy-unreachable when all free inputs are inaccessible under the active coverage criterion, and exclude them from the denominator of the mutation score on that basis, in line with the mutant classification framework described in Section 4.2.

The reduction analysis identified a significant interaction effect where *Coverage Preserving Reduction* and *Similarity Reduction* can cause mutation score drops of 11-16 percentage points when combined with *Transition Pair Coverage* on large FSMs. This is not a minor edge case but a systematic consequence of aggressive coverage preservation discarding the path-diversity that *Transition Pair Coverage* relies upon to detect faults. The final artefact should

include explicit reporting of pre and post-reduction mutation scores, so that practitioners are made aware of this cost rather than observing only a reduced suite size.

Finally, *Mutation Score Preserving Reduction* exhibited unacceptable generation times on larger models, reaching up to approximately 46 seconds on Synthetic100FSM. This scalability limitation means the strategy is not suitable as a default for large FSMs. The final artefact should either restrict *Mutation Score Preserving Reduction* to small-scale contexts or include a time-budget parameter that halts the search once a specified limit is reached, returning the best solution found so far.

Taken together, these improvements demonstrate how the insights obtained from the experimental artefact impact the design of the final system. This progression reflects the central contribution of DSR where knowledge is generated through artefact construction and evaluation. With regard to the second research question, concerning the integration of FSM-based test generation into CI/CD workflows, the computational cost observed across all configurations and the strategy selection principles established in Section 4.4 lay the base for the design of the final artefact.

4.5.1 Requirements

The design improvements identified from the experimental evaluation translate into a set of functional and non-functional requirements for the final artefact. Table 4.10 lists these requirements. They extend and refine those of the experimental artefact (Table 4.1), reflecting both the capabilities that were validated and the limitations that were identified.

ID	Type	Requirement
FR1	Functional	Accept a deterministic FSM as input and produce test sequences from it
FR2	Functional	Support configurable coverage strategies, with the sequence length exposed as a configurable parameter for <i>Sequence(i)</i> coverage
FR3	Functional	Support configurable reduction strategies, with an FSM size threshold for <i>Mutation Score Preserving Reduction</i> that warns the practitioner when significant computational overhead is expected
FR4	Functional	Support configurable prioritisation strategies: coverage-based and weighted-transition
FR5	Functional	Classify generated mutants as killed, strategy-unreachable, or surviving prior to computing the mutation score, and exclude strategy-unreachable mutants from the denominator. Equivalent mutants, if present, fall into the surviving class.
FR6	Functional	Report evaluation metrics per configuration, including pre and post-reduction mutation score, per-operator mutation score, coverage, APFD, suite size, total steps, and computational cost
FR7	Functional	Integrate with CI/CD pipelines by triggering test suite generation automatically when the FSM model specification is updated
NFR1	Non-functional	Deterministic and reproducible execution to ensure predictable behaviour across pipeline runs
NFR2	Non-functional	Extensible architecture allowing new strategies to be integrated without modifying the existing processing stages
NFR3	Non-functional	Configurable strategy selection with <i>Transition Pair Coverage</i> as the default coverage criterion
NFR4	Non-functional	Scalable execution compatible with CI/CD time constraints for all strategies across FSMs up to the scale evaluated in the experimental prototype
NFR5	Non-functional	Transparent reporting of operator-level mutation scores and pre and post-reduction metrics to support informed practitioner decisions

Table 4.10: Functional and non-functional requirements of the final artefact

Chapter 5

Final Artefact

The final artefact is the direct outcome of the DSR design-evaluation cycle completed in Chapter 4. It consolidates the validated design choices from the experimental prototype into a tool as a single executable that accepts a deterministic FSM model and a configuration file, generates a test suite according to the specified strategy combination, and produces structured JSON outputs ready for consumption in a CI/CD pipeline.

The chapter is organised as follows. Section 5.1 presents the design of the final artefact, covering its component architecture, technology selection, and processing workflow. Section 5.2 describes the mutant classification framework introduced as a key improvement over the experimental prototype. Section 5.3 addresses CI/CD integration. Section 5.4 summarises the improvements the final artefact introduces over the experimental prototype.

5.1 Artefact Design

This section describes the design of the final artefact. It covers the component architecture that structures its implementation, the technology choices, and the processing workflow that constitutes a complete tool invocation.

5.1.1 Architecture

The final artefact is implemented as a single .NET 8 executable and builds directly on the experimental prototype's codebase. All ten strategy implementations and all six mutation operator implementations are carried over without modification, as is the core domain model (*FSM*, *State*, *Transition*) and the test model types (*TestSuite*, *TestCase*, *TestStep*). This reuse was made possible by the Strategy pattern adopted in the prototype, which isolated each algorithm behind an interface and allowed the implementations to be lifted into the final artefact with no changes to the consuming components, directly satisfying NFR2 from Section 4.5.1. The components introduced for the first time in the final artefact are *Program*, *ToolConfiguration*, *FsmJsonLoader*, *FsmValidator*, *TestGenerationPipeline*, *MutantClassifier*, *TestSuiteExporter*, and *MetricsReportExporter*. The component architecture is illustrated in Appendix I.

The root contains *Program*, as the Command-Line Interface (CLI) entry point. *Program* carries no domain knowledge and it resolves the configuration file path (using the first command-line argument if provided, otherwise falling back to *toolconfig.json* in the working directory), deserialises the file into a typed *ToolConfiguration* object (defined in the *Configuration* namespace), loads the FSM model via *FsmJsonLoader* (defined in the *IO*

namespace), and hands control to *FsmValidator* and then to *TestGenerationPipeline*. All generation logic resides in the *Pipeline* namespace.

The *Core* namespace provides the domain model, composed of three types: *FSM*, *State*, and *Transition*. These are the structural foundation shared by all other namespaces. *FsmJsonLoader* deserialises a JSON model file into a *Core.FSM* instance and any state referenced in a transition definition but absent from the *states* array is created automatically during loading.

Input Validation

Structural correctness is asserted by *FsmValidator* (defined in the *Validation* namespace), which performs five sequential checks before any generation work begins. The first three are hard errors that terminate execution with exit code 1: the state set must be non-empty, the declared initial state must be present in the state set, and no state may have two outgoing transitions on the same input symbol. The fourth check, reachability, is a non-blocking warning: any state unreachable from the initial state via BFS is listed in a warning that is forwarded through the pipeline and included in the output report, but does not halt execution. A fifth, performance-oriented warning is emitted if the active reduction strategy is *Mutation Score Preserving Reduction* and the transition count of the FSM exceeds the configured *fsmSizeThreshold* (default 50); this informs the practitioner of the expected computational overhead without blocking execution, addressing the scalability concern identified in FR3 from Section 4.5.1.

Strategies

The *Strategies* namespace defines three interfaces and their ten concrete implementations, summarised in Table 5.1, satisfying FR2, FR3, and FR4 from Section 4.5.1.

Interface	Count	Concrete Implementations
ICoverageStrategy	4	State Coverage, Transition Coverage, TransitionPair Coverage, Sequence Coverage
IReductionStrategy	4	Duplicate Removal, Coverage Preserving Reduction, Similarity Reduction, Mutation Score Preserving Reduction
IPrioritizationStrategy	2	Coverage-Based Prioritisation, Weighted-Transition Prioritisation

Table 5.1: Strategy interfaces and their concrete implementations

The *Engine* namespace provides two shared primitives. *TestGenerator*, which wraps the active coverage strategy and converts raw transition paths into *TestCase* objects defined in the *TestModel* namespace, and *FSMExecutor*, the simulation process used by both *MutationTester* and the APFD computation to walk a test case step by step against an FSM. If a step requests an input symbol for which no transition is defined at the current state, *FSMExecutor* appends the token *ERROR* and exits. The presence of this token in the output sequence counts as a kill signal during mutation testing.

Mutation Analysis

The *Mutation* namespace contains four collaborating types. *MutationEngine* holds a list of six *IMutationOperator* instances and calls *Apply(fsm)* on each in turn, collecting all returned *Mutant* objects into a flat list. A *Mutant* wraps a deep copy of the original FSM with one fault injected, the name of the generating operator, and a description. *MutationTester* consumes the mutant list together with a test suite. It first runs the original FSM against every test case to build a map of expected output sequences keyed by test-case identifier, then replays each test case against every mutant via *FSMExecutor*, marking a mutant as killed as soon as any test case produces a diverging output or triggers the error token. *MutantClassifier* partitions the annotated mutant list into three classes, which are described in detail in Section 5.2.

A key structural feature of the final artefact is that *MutationTester* is invoked twice within a single pipeline run: once on the raw, unreduced suite to establish a pre-reduction baseline mutation score, and once on the final, reduced and prioritised suite to confirm the post-reduction score. Between the two runs, the *IsKilled* flag is reset on all mutants so that each run starts from a clean baseline. This dual-run pattern is a direct response to the experimental finding that certain reduction strategies caused measurable mutation score drops, as reported in Section 4.1.7. Making both scores visible in the output allows practitioners to assess the fault-detection cost of the chosen reduction strategy before accepting the reduced suite.

Output Serialisation

The *Output* namespace contains *TestSuiteExporter* and *MetricsReportExporter*, which serialise the in-memory *GenerationReport* into *test_suite.json* and *metrics_report.json* respectively in the configured output directory.

The central architectural departure from the experimental prototype is the elimination of the *ExperimentRunner* component. The prototype drove a full-factorial loop over all thirty-two combinations of coverage, reduction, and prioritisation strategies across five benchmark models, accumulating results into a single CSV file. Its purpose was scientific comparison under controlled conditions. The final artefact instead executes a single configuration per invocation, as declared in *toolconfig.json*. This shift reflects the change in role from a research instrument to a deployable engineering tool. To evaluate a different configuration, the practitioner edits *toolconfig.json* and re-runs the tool. The full-factorial CSV format is replaced by two structured JSON files designed for programmatic consumption, whose schemas are described in Section 5.3.

5.1.2 Technology Selection

The technology stack of the final artefact is inherited directly from the experimental prototype, which is consistent with the class reuse described in Section 5.1.1. The implementation language is C# targeting .NET 8, for the same reasons established in Section 4.1.1. The strategy implementations, mutation operators, and domain types that were reused from the prototype required no changes to compile and execute correctly within the final artefact.

System.Text.Json is retained as the JSON serialisation library, consistent with the experimental prototype. It is part of the .NET 8 standard library, requires no additional NuGet

package, and is sufficient for the structured but straightforward JSON schemas used by both *toolconfig.json* and the two output files.

The output format is a direct departure from the prototype. The prototype accumulated all 160 experimental results into a single CSV. The final artefact instead produces two structured JSON files. *test_suite.json*, which contains the ordered test suite ready for execution, and *metrics_report.json*, which contains the full evaluation report for the run. JSON was chosen over CSV because the output is hierarchical rather than tabular since a test suite consists of nested test cases and steps and because JSON is directly consumable by CI/CD pipeline tooling without requiring a parser for a flat format. The schemas of both files are described in Section 5.3.

Unit testing is performed using NUnit and NSubstitute, enabling each strategy module to be verified in isolation against known FSM inputs.

5.1.3 Processing Workflow

A complete tool invocation proceeds through eight stages, from startup to output serialisation, fulfilling FR1 from Section 4.5.1. Figure 5.1 illustrates these stages and their ordering.

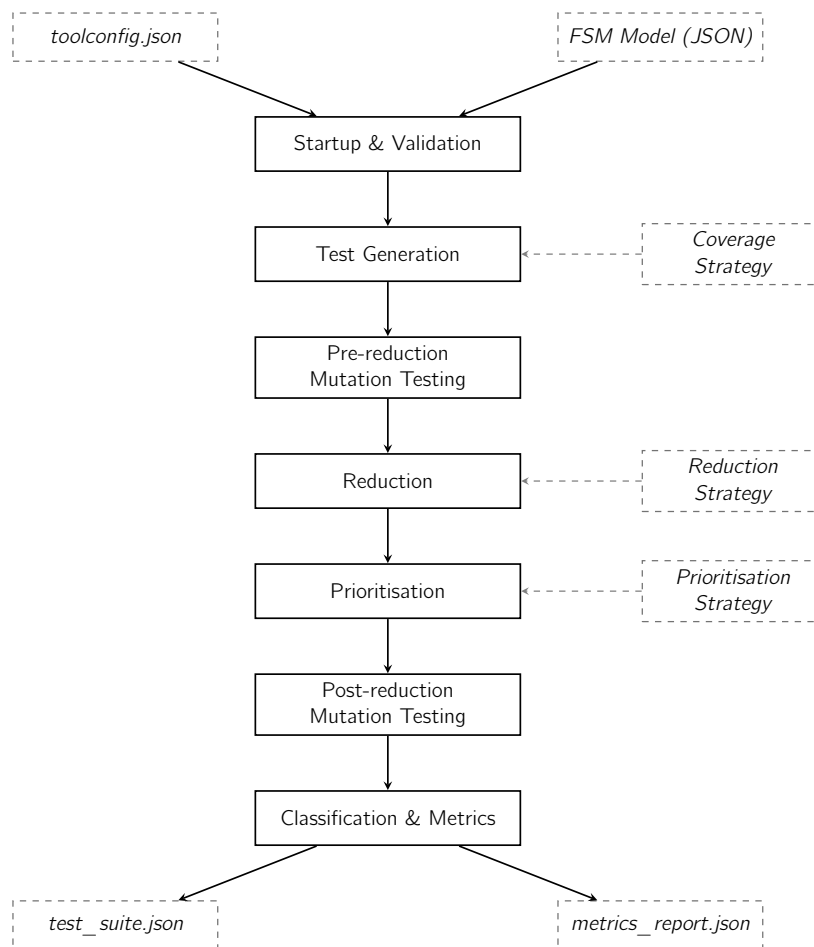


Figure 5.1: Processing workflow of the final artefact

The first stage is startup. *Program* reads *toolconfig.json* to obtain the FSM model path, output directory, and strategy configuration, then loads the FSM model and passes it to

FsmValidator. If any hard error is detected, execution terminates at this point and no output files are written. The detailed startup sequence is illustrated in Appendix J.

Once validation passes, *TestGenerationPipeline* takes over. It resolves the three strategy instances from the configuration and generates the raw test suite via the coverage strategy. It then generates the full mutant set via *MutationEngine* and runs the first mutation testing pass on the raw suite, recording the pre-reduction mutation score as a baseline before any modification to the suite.

With the baseline established, the reduction strategy is applied to produce a smaller suite, followed by the prioritisation strategy to order its test cases. The generation timer spans these three stages, from raw test generation through prioritisation. The mutation testing passes are excluded from this timer because they represent evaluation overhead rather than generation cost.

After prioritisation, a second mutation testing pass is run on the final suite. *MutantClassifier* then partitions the annotated mutants into three classes, as described in Section 5.2, and the full set of metrics is computed. Finally, *TestSuiteExporter* and *MetricsReportExporter* serialise the results to *test_suite.json* and *metrics_report.json* in the configured output directory. The detailed sequence of interactions across all participating components during a complete run is illustrated in Appendix K. The workflow is fully deterministic where given the same *toolconfig.json* and the same FSM model, every invocation follows identical stages and produces identical outputs, satisfying NFR1 from Section 4.5.1.

5.2 Mutant Classification

The experimental prototype used a binary killed/alive classification for mutants, where any mutant not killed by the test suite was counted in the denominator of the mutation score regardless of whether it was structurally unkillable. As identified in Section 4.5, this caused *Transition Addition* mutants on protocol-scale FSMs to deflate the reported score without providing any discriminating signal, since those mutants add transitions on inputs absent from the original model and can never be triggered by any generated test case. The final artefact addresses this by introducing *MutantClassifier*, which partitions every mutant into exactly one of three disjoint classes before the mutation score is computed, as defined in Table 5.2, realising FR5 from Section 4.5.1.

Class	Condition	Meaning
Killed	<code>IsKilled == true</code>	Detected by at least one test case in the final suite
Strategy-unreachable	<code>IsKilled == false && OperatorName == "TransitionAddition"</code>	Structurally unkillable under any coverage strategy derived from the original FSM
Surviving	<code>IsKilled == false && not strategy-unreachable</code>	Not killed and not strategy-unreachable. Represents either a genuine detection gap or an equivalent mutant not separately identified, since equivalent mutant detection is not implemented

Table 5.2: Mutant classification classes in the final artefact

This classification differs from the preliminary framework described in FR5 in one important respect. FR5 was initially formulated in terms of potential killability, distinguishing equivalent mutants (behaviourally indistinguishable from the original) from strategy-unreachable ones (unreachable under the chosen strategy) and from genuinely killable mutants. After the test suite has run, a mutant is either killed, strategy-unreachable, or surviving. Equivalent mutants are not separately detected and if present, they appear in the surviving class, which inflates the adjusted mutation score denominator slightly but remains the most practical boundary.

The rationale for classifying *Transition Addition* mutants as strategy-unreachable is as follows. This operator injects a new transition on an input symbol for which no transition exists at the source state in the original FSM. Since every coverage strategy derives its test cases from the structure of the original model, the injected transition is never exercised by any generated test case. These mutants are therefore unkillable not because they are behaviourally equivalent to the original FSM, but because the generation strategy cannot produce an input sequence that reaches them. Counting them in the denominator would suppress the reported score without any meaningful interpretation. Strategy-unreachable mutants are excluded from both the numerator and the denominator of the adjusted mutation score, and their per-operator score is reported as null in *metrics_report.json*.

The adjusted mutation score is defined as:

$$\text{Adjusted Mutation Score} = \frac{\#km}{\#km + \#sm}$$

where $\#km$ is the number of killed mutants and $\#sm$ is the number of surviving mutants. This differs from the mutation score defined in Section 4.1.4, which used $\#tm$ (total mutants) as the denominator and therefore included strategy-unreachable mutants. The adjusted score reflects the true fault detection capability of the suite against the mutants that the chosen coverage strategy is capable of killing.

To illustrate the effect of this classification, consider the run on *TcpConnectionFSM* with *Transition Pair Coverage*, *Mutation Score Preserving Reduction*, and *Coverage-Based Prioritisation*. The mutation engine generates 671 mutants in total, of which 110 are produced

by the *Transition Addition* operator. All 110 are classified as strategy-unreachable because *TcpConnectionFSM* is a partially specified model with free inputs on several states. Of the remaining 561 killable mutants, all 561 are killed by the final suite, and no surviving mutants remain. The post-reduction mutation score computed over all mutants is $561/671 = 0.8361$, whereas the adjusted mutation score is $561/561 = 1.000$. The gap between the two values is entirely attributable to the 110 strategy-unreachable mutants that inflate the denominator without being killable. Both scores are reported in *metrics_report.json* so that the practitioner can observe this distinction directly.

The detailed sequence of interactions within *MutantClassifier* during a classification pass is illustrated in Appendix L.

5.3 CI/CD Integration

Section 4.3 established CI/CD integration as a design objective of the final artefact, and FR7 and NFR4 in Section 4.5.1 formalised the corresponding requirements. This section describes how those requirements are realised, first through the interface design of the tool itself, and then through a concrete GitHub Actions [42] workflow that demonstrates the integration in practice.

Three design decisions make the tool inherently CI/CD friendly without requiring any workflow adaptation. First, *toolconfig.json* is the configuration interface and no command-line flags, interactive prompts, or environment variables are involved, so a pipeline step invokes the tool with a single *dotnet run* command. Second, *Program* terminates with exit code 0 on success and exit code 1 on any validation error, which means the CI step fails naturally if the FSM model is invalid or the configuration file is missing, without requiring explicit error-checking logic in the workflow. Third, both output files are written to a configured directory, so other steps can assert their existence and parse their contents without knowing anything about the FSM or the chosen strategies.

5.3.1 Configuration Interface

The *toolconfig.json* file is the single artefact that a user commits to the repository to control the tool's behaviour. Listing 5.1 shows its structure.

```

1 {
2   "fsmModel":      "path/to/fsm.json",
3   "outputDirectory": "out/",
4   "coverage": {
5     "type":        "TransitionPairCoverage",
6     "sequenceLength": 3
7   },
8   "reduction": {
9     "type":        "MutationScorePreserving",
10    "fsmSizeThreshold": 50,
11    "similarityThreshold": 0.8
12  },
13  "prioritisation": {
14    "type": "CoverageBased"
15  }
16 }
```

Listing 5.1: Structure of *toolconfig.json*

Table 5.3 describes each field, the default coverage strategy is *Transition Pair Coverage*, consistent with NFR3 from Section 4.5.1. All property names are case-insensitive on read and unknown fields are silently ignored, making the file tolerant of minor formatting variations.

Field	Default	Meaning
fsmModel	—	Path to the FSM model JSON file
outputDirectory	output/	Directory where output files are written
coverage.type	TransitionPair Coverage	Coverage strategy identifier
coverage.sequenceLength	3	Sequence length k for Sequence Coverage; ignored otherwise
reduction.type	MutationScore Preserving	Reduction strategy identifier
reduction.fsmSize Threshold	50	Transition count above which a performance warning is emitted for <i>Mutation Score Preserving Reduction</i>
reduction.similarity Threshold	0.8	BMI threshold for Similarity Reduction; ignored otherwise
prioritisation.type	CoverageBased	Prioritisation strategy identifier

Table 5.3: *toolconfig.json* fields

5.3.2 Output Files

Each tool invocation produces two JSON files in the configured output directory, fulfilling FR6 and NFR5 from Section 4.5.1. Their schemas are shown in Listings 5.2 and 5.3.

test_suite.json contains the prioritised, reduced test suite ready for an external test runner. Each test case is self-contained where every step records the full transition so that no knowledge of the original FSM model file is required to execute or interpret the suite. The *priority* field is an explicit integer rather than an implicit array position, allowing downstream tools to sort and filter by execution rank independently of array ordering.

```

1 {
2   "metadata": {
3     "fsmName":           string,
4     "generatedAt":       string,    // ISO 8601 UTC
5     "coverageStrategy":  string,
6     "reductionStrategy": string,
7     "prioritisationStrategy": string,
8     "totalTestCases":    int
9   },

```

```

10  "testCases": [
11    {
12      "id":          string,    // e.g. "TC0031"
13      "priority":    int,       // 1-based execution order
14      "steps": [
15        {
16          "fromState":    string,
17          "input":        string,
18          "expectedOutput": string,
19          "toState":      string
20        }
21      ]
22    }
23  ]
24 }

```

Listing 5.2: Schema of test_suite.json

metrics_report.json contains the full evaluation report for the run. It is the file a CI/CD dashboard or a thesis evaluator would parse. It reports suite compactness metrics, coverage, the three mutation scores described in Section 5.2, per-operator scores, generation time, and any validation warnings emitted during startup.

```

1  {
2    "fsmName": string, "generatedAt": string,
3    "configuration": {
4      "coverageStrategy": string,
5      "reductionStrategy": string,
6      "prioritisationStrategy": string
7    },
8    "suiteMetrics": {
9      "suiteSize": int, "totalSteps": int,
10     "preReductionSuiteSize": int, "preReductionTotalSteps": int,
11     "reductionRatio": double, "executionTimeSavings": double,
12     "stateCoverage": double, "transitionCoverage": double,
13     "apfd": double, "generationTimeMs": double
14   },
15   "mutationMetrics": {
16     "preReductionMutationScore": double,
17     "postReductionMutationScore": double,
18     "adjustedMutationScore": double,
19     "totalMutants": int, "killedMutants": int,
20     "strategyUnreachableMutants": int, "survivingMutants": int,
21     "perOperatorScores": {
22       "TransitionTargetMutation": double | null,
23       "OutputMutation": double | null,
24       "TransitionRemoval": double | null,
25       "TransitionAddition": double | null,
26       "InputMutation": double | null,
27       "InitialStateMutation": double | null
28     }
29   },
30   "warnings": [string]
31 }

```

Listing 5.3: Schema of metrics_report.json

5.3.3 GitHub Actions Workflow

The CI/CD integration is realised as a GitHub Actions [42] workflow stored at `.github/workflows/generate-tests.yml`. Its purpose is to run the full test-generation workflow automatically whenever the FSM model, the tool configuration, or the tool's source code changes, and to make the resulting test suite and metrics report available as a downloadable build artefact.

The workflow is triggered by two automatic events and one manual event. On *push* and *pull_request*, it fires only when files under *FinalArtefact/* or *toolconfig.json* are modified and changes to unrelated files sources do not trigger it. The *workflow_dispatch* event allows a developer to invoke the workflow manually from the GitHub Actions interface and optionally supply a path to an alternative configuration file.

Listing 5.4 shows the key structural elements of the workflow.

```

1 name: Generate FSM Test Suite
2
3 on:
4   push:
5     paths: ['FinalArtefact/**', 'toolconfig.json']
6   pull_request:
7     paths: ['FinalArtefact/**', 'toolconfig.json']
8   workflow_dispatch:
9     inputs:
10      config:
11        description: 'Path to configuration file'
12        default: 'toolconfig.json'
13
14 jobs:
15   generate:
16     runs-on: ubuntu-latest
17     steps:
18       - uses: actions/checkout@v4
19       - uses: actions/setup-dotnet@v4
20       with:
21         dotnet-version: '8.0.x'
22       - run: dotnet restore FinalArtefact/FinalArtefact.sln
23       - run: dotnet build FinalArtefact/FinalArtefact.sln
24         --no-restore --configuration Release
25       - name: Generate test suite
26         working-directory: FinalArtefact
27         run: dotnet run --project FinalArtefact.csproj
28           --configuration Release --no-build
29           -- "${{ github.event.inputs.config
30             || 'toolconfig.json' }}"
31       - name: Upload outputs
32         uses: actions/upload-artifact@v4
33         with:
34           name: fsm-test-suite

```

```
35     path: |
36         FinalArtefact/out/test_suite.json
37         FinalArtefact/out/metrics_report.json
38     retention-days: 30
39 - name: Report metrics summary
40   if: always()
41   run: |
42       # Writes a Markdown table to $GITHUB_STEP_SUMMARY
43       # extracting suite size, coverage, mutation scores,
44       # and APFD from metrics_report.json via jq
```

Listing 5.4: GitHub Actions workflow for FSM test suite generation (simplified)

After the generation step completes, both output files are packaged as a named build artefact (*fsm-test-suite*) retained for 30 days. This is the primary handoff mechanism where a downstream process, a CI step in another repository, or a human reviewer can download the artefact and consume *test_suite.json* directly. A final step, which runs even when earlier steps fail, writes a Markdown table to the GitHub Actions job summary page displaying suite size, transition coverage, pre and post-reduction mutation scores, adjusted mutation score, and APFD. This gives the team a zero-friction metrics dashboard without requiring access to the raw JSON files.

The workflow demonstrates that the interface design decisions described at the start of this section translate directly into a practical CI/CD integration requiring no workflow-specific adaptation of the tool itself.

5.4 Improvements Over the Experimental Prototype

The final artefact is an iterative refinement of the experimental prototype, incorporating the design implications identified in Section 4.2 and the requirements established in Section 4.5.1. Table 5.4 summarises the improvements across the dimensions most affected by that refinement.

Dimension	Experimental Prototype	Final Artefact
Output format	CSV rows per configuration, intended for offline data analysis	Structured JSON files (<code>test_suite.json</code> , <code>metrics_report.json</code>) ready for downstream pipeline consumption
Configuration	Programmatic, selected per experimental run	Single <code>toolconfig.json</code> file committed alongside the FSM model
CI/CD integration	Not implemented; prototype designed exclusively for controlled strategy evaluation	Deterministic exit codes, file-based outputs, and a reference GitHub Actions [42] workflow
Input validation	None	Five-check <code>FsmValidator</code> : hard errors for malformed models, non-blocking warnings for reachability and scalability
Mutant classification	All mutants treated uniformly; strategy-unreachable mutants deflated the aggregate score without providing discriminating signal	Three-class classification (Killed, Strategy-unreachable, Surviving) with an adjusted mutation score that excludes strategy-unreachable mutants from the denominator
Mutation score reporting	Single aggregate score per configuration	Pre-reduction baseline score and post-reduction score reported separately, making the fault-detection cost of reduction visible

Table 5.4: Experimental prototype versus final artefact across key dimensions

Taken together, these improvements represent the transition from a research instrument designed for controlled comparison to a tool designed for practical use. The experimental prototype generated the evidence and the final artefact acts on it. This progression, from artefact construction through evaluation to a refined solution informed by the results, reflects the DSR cycle that structures this dissertation and directly addresses the research questions identified in Chapter 1.

Chapter 6

Conclusion

This chapter closes the dissertation. Section 6.1 summarises the work conducted across all chapters. Section 6.2 revisits the three research questions in light of the evidence produced. Section 6.3 identifies the limitations of the approach. Section 6.4 outlines directions for future work.

6.1 Summary

This dissertation addressed a practical problem in FSM-based MBT where existing approaches for generating test suites from FSM models tend to produce excessively large, redundant suites with limited integration into modern software development workflows. Chapter 1 framed this problem and defined three research questions addressing the optimisation of FSM-based test generation, its integration into CI/CD workflows, and how such a framework should be evaluated prior to implementation.

Chapter 2 addressed the project management dimension of the work. A competency analysis, SWOT assessment, and structured action plan established the skills required to conduct the research and defined targeted measures to address identified gaps.

Chapter 3 established the conceptual and empirical foundation of the dissertation through a systematic mapping review of FSM-based test generation literature. The review characterised the state of the art across coverage criteria, reduction strategies, prioritisation approaches, and mutation-based evaluation, identifying the techniques, strengths, and limitations of existing work. These findings informed the strategy selection and evaluation design adopted in Chapter 4.

Chapter 4 presented the experimental prototype and its evaluation. The prototype combined four coverage strategies, four reduction strategies, and two prioritisation strategies in a full-factorial experimental design across five benchmark FSM models. The evaluation measured mutation score, transition coverage, APFD, suite size, total steps, and computational cost across all 32 strategy configurations. Its findings were translated into a set of design implications, strategy selection principles, and requirements for the final artefact.

Chapter 5 presented the final artefact. A single .NET 8 executable that consolidates those design implications into a tool suitable for deployment in real-world engineering workflows. The artefact reuses all strategy and mutation operator implementations from the experimental prototype, and adds a file-based configuration interface (`toolconfig.json`), structured JSON outputs, input validation, three-class mutant classification with an adjusted mutation score, dual mutation testing runs for pre and post-reduction score comparison, and a

reference GitHub Actions workflow. Together these capabilities address all twelve requirements established in Section 4.5.1 and fulfil the CI/CD integration objective identified in Section 4.3.

6.2 Research Questions

This section revisits the three research questions defined in Section 1.4 and states the answer each yields based on the evidence produced in this dissertation.

RQ1. How can FSM-based automated test generation be optimised to reduce redundancy and computational cost while preserving coverage and fault detection?

The experimental evaluation in Chapter 4 provided concrete, empirically grounded answers to this question. Regarding coverage, *Transition Pair Coverage* demonstrated the strongest and most consistent fault detection across both protocol-scale and synthetic benchmark models, and is established as the recommended default criterion. *State Coverage* was found to be insufficient as a primary criterion, and *Sequence Coverage* requires a model-specific path length to be effective. Regarding redundancy reduction, all four reduction strategies delivered meaningful suite size savings, with *Coverage Preserving Reduction* and *Similarity Reduction* achieving approximately 52-56% size reductions at modest fault detection costs on protocol-scale models. However, both strategies can cause mutation score drops of 11-16 percentage points on larger models when combined with *Transition Pair Coverage*, an interaction effect that practitioners must be made aware of. *Mutation Score Preserving Reduction* uniquely guarantees zero fault detection loss but scales poorly to large models, making it suitable only for small-scale contexts. Regarding prioritisation, both *Coverage-Based Prioritisation* and *Weighted-Transition Prioritisation* produced similar APFD values and neither consistently dominated the other, and both are offered as equivalent options in the final artefact. Finally, the analysis of mutation operators showed that *Transition Addition* mutants are structurally unkillable under any strategy derived from the original FSM, and that excluding them from the mutation score denominator yields a more accurate measure of fault detection capability. The final artefact operationalises all of these findings through its configurable strategy selection, dual mutation testing runs, and adjusted mutation score.

RQ2. How can FSM-based test generation be integrated into modern software testing processes and CI/CD workflows?

This question is answered by the design of the final artefact described in Chapter 5, and specifically by the three interface decisions described in Section 5.3. First, `toolconfig.json` serves as the sole configuration interface, meaning a CI/CD pipeline step invokes the tool with a single command and no workflow-specific adaptation is required. Second, deterministic exit codes allow the pipeline to detect failures without explicit error-checking logic. Third, both output files are written to a predictable path, making them directly consumable by downstream steps. The reference GitHub Actions workflow in Section 5.3.3 demonstrates this integration in practice, showing how the tool can be triggered automatically on model or configuration changes, with outputs published as a build artefact and a Markdown metrics summary written to the job summary page.

RQ3. How should an FSM-based test generation framework be evaluated prior to implementation?

This question is addressed primarily by Chapter 3 and by the evaluation design in Chapter 4. The systematic mapping review in Chapter 3 characterised the evaluation practices used

in existing FSM-based test generation work, identifying the metrics most commonly used to assess coverage, fault detection, redundancy, and computational efficiency. Building on those findings, the evaluation of the experimental prototype in Chapter 4 adopted a GQM-driven approach, defining goals, questions, and metrics before any data was collected. The selected metrics collectively reflect the competing objectives of fault detection capability, structural completeness, execution ordering, compactness, and practical feasibility. This combination provides a multi-dimensional evaluation basis that exposes trade-offs rather than optimising a single criterion in isolation, and constitutes the recommended evaluation framework for FSM based test generation tools of this kind.

6.3 Limitations

Several limitations constrain the scope of the results presented in this dissertation.

Deterministic FSMs only. Both artefacts are scoped to classical deterministic FSMs following the Mealy machine model. Non-deterministic FSMs, extended FSMs (EFSMs) with data variables and guards, or other models are not supported. Applying the approach to these richer formalisms would require non-trivial changes to the coverage, reduction, and mutation components.

Benchmark scale and diversity. The experimental evaluation was conducted on five benchmark models, two protocol-scale FSMs and three synthetic models of increasing size. While the benchmark set was selected to expose scale-dependent effects, the findings may not generalise to all FSM topologies, densities, or application domains. In particular, the interaction effect between *Transition Pair Coverage* and aggressive reduction strategies was observed consistently but only within the scale range covered by the benchmarks.

Single FSM per invocation. The tool processes one FSM model per run. Systems composed of multiple interacting FSMs, cannot be handled without manual work by the practitioner.

Reduction strategy isolation. The experimental design applied exactly one reduction strategy per configuration to isolate individual effects. Chaining multiple reduction strategies was explicitly excluded from the evaluation, as doing so would make it difficult to attribute changes in mutation score or suite size to a specific strategy. The combined effect of chained reductions therefore remains uncharacterised.

Equivalent mutant detection. The three-class mutant classification in the final artefact identifies strategy-unreachable mutants but does not detect truly equivalent mutant, those that are behaviourally indistinguishable from the original FSM for all possible inputs. Such mutants would be classified as surviving, which could overstate the number of genuine detection gaps in the reported metrics.

No test execution integration. The final artefact generates `test_suite.json` as a structured, self-contained test suite ready for execution, but does not include a test runner. The gap between the generated test suite and its actual execution against a system under test must be bridged by the practitioner using external tooling.

6.4 Future Work

The limitations identified in Section 6.3 and the design choices deferred throughout this dissertation suggest several directions for future work.

Equivalent mutant detection. The current mutant classification identifies strategy-unreachable mutants but cannot detect true equivalent mutants. Incorporating an equivalence-checking procedure, would allow equivalent mutants to be excluded from both the numerator and denominator of the mutation score, resulting in an even more accurate measure of fault detection capability.

Extended FSM formalisms. Extending the approach to EFSMs, which augment transitions with data guards and actions, would substantially broaden the set of systems that can be modelled and tested. This would require revisiting the coverage criteria, mutation operators, and the FSM simulation logic in `FSMExecutor` to account for variable assignments and guard evaluation.

Reduction strategy chaining. The experimental design deliberately isolated reduction strategies to measure their individual effects. A natural extension is to evaluate chained combinations, for example applying *Duplicate Removal* as a first pass followed by *Similarity Reduction*, to determine whether sequenced reductions can achieve greater compactness than any single strategy.

Broader benchmark evaluation. The experimental findings are grounded in five benchmark models. Replicating the evaluation on a larger and more diverse set of FSMs, drawn from additional application domains such as embedded systems, user interface workflows, or safety-critical protocols, would strengthen the generalisability of the strategy selection recommendations and may reveal domain-specific interaction effects not observed in the current benchmarks.

Test runner integration. The structured format of `test_suite.json` was designed to be consumable by external tooling without knowledge of the original FSM model. A concrete next step would be to develop an adapter that maps the generated test cases to a specific test framework, allowing the suite to be executed against an actual implementation and the results to be fed back into the CI/CD pipeline as a standard test report.

Bibliography

- [1] M. L. Mohd-Shafie et al. "Model-based test case prioritization using selective and even-spread count-based methods with scrutinized ordering criterion". In: *Plos One* (2020).
- [2] Arineiza C. Pinheiro, Adenilso Simão, and Ana Maria Ambrosio. "FSM-Based Test Case Generation Methods Applied to Test the Communication Software on Board the ITASAT University Satellite: A Case Study". In: *Journal of Aerospace Technology and Management* 6.4 (2014), pp. 447–461. doi: 10.5028/jatm.v6i4.369.
- [3] Wen-ling Huang and Jan Peleska. *Complete Requirements-based Testing with Finite State Machines*. 2021. arXiv: 2105.11786 [cs.SE].
- [4] Shan Zhou et al. "An Approach to the State Explosion Problem: SOPC Case Study". In: *Electronics* 12.24 (2023). issn: 2079-9292. doi: 10.3390/electronics12244987.
- [5] Vahid Garousi et al. *Model-based testing in practice: An experience report from the web applications domain*. 2021. arXiv: 2104.02152 [cs.SE].
- [6] Jan vom Brocke, Alan Hevner, and Alexander Maedche. "Introduction to Design Science Research". In: Sept. 2020. doi: 10.1007/978-3-030-46781-4_1.
- [7] Gonçalo Miranda. *FSM-Based Test Generation Framework: Prototype, Benchmark Models, and Experimental Results*. <https://github.com/1190622-GoncaloMiranda/software-testing-with-finite-state-machines>. Accessed: 2026-06-14. 2026.
- [8] Don Gotterbarn, Keith Miller, and Simon Rogerson. "Software engineering code of ethics". In: *Commun. ACM* (1997). doi: 10.1145/265684.265699.
- [9] Antonia Bertolino. "Software Testing Research: Achievements, Challenges, Dreams". In: *Future of Software Engineering (FOSE '07)*. 2007. doi: 10.1109/FOSE.2007.25.
- [10] Arash Sabbaghi and Mohammad Reza Keyvanpour. "State-based models in model-based testing: A systematic review". In: *2017 IEEE 4th International Conference on Knowledge-Based Engineering and Innovation (KBEI)*. 2017. doi: 10.1109/KBEI.2017.8324934.
- [11] Qaisar A. Malik et al. "Model-Based Testing Using System vs. Test Models - What Is the Difference?" In: *2010 17th IEEE International Conference and Workshops on Engineering of Computer Based Systems*. 2010. doi: 10.1109/ECBS.2010.41.
- [12] Johan Blom, Bengt Jonsson, and Sven-Olof Nyström. "Industrial Evaluation of Test Suite Generation Strategies for Model-Based Testing". In: *2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 2016. doi: 10.1109/ICSTW.2016.42.
- [13] Gregory Gay et al. "The Risks of Coverage-Directed Test Case Generation". In: *IEEE Transactions on Software Engineering* (2015). doi: 10.1109/TSE.2015.2421011.
- [14] R. Ubar et al. "Functional Test Generation for Finite State Machines". In: *2006 International Biennial Baltic Electronics Conference*. 2006. doi: 10.1109/BEC.2006.311099.
- [15] Sofia Costa Paiva and Adenilso Simao. "Generation of complete test suites from mealy input/output transition systems". In: (2016). doi: 10.1007/s00165-015-0350-2.
- [16] Nutchakorn Ngamsaowaros and Peraphon Sophatsathit. "A Novel Framework for Test Domain Reduction using Extended Finite State Machine". In: *International Conference*

- on *Software Engineering Advances (ICSEA 2007)*. 2007. doi: 10.1109/ICSEA.2007.9.
- [17] Samrat S. Batth et al. "Fault Modeling and Detection Capabilities for EFSM Models". In: *IEEE Transactions on Instrumentation and Measurement* (2008). doi: 10.1109/TIM.2007.915145.
 - [18] Wen-ling Huang, Niklas Krafczyk, and Jan Peleska. "Exhaustive property oriented model-based testing with symbolic finite state machines". In: *Science of Computer Programming* 231 (2024). issn: 0167-6423. doi: <https://doi.org/10.1016/j.scico.2023.103005>.
 - [19] Alexandre Petrenko. "Checking Experiments for Symbolic Input/Output Finite State Machines". In: *2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 2016. doi: 10.1109/ICSTW.2016.9.
 - [20] Mariana Soller Ramada et al. "Generating Reduced Tests for FSMs using a Search-Based Testing Approach". In: *2019 IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI)*. 2019. doi: 10.1109/ICTAI.2019.00063.
 - [21] Alfredo Ibias, Manuel Núñez, and Robert M. Hierons. "Using mutual information to test from Finite State Machines: Test suite selection". In: *Information and Software Technology* (2021). doi: <https://doi.org/10.1016/j.infsof.2020.106498>.
 - [22] Alfredo Ibias. "Using mutual information to test from Finite State Machines: Test suite generation". In: *Journal of Systems and Software* (2022). doi: <https://doi.org/10.1016/j.jss.2022.111391>.
 - [23] Andre Takeshi Endo and Adenilso Simao. "Evaluating test suite characteristics, cost, and effectiveness of FSM-based testing methods". In: *Information and Software Technology* (2013). doi: <https://doi.org/10.1016/j.infsof.2013.01.001>.
 - [24] Shari Lawrence Pfleeger and Barbara Ann Kitchenham. "Evidence-Based Software Engineering Guidelines Revisited". In: *IEEE Transactions on Software Engineering* 51.3 (2025). doi: 10.1109/TSE.2025.3526730.
 - [25] Mathew Azarian et al. "Do We Perform Systematic Literature Review Right? A Scientific Mapping and Methodological Assessment". In: *Logistics* 7.4 (2023). issn: 2305-6290. doi: 10.3390/logistics7040089.
 - [26] Matthew J Page et al. "PRISMA 2020 explanation and elaboration: updated guidance and exemplars for reporting systematic reviews". In: *BMJ* (2021). doi: 10.1136/bmj.n160.
 - [27] Muhammad Nouman Zafar et al. "Optimizing Model-based Generated Tests: Leveraging Machine Learning for Test Reduction". In: *2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 2024. doi: 10.1109/ICSTW60967.2024.00020.
 - [28] Muhammad Nouman Zafar. *TIGER+: Test Script Generation Framework*. <https://github.com/MuhammadNoumanZafar/TIGERPlus>. Accessed: 2026-06-10. 2024.
 - [29] Uraz Cengiz Türker, Robert M. Hierons, and Guy-Vincent Jourdan. "Minimizing Characterizing sets". In: *Science of Computer Programming* 208 (2021). issn: 0167-6423. doi: <https://doi.org/10.1016/j.scico.2021.102645>.
 - [30] Muhammad Luqman Mohd-Shafie et al. "An EFSM-Based Test Data Generation Approach in Model-Based Testing". In: *Computers, Materials and Continua* 71.3 (2022). issn: 1546-2218. doi: <https://doi.org/10.32604/cmc.2022.023803>.
 - [31] Vaclav Rechtberger et al. *Prioritized Variable-length Test Cases Generation for Finite State Machines*. 2022. arXiv: 2203.09596 [cs.SE].
 - [32] István Forgács and Attila Kovács. "Action-State Testing—A Model for Test Design Automation". In: *IEEE Access* 13 (2025). doi: 10.1109/ACCESS.2025.3563337.

- [33] Achiya Elyasaf et al. "Generalized Coverage Criteria for Combinatorial Sequence Testing". In: *IEEE Transactions on Software Engineering* 49.8 (2023). doi: 10.1109/TSE.2023.3279570.
- [34] Rasha Gh. Alsarraj, Atica M. Altaie, and Esra Zuhair Majeed. "Developing an Automated Model-Based Software Testing Tool From the Design Phase". In: *IEEE Access* 13 (2025). doi: 10.1109/ACCESS.2025.3553967.
- [35] Keith Stobie. "Model Based Testing in Practice at Microsoft". In: *Electronic Notes in Theoretical Computer Science* (2005). doi: <https://doi.org/10.1016/j.entcs.2004.12.004>.
- [36] Mutlu Beyazit, Tugkan Tuglular, and Dilek Öztürk Kaya. "Incremental Testing in Software Product Lines—An Event Based Approach". In: *IEEE Access* 11 (2023). doi: 10.1109/ACCESS.2023.3234186.
- [37] Wesley M. Eddy. *Transmission Control Protocol (TCP)*. RFC 9293. Obsoletes RFC 793. Internet Engineering Task Force, 2022. url: <https://www.rfc-editor.org/info/rfc9293>.
- [38] Joeri de Ruiter and Erik Poll. "Protocol State Fuzzing of TLS Implementations". In: *Proceedings of the 24th USENIX Security Symposium*. USENIX Association, 2015, pp. 193–206.
- [39] OASIS. *MQTT Version 3.1.1*. OASIS Standard. Also published as ISO/IEC 20922:2016. OASIS, 2014.
- [40] Martin Tappler, Bernhard K. Aichernig, and Roderick Bloem. "Model-Based Testing IoT Communication via Active Automata Learning". In: *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST 2017)*. IEEE, 2017, pp. 276–287. doi: 10.1109/ICST.2017.32.
- [41] Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. "The Goal Question Metric Approach". In: *Encyclopedia of Software Engineering*. Ed. by John J. Marciniak. Vol. 1. Wiley, 1994, pp. 528–532.
- [42] GitHub Inc. *GitHub Actions Documentation*. <https://docs.github.com/en/actions>. Accessed: 2026-06-14. 2025.
- [43] Sonali Pradhan, Mitrabinda Ray, and Santosh Kumar Swain. "Transition coverage based test case generation from state chart diagram". In: *Journal of King Saud University - Computer and Information Sciences* 34.3 (2022). issn: 1319-1578. doi: <https://doi.org/10.1016/j.jksuci.2019.05.005>.
- [44] Ahmad Mustafa et al. "Automated Test Case Generation from Requirements: A Systematic Literature Review". In: *Computers, Materials and Continua* 67.2 (2021). issn: 1546-2218. doi: <https://doi.org/10.32604/cmc.2021.014391>.
- [45] Gongzheng Lu and Huaikou Miao. "An Approach to Generating Test Data for EFSM Paths Considering Condition Coverage". In: *Electronic Notes in Theoretical Computer Science* (2014). doi: <https://doi.org/10.1016/j.entcs.2014.12.003>.
- [46] Yuan Chen et al. "Automatic Test Transition Paths Generation Approach from EFSM Using State Tree". In: *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. 2018. doi: 10.1109/QRS-C.2018.00029.
- [47] Rui Yang et al. "Improve the Effectiveness of Test Case Generation on EFSM via Automatic Path Feasibility Analysis". In: *2011 IEEE 13th International Symposium on High-Assurance Systems Engineering*. 2011. doi: 10.1109/HASE.2011.12.
- [48] P. Maragathavalli and S. Kanmani. "Incorporating Equivalence Partitioning on Classification Trees for Test Suite Size Reduction". In: *Proceedings of the International Conference on Informatics and Analytics*. 2016. doi: 10.1145/2980258.2980476.

- [49] Thaise Yano, Eliane Martins, and Fabiano L. de Sousa. "MOST: A Multi-objective Search-Based Testing from EFSM". In: *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. 2011. doi: 10.1109/ICSTW.2011.37.
- [50] G. Friedman et al. "Projected state machine coverage for software testing". In: (2002). doi: 10.1145/566171.566192.
- [51] Sezer Gören and F. Joel Ferguson. "Test sequence generation for controller verification and test with high coverage". In: (2006). doi: 10.1145/1179461.1179467.
- [52] Adenilso Simão and Alexandre Petrenko. "Fault Coverage-Driven Incremental Test Generation". In: *The Computer Journal* 53.9 (2010), pp. 1508–1522. issn: 0010-4620. doi: 10.1093/comjnl/bxp073.
- [53] Vinicius H. Fragal et al. "Extending HSI Test Generation Method for Software Product Lines". In: *The Computer Journal* 62.1 (2019), pp. 109–129. doi: 10.1093/comjnl/bxy046.
- [54] Jiangyuan Yao et al. "Formal Modeling and Systematic Black-Box Testing of SDN Data Plane". In: *2014 IEEE 22nd International Conference on Network Protocols (ICNP)*. 2014, pp. 179–190. isbn: 9781479962044. doi: 10.1109/ICNP.2014.37.
- [55] Jovan Muhamad Astrianto, Reza Ramdhani Riskiana, and Dade Surya Kusumo. "A Three-Stage Microflow-to-EFSM Transformation Approach for Test Coverage Enhancement in Mendix". In: *2025 International Conference on Artificial Intelligence, Blockchain, Cloud Computing, and Data Analytics (ICoABCD)*. 2025, pp. 1–8. doi: 10.1109/ICoABCD67551.2025.11470916.
- [56] Bo Song, Shengwen Gong, and Shengbo Chen. "Model Composition and Generating Tests for Web Applications". In: *2011 Seventh International Conference on Computational Intelligence and Security (CIS)*. 2011, pp. 568–572. isbn: 9780769545844. doi: 10.1109/CIS.2011.131.
- [57] Ting Shu et al. "Generating feasible protocol test sequences from EFSM models using Monte Carlo tree search". In: *Information and Software Technology* 135 (2021). issn: 0950-5849. doi: 10.1016/j.infsof.2021.106557.
- [58] Savaş Takan. "Fourier Analysis Based Testing of Finite State Machines". PhD thesis. Izmir Institute of Technology, 2019. isbn: 9798597073996.
- [59] Alexandre Petrenko. "Toward testing from finite state machines with symbolic inputs and outputs". In: *Software and Systems Modeling* 18.2 (2019), pp. 825–835. issn: 1619-1366. doi: 10.1007/s10270-017-0613-x.
- [60] Alexandre Petrenko and Adenilso Simao. "Checking Experiments for Finite State Machines with Symbolic Inputs". In: *Proceedings of the 27th IFIP WG 6.1 International Conference on Testing Software and Systems (ICTSS 2015)*. 2015, pp. 3–18. isbn: 9783319259444. doi: 10.1007/978-3-319-25945-1_1.

Appendix A

Competencies Matrix

Figure A.1 presents the competencies matrix developed as part of the skills management process described in Chapter 2. Each row corresponds to a competency relevant to the dissertation, identifying the required proficiency level, the initial self-assessed level, and the targeted development actions defined to address identified gaps.

COMPETÊNCIAS	Importância	Auto-avaliação	Hetero-avaliação
Análise e resolução de problemas	5	4	
Aprendizagem ao longo da vida	3	4	
Trabalho em equipa e colaboração	1	5	
Motivação para a excelência	3	3	
Adaptabilidade e flexibilidade	2	4	
Ética e responsabilidade social	3	4	
Domínio de línguas estrangeiras	1	3	
Competências técnicas da área específica de conhecimento	3	3	
Tomada de decisão	2	4	
Gestão do tempo	5	4	
Capacidade para assumir riscos	2	3	
Comunicação oral	5	4	
Comunicação escrita	4	4	
Resiliência	5	4	
Escuta ativa	1	4	
Relacionamento interpessoal	3	4	
Liderança	1	4	
Criatividade e inovação	3	2	
Adaptação e flexibilidade	3	3	
Pensamento crítico	5	4	
Planeamento e organização	5	3	
Gestão das emoções	2	4	
Negociação	1	2	
Empatia	1	5	
Assertividade	4	3	
Gestão do stress	2	5	
Proactividade	4	2	

Figure A.1: Competencies Matrix

Appendix B

Project Charter

This appendix contains the Project Charter produced during the project planning phase described in Chapter 2. The document defines the project scope, objectives, key deliverables, stakeholders, and the initial risk register that guided the management of the dissertation work.

PROJECT CHARTER

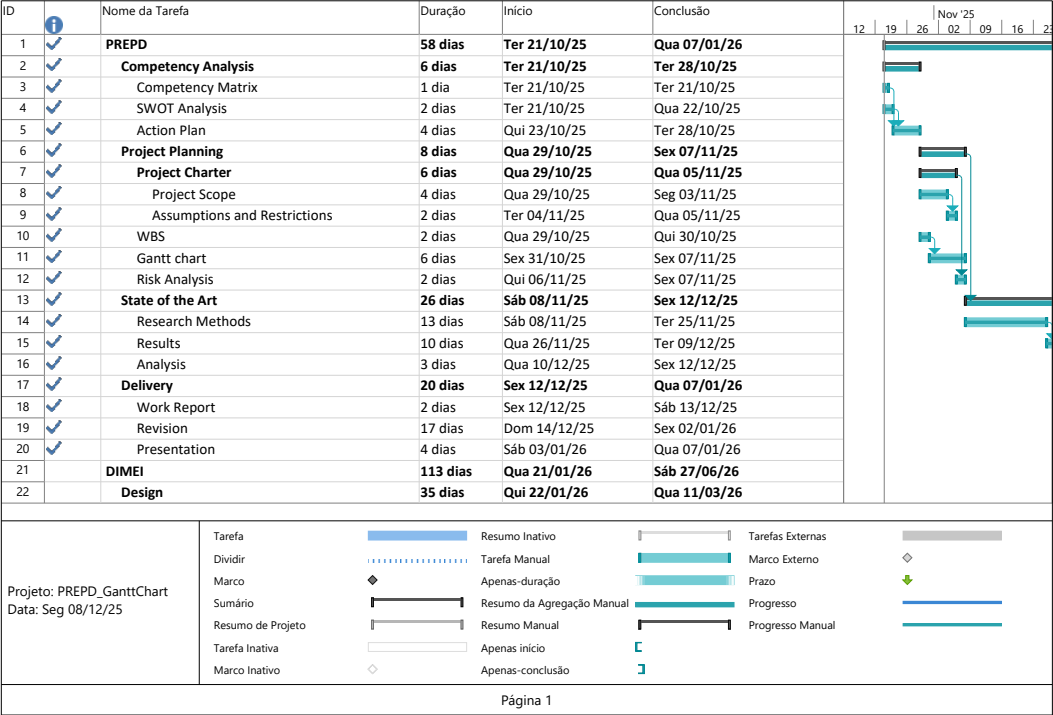
1. Informação geral				
Nome do Projeto:	Software Testing with Finite State Machines			
Sponsor:	Isabel Azevedo			
Instituição / Departamento	ISEP/DEI			
2. Equipa do Projeto				
Cargo	Nome	Departamento	Contact Tel	E-mail
Arquiteto e Desenvolvedor	Gonçalo Miranda	N/A	N/A	1190622@isep.ipp.pt
3. Stakeholders				
Nome		Poder	Interesse	
4. Âmbito				
Problema / justificação				
In software systems, reliable operation depends heavily on how well they are tested. One way to model system behaviour is using by Finite State Machines (FSMs), which allow us to capture states, transitions, and inputs/outputs formally. By starting from FSMs, we can apply model-based testing (MBT) and generate test cases that reflect the intended behaviour of the system. For example, Huang and Peleska demonstrated how FSMs can be used for requirements-based testing of deterministic FSMs [7], providing fault coverage guarantees. However, two main difficulties remain: first, generating test suites from FSMs can lead to huge numbers of test cases (the "state explosion" problem). Rechtberger, Bures, and Ahmed addressed the challenge of test path length and prioritisation in FSM-based testing, which is one characteristic of the state explosion and redundancy problem [8]. Second, in practice, many generated test cases are redundant or do not contribute to fault detection [1], yet they cost execution time and maintenance effort. Moreover, applying such approaches in industrial testing workflows (continuous integration, automated pipelines, etc.) is still often complex [2].				
Objetivos do projeto				
O1: Understand model-based testing, FSM modelling principles, and existing methods for generating test suites from FSM models. O2: Analyse existing FSM-based test generation approaches, identifying their techniques, strengths, and limitations regarding coverage, redundancy, computational cost, and industrial applicability. O3: Design and implement a framework that can automatically produce optimized test suites from provided FSM models, ensuring high coverage while avoiding redundant test cases [1]. O4: Explore techniques to control the state explosion problem in FSM-based testing, such as abstraction, model reduction or heuristic-guided test selection [3]. O5: Assess the proposed method through experiments on standard benchmark models, measuring factors like coverage, fault detection rate, and computational cost [4]. O6: Investigate practical adoption aspects, focusing on how the framework can integrate with existing testing processes and support automated execution.				

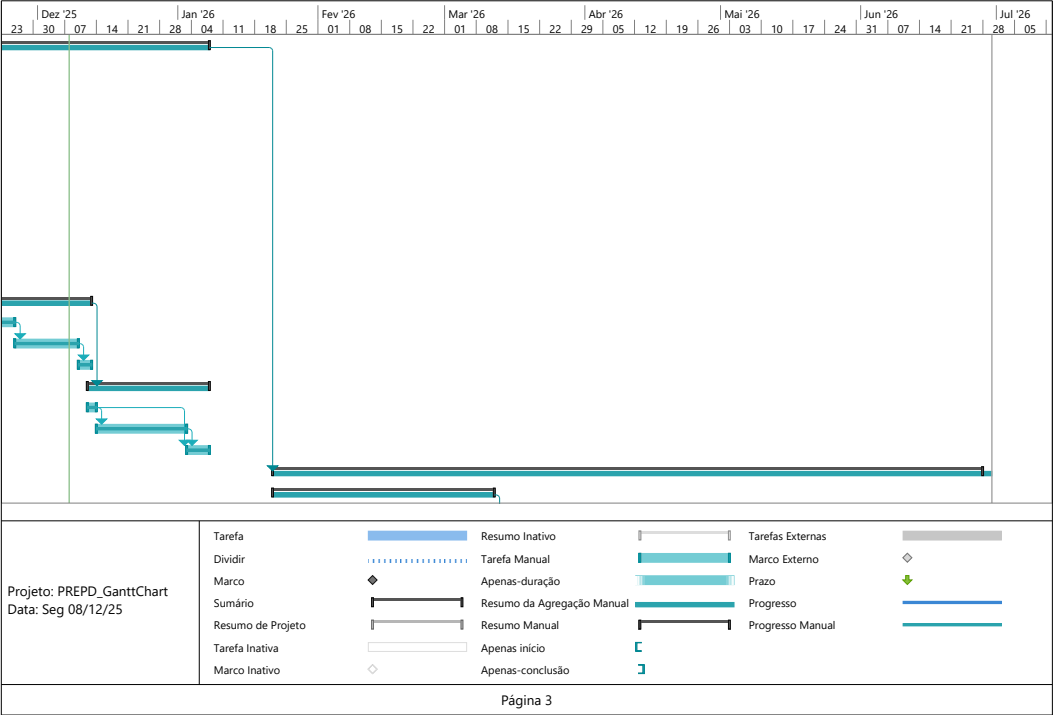
Benefícios	
Provide a designed solution, based on the gathered requirements, that can automatically generate efficient, high-coverage test suites from FSM models while avoiding redundant tests and managing scalability. Small-scale controlled experiments will be conducted to understand how to integrate with real-world development and testing processes. A Goal Question Metric (GQM) plan will be designed to guide the identification of metrics to explore the adequacy of the solution in the solution evaluation step. The metrics likely to be considered include coverage, fault detection rate, and test suite size.	
Entregáveis	
WBS & Gantt Chart State of the art Solution to automatically generate test suites from FSM models Small scale controlled experiments GQM plan Dissertation document	
5. Tempo	
Milestones / Datas	
07/01/2025 – PREPD delivery 27/06/2026 – DIMEI delivery	
6. Custo	
Fontes de custo	
N/A	
7. Pressupostos	8. Restrições
Availability of literature and benchmark models. Time allocated for each planned step is adequate.	Scope limitations on industrial integration. Prototype Fidelity.
Notas	

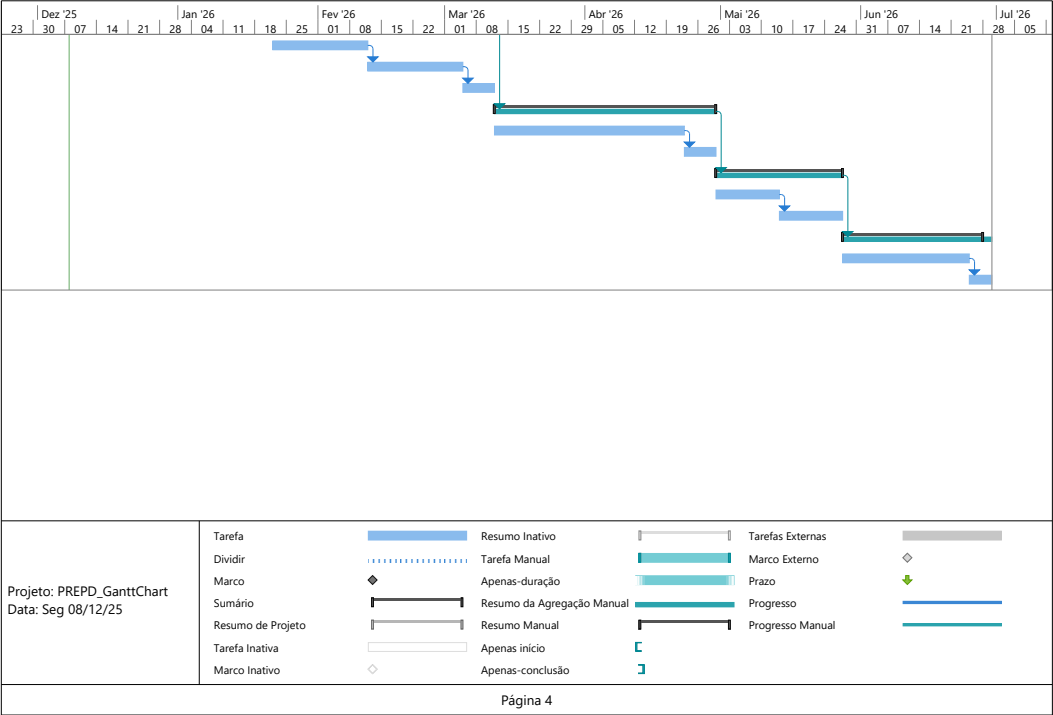
Appendix C

Gantt Chart

This appendix contains the Gantt Chart produced during the project planning phase described in Chapter 2. The chart shows the planned timeline for each dissertation phase across the full academic year, including key milestones and the dependencies between tasks.







Appendix D

Systematic Mapping Study

Tables D.1 and D.2 list the 32 primary studies included after applying the selection criteria described in Section 3.2. Each entry records the full title of the study and its corresponding bibliography reference, enabling cross-referencing with the analysis presented in Section 3.3.

Title	Reference
Minimizing Characterizing Sets	[29]
Prioritized Variable-length Test Cases Generation for Finite State Machines	[31]
Action-State Testing—A Model for Test Design Automation	[32]
Exhaustive property oriented model-based testing with symbolic finite state machines	[18]
Transition coverage based test case generation from state chart diagram	[43]
Automated Test Case Generation from Requirements: A Systematic Literature Review	[44]
A Novel Framework for Test Domain Reduction using Extended Finite State Machine	[16]
An Approach to Generating Test Data for EFSM Paths Considering Condition Coverage	[45]
Automatic Test Transition Paths Generation Approach from EFSM Using State Tree	[46]
Checking Experiments for Symbolic Input/Output Finite State Machines	[19]
Evaluating test suite characteristics, cost, and effectiveness of FSM-based testing methods	[23]
Fault Modeling and Detection Capabilities for EFSM Models	[17]
Functional Test Generation for Finite State Machines	[14]
Generating Reduced Tests for FSMs using a Search-Based Testing Approach	[20]
Improve the Effectiveness of Test Case Generation on EFSM via Automatic Path Feasibility Analysis	[47]

Table D.1: List of included studies and their corresponding references (Part one)

Title	Reference
Incorporating Equivalence Partitioning on Classification Trees for Test Suite Size Reduction	[48]
Industrial Evaluation of Test Suite Generation Strategies for Model-Based Testing	[12]
MOST: A Multi-objective Search-Based Testing from EFSM	[49]
Projected state machine coverage for software testing	[50]
Generation of complete test suites from mealy input/output transition systems	[15]
Test sequence generation for controller verification and test with high coverage	[51]
Using mutual information to test from Finite State Machines: Test suite selection	[21]
Using mutual information to test from Finite State Machines: Test suite generation	[22]
Fault Coverage-Driven Incremental Test Generation	[52]
Extending HSI Test Generation Method for Software Product Lines	[53]
Formal Modeling and Systematic Black-Box Testing of SDN Data Plane	[54]
A Three-Stage Microflow-to-EFSM Transformation Approach for Test Coverage Enhancement in Mendix	[55]
Model Composition and Generating Tests for Web Applications	[56]
Generating feasible protocol test sequences from EFSM models using Monte Carlo tree search	[57]
Fourier Analysis Based Testing of Finite State Machines	[58]
Toward testing from finite state machines with symbolic inputs and outputs	[59]
Checking Experiments for Finite State Machines with Symbolic Inputs	[60]

Table D.2: List of included studies and their corresponding references (Part two)

Appendix E

Prototype Component Architecture

Figure E.1 shows the component architecture of the experimental prototype described in Section 4.1.1. The diagram illustrates the ten strategy implementations organised behind their respective interfaces, the mutation engine and testing pipeline, and the shared core domain model and test model types that all components depend on.

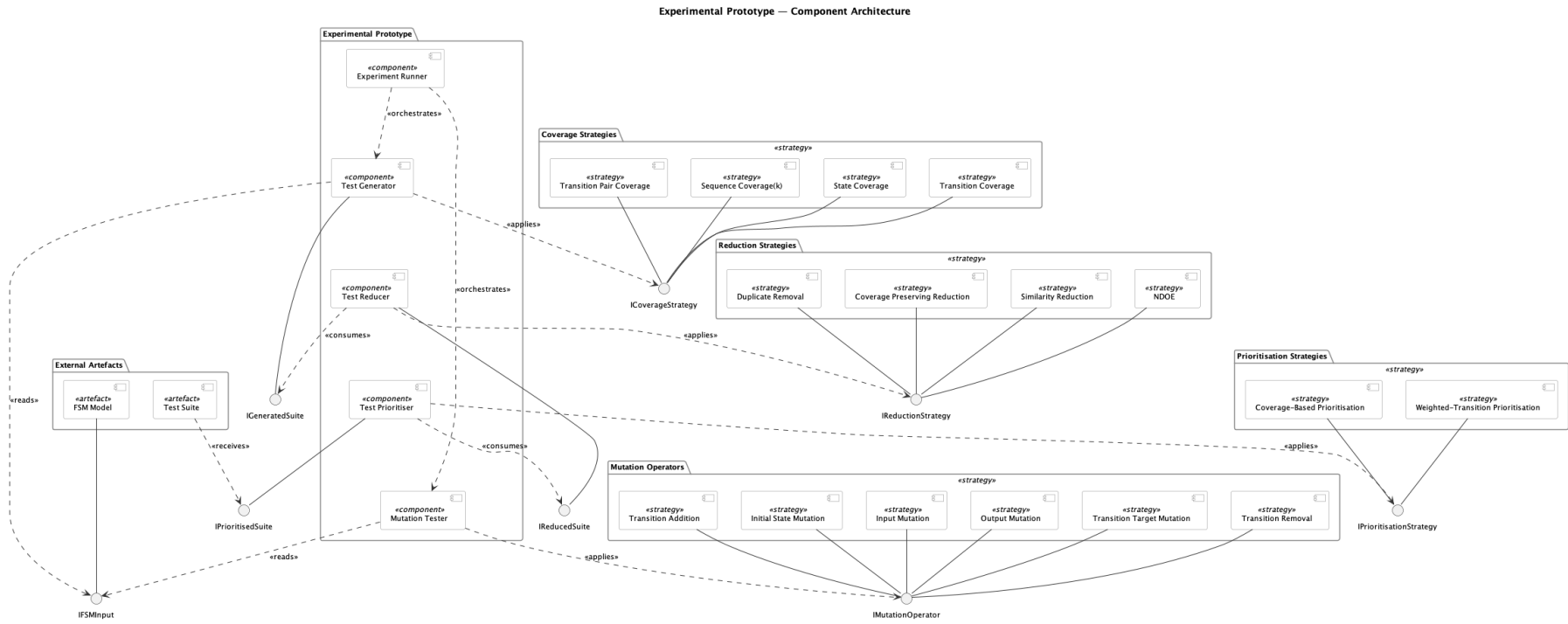


Figure E.1: Component architecture of the experimental prototype

Appendix F

Test Suite Generation Workflow

Figure F.1 details the sequence of interactions that constitute a single test suite generation run in the experimental prototype, as described in Section 4.1.1. The diagram shows how Program invokes the coverage strategy to produce raw test paths, how those paths are converted into TestCase objects, and how the resulting suite is handed off to the reduction and prioritisation stages.

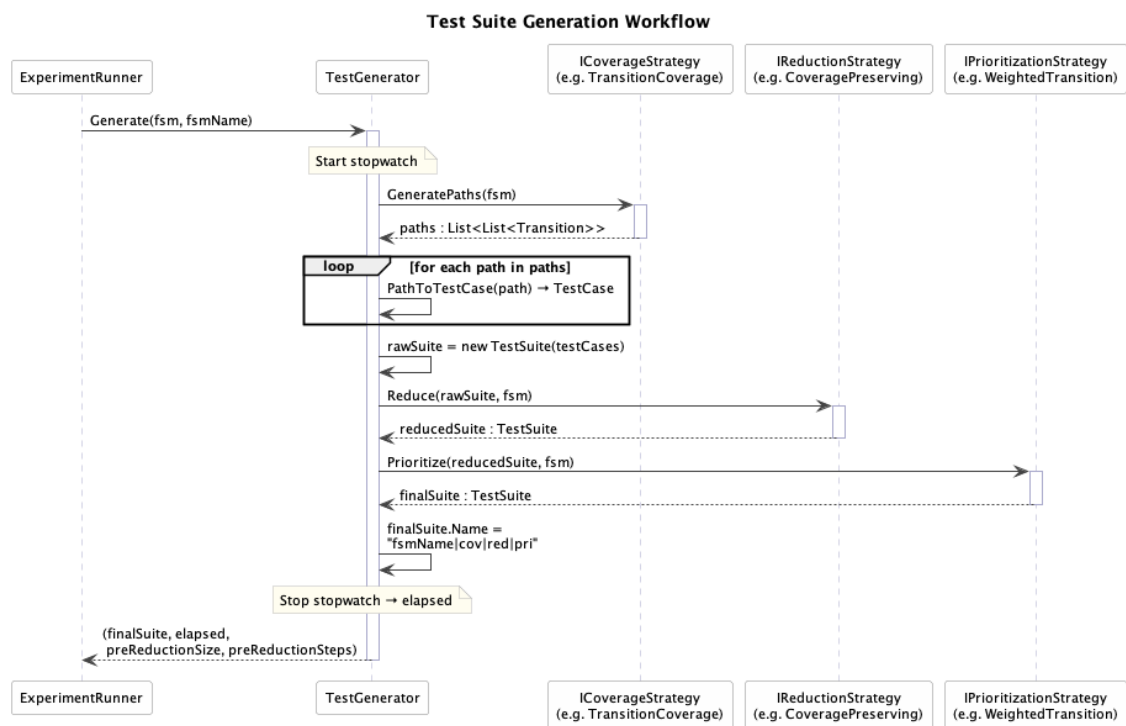


Figure F.1: Sequence diagram of the test suite generation workflow

Appendix G

Full Experiment Run Sequence

Figure G.1 shows the sequence of interactions across all 32 configurations of a full experiment run in the experimental prototype, as introduced in Section 4.1.4. The diagram illustrates how the experiment controller iterates over the coverage, reduction, and prioritisation strategy combinations, resetting mutant state between configurations to ensure each is evaluated against a clean baseline.

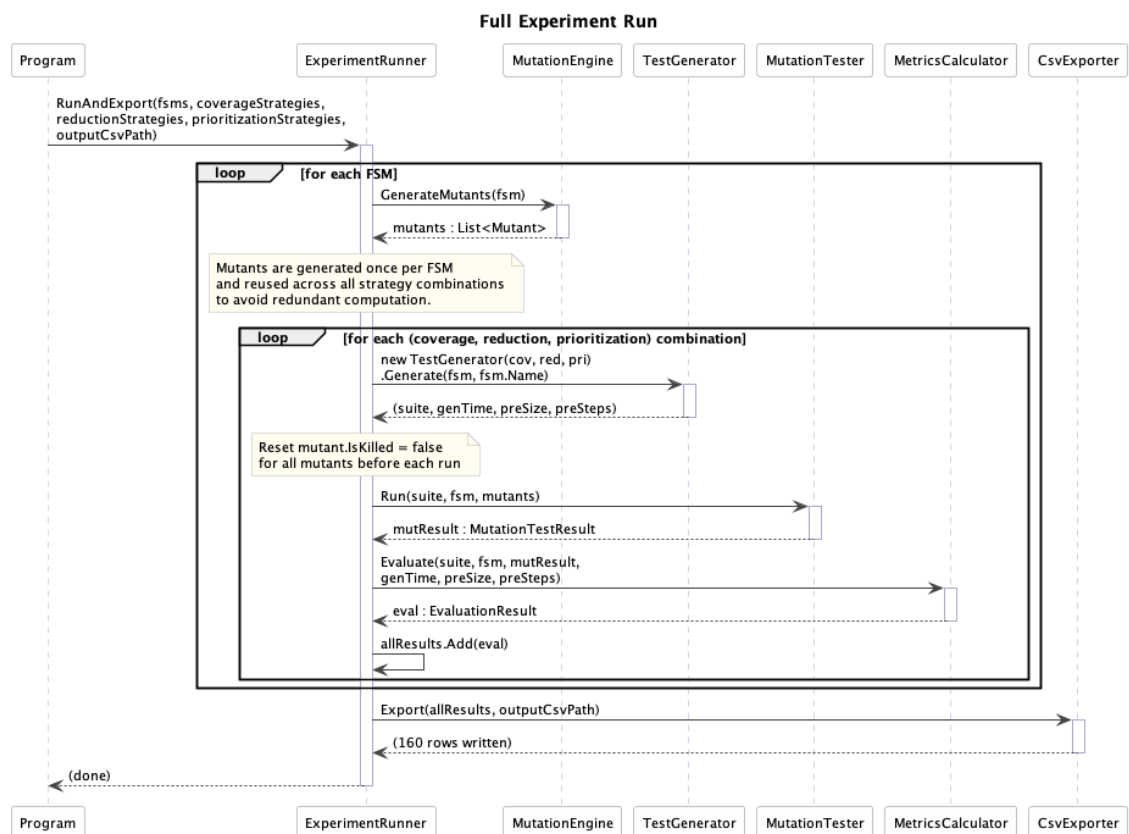


Figure G.1: Sequence diagram of a full experiment run

Appendix H

Mutation Testing Process

Figure H.1 details the two-phase mutation testing process used in the experimental prototype, as described in Section 4.1.4. In the first phase, the original FSM is executed against every test case to cache the expected output sequences. In the second phase, each test case is replayed against every mutant via `FSMExecutor`, and a mutant is marked as killed as soon as any test case produces a diverging output or triggers the error token.

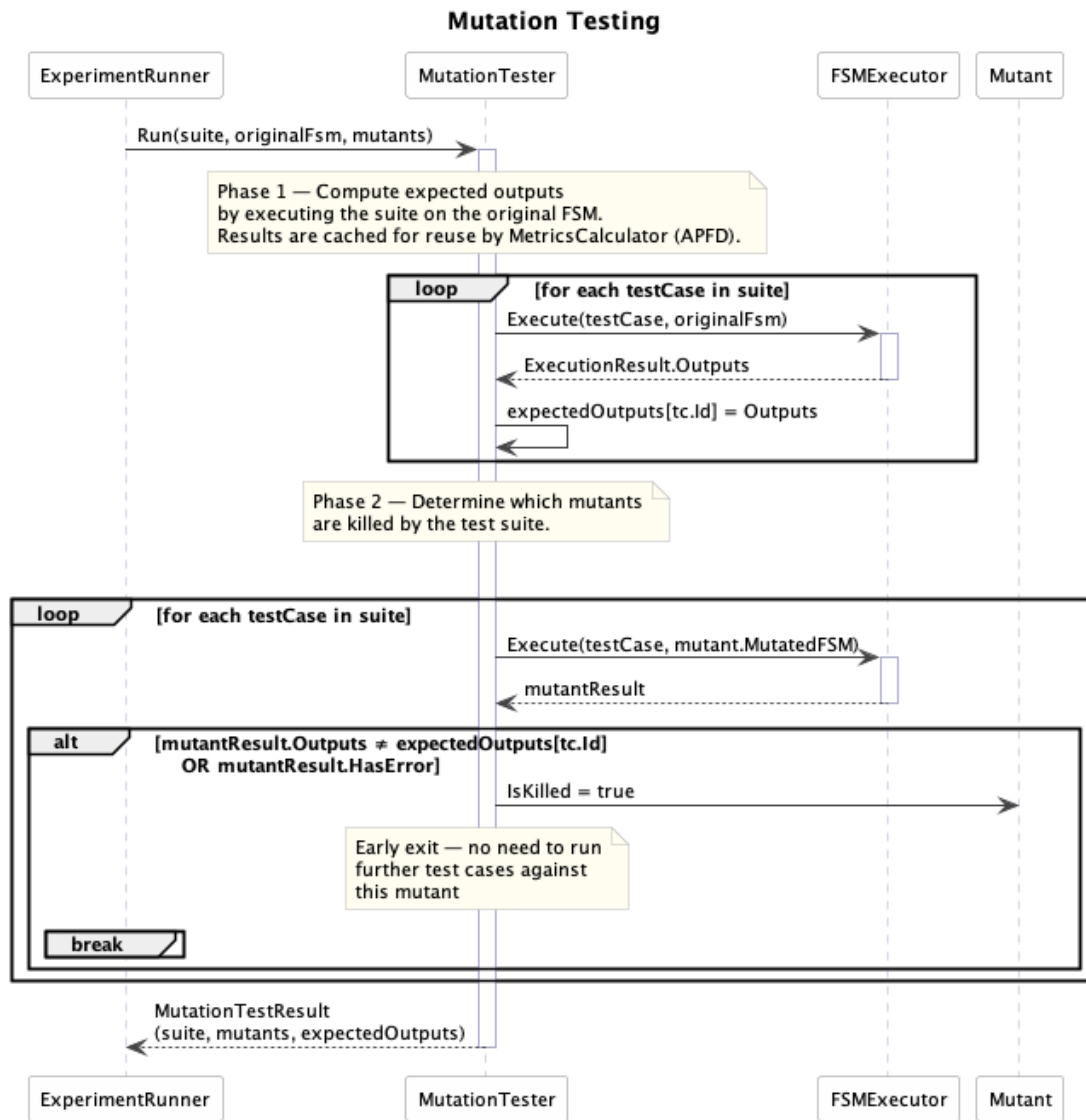


Figure H.1: Sequence diagram of the mutation testing process

Appendix I

Final Artefact Component Architecture

Figure I.1 shows the component architecture of the final artefact described in Section 5.1.1. The diagram illustrates the eight components introduced for the first time in the final artefact alongside the ten strategy implementations and six mutation operators carried over from the experimental prototype, and shows how all components are structured around the shared Core domain model.

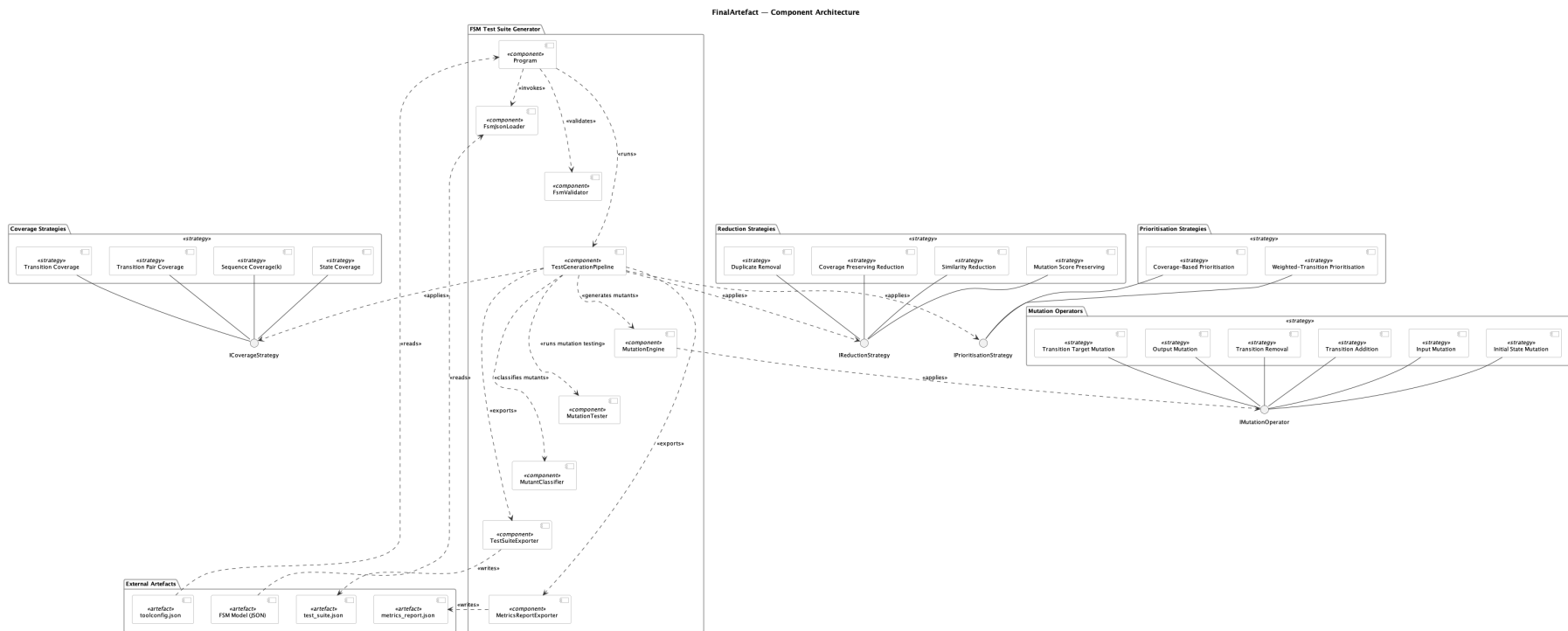


Figure I.1: Component architecture of the final artefact

Appendix J

Final Artefact — Startup and FSM Loading Sequence

Figure J.1 details the startup and FSM loading sequence of the final artefact, as described in Section 5.1. The diagram shows how `Program` resolves the configuration file path, deserialises `toolconfig.json` into a typed `ToolConfiguration` object, loads the FSM model via `FsmJsonLoader`, and passes control to `FsmValidator`, which performs five sequential checks before any generation work begins.

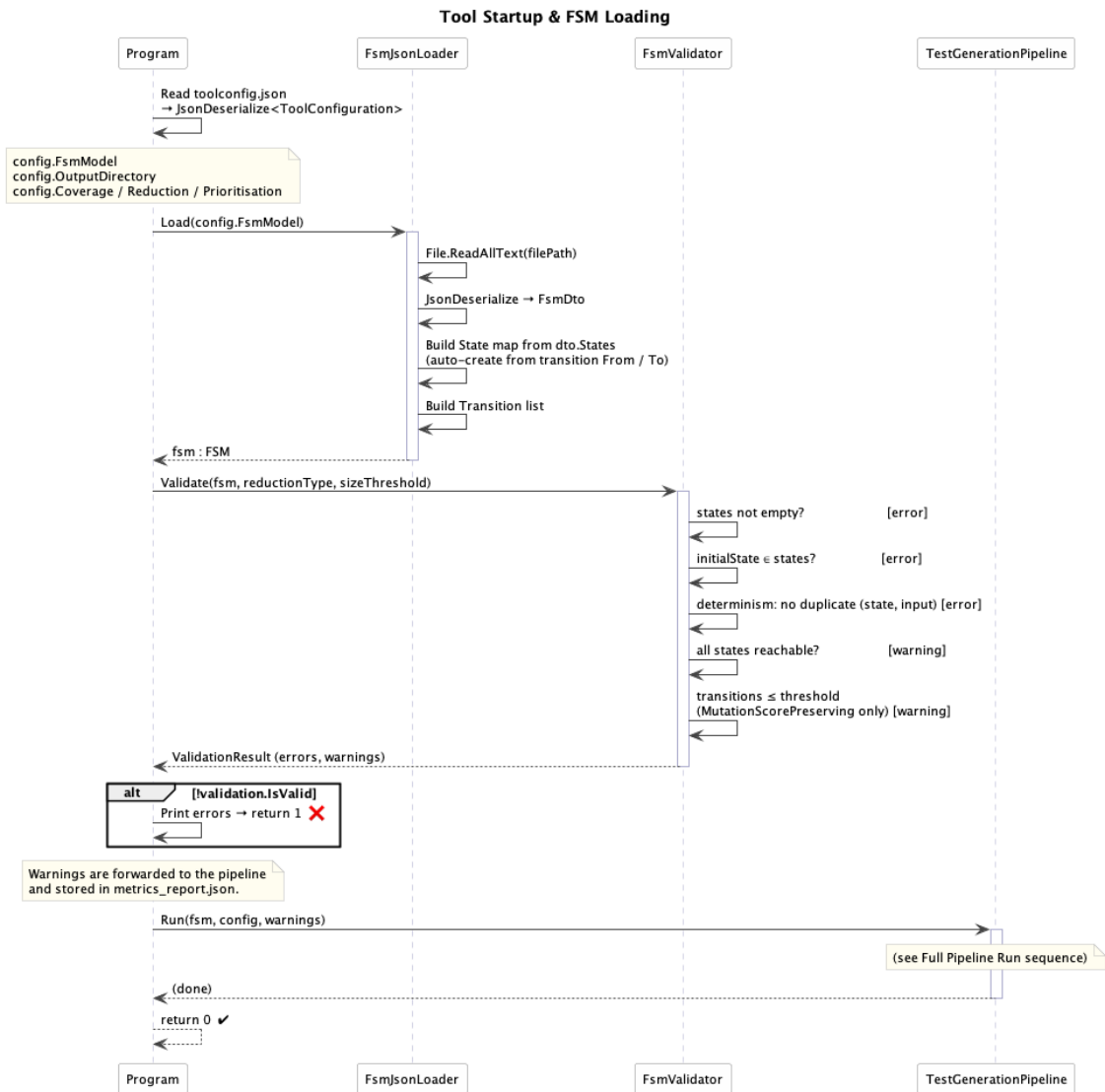


Figure J.1: Sequence diagram of the startup and FSM loading phase of the final artefact

Appendix K

Final Artefact — Full Pipeline Run Sequence

Figure K.1 shows the complete sequence of interactions across all participating components during a full pipeline run of the final artefact, as described in Section 5.1. The diagram covers the three generation stages (coverage, reduction, and prioritisation), the two mutation testing passes that capture pre and post-reduction mutation scores, the mutant classification pass, and the final export of `test_suite.json` and `metrics_report.json`.

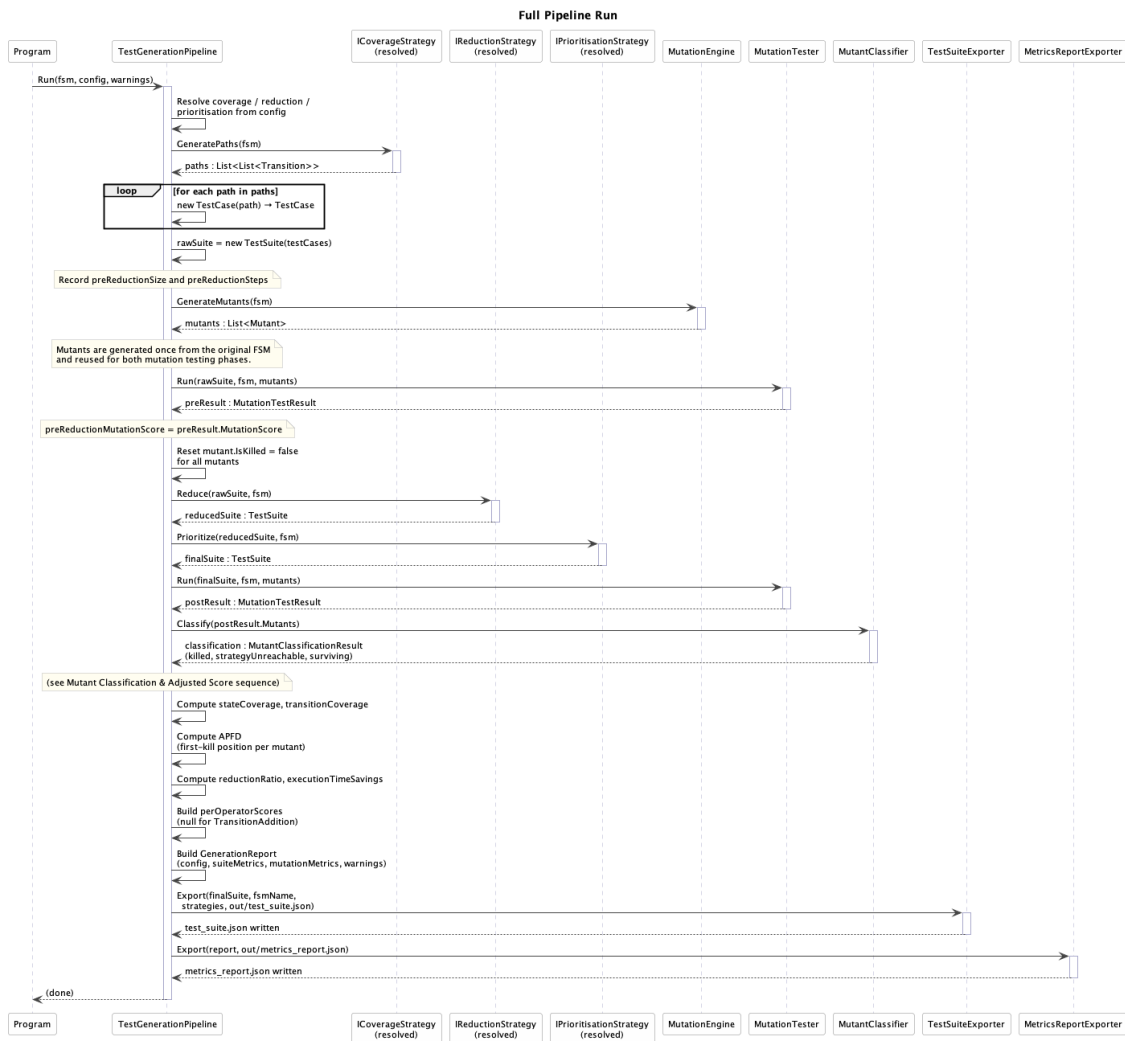


Figure K.1: Sequence diagram of a complete pipeline run of the final artefact

Appendix L

Final Artefact — Mutant Classification Sequence

Figure L.1 details the sequence of interactions within `MutantClassifier` during a classification pass, as described in Section 5.2. The diagram shows how the annotated mutant list is partitioned into three disjoint classes, how the *Transition Addition* operator check identifies strategy-unreachable mutants, and how the remaining killed and surviving counts are used to compute the adjusted mutation score.

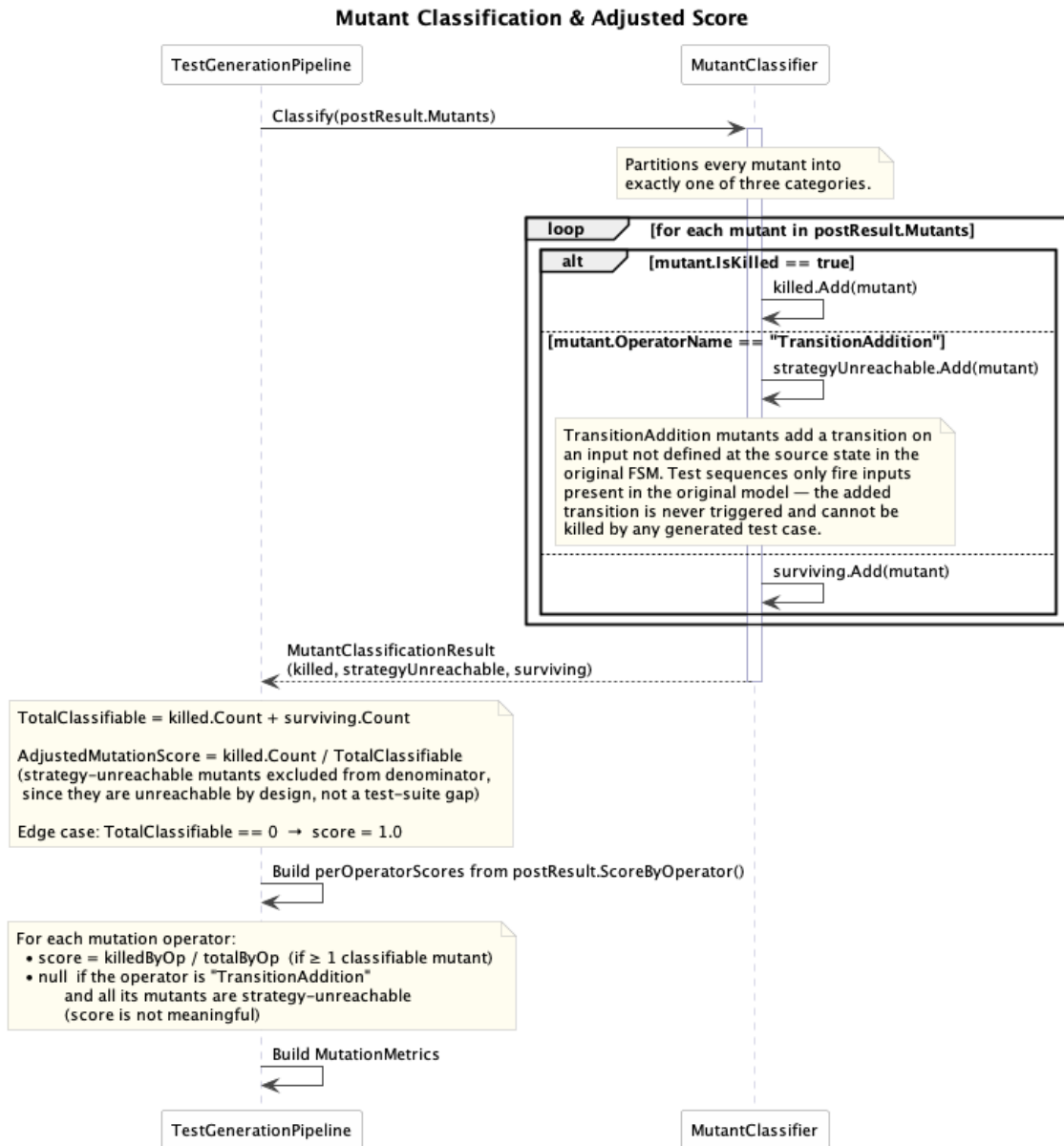


Figure L.1: Sequence diagram of the mutant classification process of the final artefact